



LCM Nav3D

True Volumetric 3D Navigation for Unreal Engine

User Manual

Version 3.4.1 • Unreal Engine 5.8.0

Generated from the documentation shipped inside the plugin.
Every chapter is also available as markdown in `Plugins/LCMNav3D/Documentation/`.

Contents

I	Getting Started	6
1	Tutorial 01: Installation	7
1.1	Before you start	7
1.2	Method A: Installed from Fab / Epic Games Launcher	7
1.3	Method B: Manual install into your project	7
1.4	Verify the install	8
1.5	The engine plugins it turns on	8
1.6	Common install problems	9
1.7	Other engine versions	9
2	Tutorial 02: Core Concepts	10
2.1	1. Why not just use Unreal's navmesh?	10
2.2	2. The Sparse Voxel Octree, in one picture	10
2.3	3. The one actor you must place	10
2.4	4. Finite vs Infinite: the decision that shapes everything	11
2.5	5. Resolution: the two settings people get wrong	11
2.6	6. How a path is actually found	12
2.7	7. Things that move	13
2.8	8. Terms you'll meet in the rest of the docs	13
2.9	Recap	13
3	Tutorial 03: Your First Flying Agent	14
3.1	What we're building	14
3.2	Step 1: Add the navigation manager	14
3.3	Step 2: Make the pawn	15
3.4	Step 3: Blackboard and Behavior Tree	15
3.5	Step 4: The AI Controller	18
3.6	Step 5: Fly	18
3.7	If it doesn't move	18
3.8	Step 6: Make it look good	19
3.9	Step 7: Several agents at once	19
3.10	Step 8: Chasing a moving target	19
3.11	Recap	20
4	Tutorial 04: The Same Agent in StateTree	21
4.1	Should you use StateTree or Behavior Tree?	21
4.2	Step 1: The pawn	21
4.3	Step 2: Create the StateTree asset	21
4.4	Step 3: Add the Fly To task	21
4.5	Step 4: The AI Controller	25
4.6	Step 5: Fly	25
4.7	The rest of the StateTree surface	25
4.8	Behaviour differences worth knowing	26
4.9	Troubleshooting	26
5	Tutorial 05: Dynamic Obstacles	27
5.1	Static vs dynamic	27
5.2	Step 1: Make a moving obstacle	27
5.3	Step 2: Add the Dynamic Obstacle component	27
5.4	Step 3: Move it	28

5.5	Step 4: See it work	28
5.6	How agents notice	28
5.7	Ready-made examples	28
5.8	The bundled Obstacle Mover	29
5.9	Performance notes	29
5.10	Troubleshooting	29
5.11	Where to go next	30
II	Going Further	31
6	Tutorial 06 - Mass Entity crowds (10K+ agents)	32
6.1	Prerequisites	32
6.2	Step 1: Create the agent config	32
6.3	Step 2: Spawn them	33
6.4	Step 3: Give them a goal	33
6.5	Step 4: Check it works	34
6.6	If nothing moves	34
6.7	Tuning	34
6.8	Scope	34
6.9	Related	34
7	Tutorial 07 - Authoring true-3D tactical EQS	35
7.1	The LCM Nav3D EQS node catalog	35
7.2	Step 1 - Author a sniper-perch query	35
7.3	Step 2 - The EQS Testing Pawn	36
7.4	Step 3 - Wire the result into a Behavior Tree	36
7.5	The five shipped example queries	37
7.6	Related	37
8	Tutorial 08 - Perception-driven tactical AI	38
8.1	Why this exists	38
8.2	Prerequisites	38
8.3	Part A - UActor agent (BT / StateTree)	38
8.4	Part B - Mass crowd variant	39
8.5	What you build by hand	39
8.6	Related	39
9	Tutorial 09 - Hybrid LCM Nav3D + Recast crowd avoidance	41
9.1	Prerequisites	41
9.2	Step 1 - Add the bridge to your LCM Nav3D pawn	41
9.3	Step 2 - Add Recast crowd agents (existing UE5 pattern)	41
9.4	Step 3 - Play	41
9.5	How the Z-band filter works	42
9.6	Tuning <code>r.LcmNav.Crowd.ZBandHeightCm</code>	42
9.7	Stacked multi-floor scenarios	42
9.8	Performance notes	42
9.9	Acceptance test (manual)	42
9.10	What about the LCM Nav3D pawn's own avoidance?	43
9.11	Mass variant	43
9.12	Related LCM Nav3D surfaces	43
10	Tutorial 10 - Cost areas, query filters and 3D NavLinks	44
10.1	What an area does	44
10.2	Step 1 - Place a cost area	44
10.3	Step 2 - Give an agent a filter	45
10.4	Step 3 - See what an agent sees	45
10.5	What happens at the edges	45
10.6	Step 4 - Query costs yourself	46

10.7	3D NavLinks	46
10.8	Try it	46
10.9	Acceptance test (manual)	46
11	Tutorial 11 - LCMNav3D in the Gameplay Debugger	47
11.1	What it shows	47
11.2	How to toggle	48
11.3	What survives PIE → editor → PIE reloads	48
11.4	Adding new fields (v2 design hook)	48
11.5	Performance	48
11.6	Related LCM Nav3D debug surfaces	49
12	Tutorial 12 - Small worlds (centimetre-scale navigation)	50
12.1	The two rules that are not obvious	50
12.2	Option A - build at 100:1 (works on every release)	50
12.3	Option B - build at 1:1 with World Scale = Small	50
12.4	Stones with caves: collision, not the mesh	51
12.5	Props that must not be solid	51
12.6	Mass crowds at small scale	51
12.7	The demo tanks	51
12.8	What to expect at small scale	51
13	Tutorial 13 - LCM Fly To Location (no Behavior Tree, no StateTree)	52
13.1	What you need	52
13.2	Using the node	52
13.3	Stopping and replacing	53
13.4	How arrival is judged	53
13.5	One Fly To, not three	53
13.6	From C++	53
13.7	Verified	54
14	Tutorial 14 - Stock Move To and EQS with LCM Nav3D	55
14.1	How it works	55
14.2	Step 1: pick your setup	55
14.3	Step 2: add a Supported Agent (mixed projects only)	55
14.4	Step 3: check it worked	55
14.5	Cost areas with the stock nodes	56
14.6	Limits	56
III	Behaviour Tree Stock Patterns	57
15	BT Wait - use stock UBTTask_Wait	58
15.1	Pattern	58
15.2	Why we didn't ship LCM_BTTask_Wait	58
15.3	LCM Nav3D-specific wait variant	58
16	BT IsAtLocation - use the stock UE5 decorator	59
16.1	Why we didn't ship LCM_BTDecorator_IsAtLocation	59
16.2	Three-step usage on an LCM Nav3D pawn	59
16.3	When to use the path-finding variant	59
16.4	Related LCM Nav3D-unique decorators	59
17	BT LookAt - choosing between the stock task and the LCM Nav3D one	61
17.1	Which one to use	61
17.2	The stock task, and what it does with the vertical axis	61
17.3	The LCM Nav3D tasks	61
17.4	Pattern: face a target, then act	62
17.5	Pattern: keep facing while doing other things	62
17.6	On 6-DOF flight bodies	62

17.7	See also	62
18	BT TurnInPlace - constant-rate turning, and the stock focus pattern	63
18.1	Pattern: continuous facing during other tasks	63
18.2	Pattern: a discrete constant-rate turn	63
18.3	Property list	63
18.4	On 6-DOF flight bodies	64
19	BT HealthBelow - use the stock blackboard + decorator pattern	65
19.1	Pattern	65
19.2	If you want a GameplayTag-based pattern	65
19.3	Why we didn't ship LCM_BTDecorator_HealthBelow	65
19.4	Related LCM Nav3D-unique decorators	65
20	BT PerceptionRelay - bind OnTargetPerceptionUpdated directly	66
20.1	Pattern (in your AIController subclass)	66
20.2	Why we didn't ship LCM_BTService_PerceptionRelay	66
20.3	Memory pattern (last-known-location with age)	67
20.4	Related LCM Nav3D-unique services	67
21	BT MemoryDecay - use FActorPerceptionInfo::GetLastStimulusLocation(&OutAge)	68
21.1	Pattern (in your AIController or a small custom service)	68
21.2	How to integrate as a BT service	68
21.3	Why we didn't ship LCM_BTService_MemoryDecay	69
21.4	Related LCM Nav3D-unique services	69
IV	Reference	70
22	LCM Nav3D - Editor Tools	71
22.1	Quick start	71
22.2	The Welcome screen	72
22.3	The Manager panel	73
22.4	Bake Volumes	76
22.5	Setup Wizard	78
22.6	Health Check	80
22.7	Auto-Tuner	81
22.8	Project Settings	86
22.9	Keyboard shortcuts	87
22.10	Where things are configured	87
23	Settings Reference: LcmNavigationManagerSVO	88
23.1	The panel at a glance	88
23.2	1. Setup	90
23.3	2. Voxels	91
23.4	3. Collision	94
23.5	4. Performance	94
23.6	5. Coarse Skeleton (optional)	95
23.7	6. Debug	96
23.8	7. Runtime State	96
23.9	Build & Tools (Blueprint-callable)	96
23.10	Per-agent settings	97
23.11	Console variables new in 3.4	99
23.12	Recommended starting points	99
24	Troubleshooting	101
24.1	Quick triage	101
24.2	The two diagnostics that answer almost everything	101
24.3	1. Nothing moves	101
24.4	2. Agent moves, but badly	102

24.5	3. "No path found"	102
24.6	4. Agents ignore geometry	103
24.7	5. Dynamic obstacles do nothing	103
24.8	6. Performance	103
24.9	7. Editor and install	104
24.10	8. Determinism and multiplayer	104
24.11	Still stuck?	104
25	LCM Nav3D - Realistic Flight/Swim Locomotion Reference	106
25.1	Concept - Guidance → Control → Dynamics	106
25.2	Quick start	106
25.3	Debugging the flight path - planned vs predicted vs flown	107
25.4	Archetypes (key <code>FlightConfig</code> params)	107
25.5	Troubleshooting - symptom to cause	110
25.6	Agent scale - flying a body in a centimetre world	111
25.7	Animation contract - <code>FLcmFlightState</code>	112
25.8	Mass crowds & scale (P7-P8)	112
25.9	Determinism	112
25.10	Bird and Fish do not share the drone control cascade	113
25.11	Fish schooling on pawns	113
25.12	Hard turns: progress does not depend on hitting a sphere	113
26	LCM Nav3D - Mass Entity Trait Reference	118
26.1	The anatomy of a config	118
26.2	The traits	119
26.3	Combination matrix	121
26.4	The editor validator	122
26.5	Migration from the old traits	122
26.6	World Partition / large-world (City Sample scale) - REQUIRED READING	122
26.7	Useful CVars	123
V	Appendix	124
27	LCM Nav3D - API Reference	125
27.1	Classes	125

Part I

Getting Started

CHAPTER 1

Tutorial 01: Installation

TL;DR: Put the plugin in `YourProject/Plugins/LCMNav3D/`, open the project and let it compile, enable **Show Plugin Content** in the Content Browser, then open a demo map and press Play. If the demo map works, your install is good.

Time: ~10 minutes (plus one compile).

1.1 Before you start

Requirement	Detail
Unreal Engine	5.8
Platform	Windows 64-bit or Linux
Project type	C++ project for a manual install (see below)
Compiler	Visual Studio 2022 with the <i>Game development with C++</i> workload (Windows)

The plugin ships as source, so something has to compile it once. Which "something" depends on how you install it.

1.2 Method A: Installed from Fab / Epic Games Launcher

This is the simplest path and works with **Blueprint-only projects**.

1. Install the plugin from the Launcher's **Library** → **Fab Library** section. It lands in `<Engine>/Plugins/Marketplace/LCMNav3D/` and is already compiled for 5.8.
2. Open your project.
3. **Edit** → **Plugins**, search "LCM Nav3D", tick **Enabled** if it isn't already.
4. Restart the editor when prompted.

Skip to Verify the install.

1.3 Method B: Manual install into your project

Use this when you have the plugin as a folder (for example from a direct purchase or a version you re-targeted yourself).

1. Close the editor.
2. Copy the LCMNav3D folder into your project so the layout is:

```
YourProject/  
  YourProject.uproject  
  Plugins/  
    LCMNav3D/  
      LCMNav3D.uplugin
```

Source/
Content/
Config/
Shaders/

Create the `Plugins` folder if your project doesn't have one. The folder must be named exactly `LCMNav3D`, because the content mount `/LCMNav3D/` depends on it.

3. If your project is **Blueprint-only**, convert it to **C++ first**. A Blueprint-only project has no build pipeline and cannot compile a source plugin. The easiest conversion: **Tools** → **New C++ Class** → **None** → **Create Class**. The editor generates a `Source/` folder and restarts. You never have to write any C++ afterwards.
4. Right-click `YourProject.uproject` → **Generate Visual Studio project files**.
5. Open the generated `.sln` and build **Development Editor | Win64**, or open the `.uproject` and click **Yes** when the editor offers to rebuild.
6. The plugin is `EnabledByDefault`, so it turns itself on. Confirm under **Edit** → **Plugins**.

1.4 Verify the install

Do all three of these. Each one catches a different kind of failure.

1.4.1 1. The plugin loaded

Edit → **Plugins** → **AI Navigation** category. You should see **LCM Nav3D**, enabled, version 3.4.1.

1.4.2 2. You can see the plugin's content

Plugin content is hidden by default in the Content Browser:

Content Browser → **Settings (the gear icon)** → tick **"Show Plugin Content"**.

A **LCM Nav3D Content** folder appears in the tree. Without this you will not find any demo map, and it is by far the most common "the plugin didn't install" false alarm.

1.4.3 3. A demo map actually runs

Open **LCM Nav3D Content** → **Demo** → **BehaviourTree** → **InfniteMaps** → **Vertical_Skyscraper_BT** and press **Play**.

You should see agents flying through the building: over floors, through gaps, not along the ground. Give it a second or two on first play: the navigation volume builds at `BeginPlay`.

Demo → **MainMenu** is a gallery of every demo in the plugin, if you would rather browse than open maps by name.

If that works, **your install is correct** and you can move on to Tutorial 02: Core Concepts.

1.5 The engine plugins it turns on

`LCMNav3D.uplugin` declares these as dependencies, so the engine enables them for you. You do not need to touch them, but it is worth knowing they are on:

Plugin	Why it's needed
<code>StateTree</code> , <code>GameplayStateTree</code>	The <code>StateTree</code> task/condition/evaluator set

Plugin	Why it's needed
MassEntity, MassGameplay, MassAI	The Mass crowd layer
SmartObjects	Smart Object behaviour definitions
DataValidation	In-editor validation of illegal Mass trait combinations

If the plugin fails to load with `GetLastError` 126, one of the Mass plugins got disabled. `MassGameplay` in particular is required at runtime. Re-enable it and restart.

1.6 Common install problems

"The following modules are missing or built with a different engine version: LCMNav3D" The plugin hasn't been compiled for your engine build. Click **Yes** to rebuild. If the rebuild fails, you are on a Blueprint-only project, so go back to Method B step 3.

I can't find the demo maps. You haven't enabled *Show Plugin Content*. See verification step 2.

The plugin is missing from the Plugins window entirely. The folder is in the wrong place or misnamed. It must be `YourProject/Plugins/LCMNav3D/LCMNav3D.uplugin`. Check you didn't end up with a doubled folder like `Plugins/LCMNav3D/LCMNav3D/`.

Compile errors mentioning `StateTree` or `Mass` types. An engine plugin dependency was disabled. Re-enable the table above and regenerate project files.

More in Troubleshooting.

1.7 Other engine versions

This copy targets **Unreal Engine 5.8**. The plugin ships as a separate copy for each supported engine version, **5.2 through 5.8**, and each one is built and verified against that engine.

Install the copy that matches your project rather than re-targeting this one. Engine APIs move between versions, and the per-version copies already carry those differences.

Next: Tutorial 02: Core Concepts. What the navigation manager is actually doing, and the three settings that decide whether it works well in your level.

CHAPTER 2

Tutorial 02: Core Concepts

TL;DR: LCM Nav3D carves your level's **empty air** into a tree of boxes (a Sparse Voxel Octree) and paths through the empty ones. You place one `LcmNavigationManagerSVO` actor per level, tell it how big your world is and how small the tightest gap is, and it does the rest.

Time: ~10 minutes reading. No project changes. This is the chapter that makes every later setting make sense.

2.1 1. Why not just use Unreal's navmesh?

Unreal's Recast navmesh is a **surface**. It answers "where can I stand?" by flattening your level onto walkable polygons. That is exactly right for a soldier and completely wrong for a dragon, because the interesting space for a flyer is the part with nothing in it.

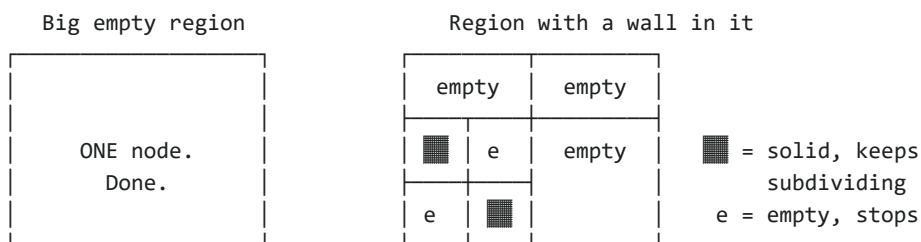
LCM Nav3D answers a different question: "**where is there empty air?**" It represents the volume, not the floor. That's the whole idea. Everything else is engineering around making that fast.

2.2 2. The Sparse Voxel Octree, in one picture

Imagine a giant cube around your level. Now:

1. Is the cube completely empty? **Yes** → done, one big box of free space.
2. **No**, something's in it? → cut it into 8 smaller cubes and ask each one the same question.
3. Repeat until the cubes reach your minimum size.

That's it. That's the octree.



"Sparse" is the important word. Open sky costs almost nothing: one node covers a huge volume. Detail only appears where geometry actually is. This is why a 2 km map doesn't cost 2 km worth of memory: most of it is air, and air is cheap.

What this means for you: the cost of your navigation data is driven by *how much clutter you have*, not by how big your level is.

2.3 3. The one actor you must place

`LcmNavigationManagerSVO`: drop one into your level. One per level, that's the rule.

It owns the octree, runs the pathfinder, and answers every query. Every agent talks to it. If it isn't in the level, nothing navigates.

Its Details panel is deliberately organised in the order you should think about it:

Category	What it's for
1. Setup	Finite or infinite world, the biggest decision
2. Voxels	Size and resolution
3. Collision	Which geometry counts as an obstacle
4. Performance	Frame-budget knobs
5. Debug	Visualisation
6. Runtime State	Read-only live info

Full field-by-field detail is in the Settings Reference. Here we only cover the concepts behind them.

2.4 4. Finite vs Infinite: the decision that shapes everything

This is **1. Setup** → **Infinite World**, and it changes which other settings even appear.

2.4.1 Finite (default, Infinite World unticked)

One fixed box of navigation, built once. You set **WorldExtent** (default **10000 cm** = a 200 m cube, since extent is measured from the centre outward).

- Built at **BeginPlay** if **Auto Build On Play** is on (it is by default).
- Simple, fast, completely predictable.
- **Use this for:** arenas, levels, dungeons, single buildings, anything with a boundary.

Start here. Most projects never need anything else.

2.4.2 Infinite (Infinite World ticked)

The world is divided into **chunks** of **ChunkSize** (default **8000 cm**). Chunks are generated around your agents as they move and thrown away behind them.

- **Use this for:** open worlds, streaming levels, procedural terrain, anything without a boundary.
- Costs more complexity: chunks must generate before an agent can route through them.

ChunkSize is not a free dial. It decides both memory *and* whether distant routes can be found at all. **8000 is the tested default. Change it only with a reason and re-test.** Larger chunks mean fewer, heavier generations; smaller means more, lighter ones, and more seams to cross.

2.5 5. Resolution: the two settings people get wrong

2.5.1 MinVoxelSize (default 100 cm)

How small the smallest box is allowed to get. **This is the single biggest driver of both quality and cost.**

- **Smaller** → finds tighter gaps, uses more memory, takes longer to build.
- **Larger** → cheaper and faster, but narrow openings vanish and agents route around them.

⚠ **It's a floor, not an exact size.** The octree halves its way down, so the real leaf size is the world (or chunk) size repeatedly divided by two until it's about your **MinVoxelSize**. Two consequences that surprise people:

- Changing **MinVoxelSize** from 100 to 90 may change **nothing** (same number of halvings).
- Changing it from 100 to 60 may cross a threshold and **double your memory** in one step.

Tune it by testing, not by assuming the number is literal.

Rule of thumb: set `MinVoxelSize` to roughly **half the narrowest gap** your agents must fly through. A 3 m window needs about 150 cm or finer.

2.5.2 ClearancePadding (default 100 cm)

Inflates obstacles by this much when voxelizing, so agents don't clip walls.

⚠ **The classic mistake: setting this too high seals your level.** If padding approaches the size of your voxels, doorways and corridors fill in completely and the pathfinder reports "no route" through an opening you can plainly see. **Keep `ClearancePadding` well below `MinVoxelSize`.** If you need more clearance than that, lower `MinVoxelSize` too.

If agents refuse to path through an opening, this setting is the first thing to check.

2.6 6. How a path is actually found

For a short hop, it's a straight search through the octree. For anything longer, there are two levels:

1. MACRO: "which chunks/regions do I cross?" (coarse, cheap, long-range)
↓
2. MICRO: "which voxels inside them?" (fine, detailed, short-range)
↓
3. SMOOTH: turn the blocky voxel path into a nice line

You don't call these yourself; it's automatic. The reason to know is that it explains a common symptom: if a long route fails but a short one works, the problem is usually **macro**. Either chunks are not generated yet, or `ChunkSize` is interacting badly with your layout.

2.6.1 Choosing a solver

Solver	Character	Use when
A* (<i>Fastest, Grid Locked</i>)	Cheapest; paths follow voxel edges, so turns look square	You'll smooth the result anyway, or you need max throughput
Theta* (<i>Accurate, Any Angle</i>)	Most direct; cuts corners properly	Quality matters more than cost
Lazy Theta* (<i>Balanced</i>)	Nearly Theta* quality, near A* cost	The sensible default

2.6.2 Smoothing

Mode	Result
Raw Path	Straight out of the solver, blocky
Linear Shortcut	Removes redundant waypoints; straight segments
Curved Spline	Smooth curves: what you want for flying things

Smoothing runs *after* pathfinding and is validated against geometry, so a smoothed path won't cut through a wall.

2.6.3 Where the work runs

Path solving runs on the CPU, on a worker pool, off the game thread. It is deterministic: same input, same output, every time - which is what makes replays, lockstep multiplayer and automated tests reproducible.

The GPU is used for **voxelization**: turning your geometry into the octree, where it is substantially faster. If no suitable GPU is present it falls back to the CPU automatically, so nothing breaks.

2.7 7. Things that move

Static geometry is baked into the octree. For things that move, add a `LcmDynamicObstacleComponent` to the actor. It restamps the octree as the actor moves, and agents reroute around it. That's Tutorial 05.

2.8 8. Terms you'll meet in the rest of the docs

Term	Meaning
SVO	Sparse Voxel Octree, the navigation data
Voxel / node	One box in the octree; a leaf is a smallest one
Chunk	One tile of an infinite world
Portal	A connection between two chunks: how routes cross a seam
Macro / micro	Coarse long-range vs fine short-range pathfinding
Clearance	How much room an agent needs to fit
Flow field	One shared steering field many agents sample (for crowds)

2.9 Recap

- The octree stores **empty space**, and empty space is cheap.
- One `LcmNavigationManagerSVO` per level.
- **Finite** unless you genuinely have an open world.
- `MinVoxelSize` $\approx$ half your tightest gap; it snaps to powers of two.
- `ClearancePadding` below `MinVoxelSize`, or you'll seal your own level.
- **Lazy Theta*** + **Curved Spline** is a good default for a flyer.

Next: Tutorial 03: Your First Flying Agent. Build a working agent in your own level.

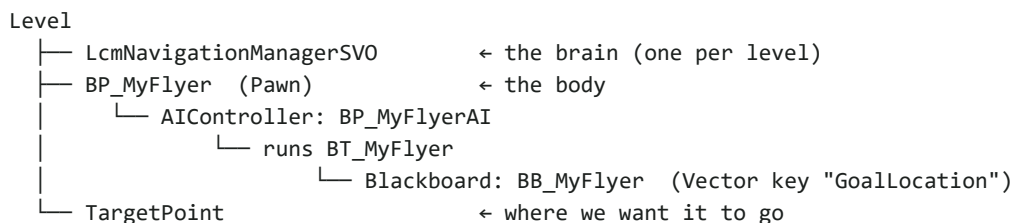
CHAPTER 3

Tutorial 03: Your First Flying Agent

TL;DR: Three ingredients: an `LcmNavigationManagerSVO` in the level, a **pawn with an AIController**, and a Behavior Tree whose only task is `LCM Fly To (SVO)` pointed at a blackboard key. No C++.

Time: ~20 minutes. **You'll end with:** a pawn that flies through 3D space to a goal in your own level.

3.1 What we're building



3.2 Step 1: Add the navigation manager

1. In the **Place Actors** panel, search `LcmNavigationManagerSVO`.
2. Drag one into your level. Position doesn't matter much. It uses its own location as the centre of the navigation volume, so put it roughly in the middle of your playable space.
3. Select it and set, in the Details panel:

Category	Field	Value
1. Setup	Infinite World	unticked (finite, see Tutorial 02)
1. Setup	Auto Build On Play	ticked
2. Voxels	World Extent	large enough to cover your play space (default 10000 = 200 m cube)
2. Voxels	Min Voxel Size	100 to start
2. Voxels	Clearance Padding	100 to start

World Extent measures **outward from the actor**, so 10000 gives you a 200 m cube centred on the manager. If your agents will fly outside that box, increase it: outside the volume there is no navigation data and paths will fail.

3.2.1 See what it built

Tick 5. **Debug** → **Draw Debug Volumes** and press Play. You'll see the octree drawn over your level: lots of big boxes in open air, small ones packed around geometry. **This is the single most useful thing you can do while tuning.** If a doorway you care about shows no small voxels, the pathfinder can't see it either.

Turn it back off when you're done; it's expensive to draw.

3.3 Step 2: Make the pawn

You need something that can move in 3D without gravity.

1. **Content Browser** → **Add** → **Blueprint Class** → **Pawn**. Name it **BP_MyFlyer**.
2. Open it and add a **Floating Pawn Movement** component. (Add Component → search "Floating Pawn Movement".) This is what makes it fly rather than fall.
3. Select **FloatingPawnMovement** and set **Max Speed** to **600**. Match this to the task's **Flight Speed** later.
4. Add a **Static Mesh** component so you can see it. A cone or sphere is fine. Scale it down to something sensible.
5. In **Class Defaults**, set **Auto Possess AI** to **Placed in World or Spawned**.

Collision tip. For your first test, set the mesh's collision to **No Collision**. Physics fighting the navigation is a very common source of "my agent jitters / gets stuck", and you want a clean first result. Add collision back once flying works.

3.4 Step 3: Blackboard and Behavior Tree

3.4.1 The blackboard

1. **Add** → **Artificial Intelligence** → **Blackboard**. Name it **BB_MyFlyer**.
2. Add one key:
 - **Type: Vector, Name: GoalLocation**

The Fly To task also accepts an **Object** key holding an Actor. Use a Vector for now; the Actor form is what you'd use for chasing, covered at the end.

3.4.2 The behavior tree

1. **Add** → **Artificial Intelligence** → **Behavior Tree**. Name it **BT_MyFlyer**.
2. Open it, and in the Details panel set **Blackboard Asset** to **BB_MyFlyer**.
3. Drag from **ROOT** and add a **Sequence**.
4. Drag from the Sequence and add the task named **Fly To**; it appears in the tree as **LCM Fly To (SV0)**.
5. Select that task and set:

Field	Value
Blackboard Key	GoalLocation
Acceptance Radius	100
Flight Speed	600

Leave everything else at defaults for now.

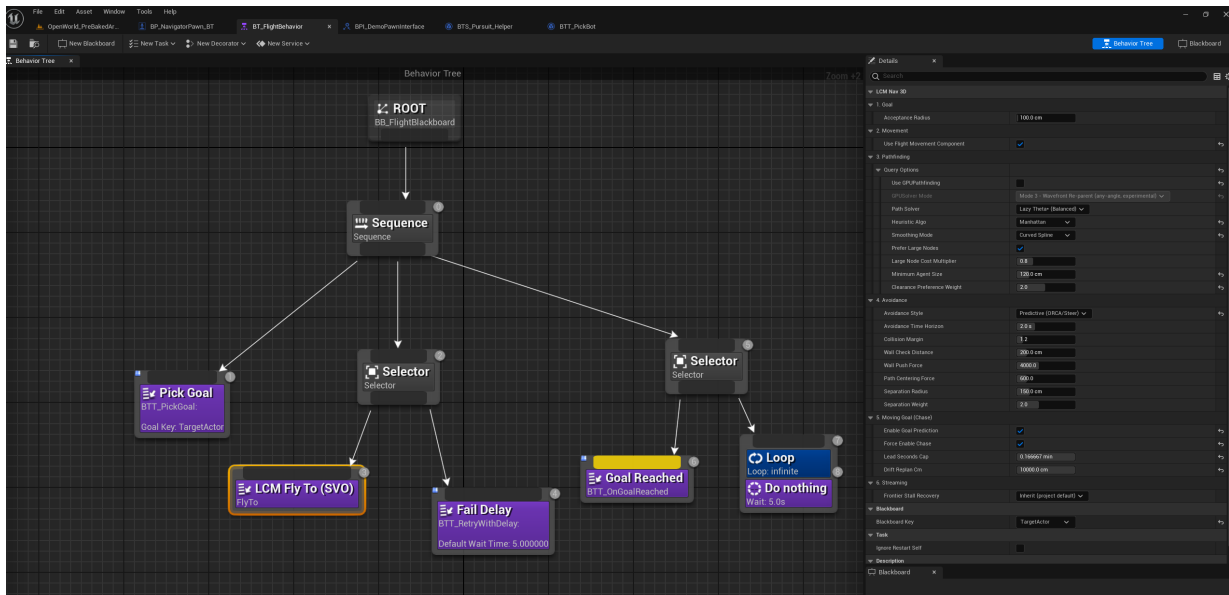


Figure 03.1 - Where LCM Fly To (SVO) sits. This is the shipped BT_FlightBehavior demo, one step past what you have just built: the same Fly To task, with a goal picker in front of it and a retry-with-delay branch beside it so a failed plan does not kill the tree. Yours needs only the highlighted node to start flying.

3.4.3 The FlyTo settings, in full

You will not need most of these on day one, but it is worth knowing the shape of the surface before you go looking for a specific knob.

The screenshot shows the LCM Nav 3D configuration window. At the top, there are tabs for 'Behavior Tree' and 'Blackboard'. Below is a search bar and a list of sections. The sections are:

- 1. Goal**: Acceptance Radius (100.0 cm)
- 2. Movement**: Use Flight Movement Component (checked)
- 3. Pathfinding**:
 - Query Options: Use GPUPathfinding (unchecked)
 - GPUSolver Mode: Mode 3 - Wavefront Re-parent (any-angle, experimental)
 - Path Solver: Lazy Theta* (Balanced)
 - Heuristic Algo: Manhattan
 - Smoothing Mode: Curved Spline
 - Prefer Large Nodes (checked)
 - Large Node Cost Multiplier: 0.8
 - Minimum Agent Size: 120.0 cm
 - Clearance Preference Weight: 2.0
- 4. Avoidance**:
 - Avoidance Style: Predictive (ORCA/Steer)
 - Avoidance Time Horizon: 2.0 s
 - Collision Margin: 1.2
 - Wall Check Distance: 200.0 cm
 - Wall Push Force: 4000.0
 - Path Centering Force: 600.0
 - Separation Radius: 150.0 cm
 - Separation Weight: 2.0
- 5. Moving Goal (Chase)**:
 - Enable Goal Prediction (checked)
 - Force Enable Chase (checked)
 - Lead Seconds Cap: 0.166667 min
 - Drift Replan Cm: 10000.0 cm
- 6. Streaming**:
 - Frontier Stall Recovery: Inherit (project default)
- Blackboard**:
 - Blackboard Key: TargetActor
- Task**:
 - Ignore Restart Self (unchecked)

Figure 03.2 - The LCM Fly To task, all six sections. Read it as a pipeline: **1 Goal** is where you are going, **2 Movement** is what flies you there, **3 Pathfinding** is how the route is computed, **4 Avoidance** is how you get past other agents and walls, **5 Moving Goal** is only for chasing something that moves, and **6 Streaming** matters only in an infinite world.

Two fields decide the shape of everything below them:

- **Use Flight Movement Component** - ticked, the whole of section 2 collapses, because the movement component owns speed and dynamics instead of the task.
- **Avoidance Style** - set to CBS, the seven tuning fields under it hide; CBS is a planner-level solver and does not use them.

Sections you have not configured are not silently active. The panel hides any property the current configuration will ignore, so what you can see is what will run.

3.5 Step 4: The AI Controller

1. Add → Blueprint Class → **AIController**. Name it **BP_MyFlyerAI**.
2. Open it. On the **Event BeginPlay** node:
 - Add **Run Behavior Tree**, set **BTAsset** to **BT_MyFlyer**.
3. We need to put a goal in the blackboard. Still on **BeginPlay**, **after** **Run Behavior Tree**:
 - Add **Get Blackboard → Set Value as Vector**.
 - **Key Name:** **GoalLocation**
 - **Value:** for a first test, drag in a **Get Actor Of Class** node with **Actor Class = TargetPoint**, then **GetActorLocation** from it.

```
BeginPlay → Run Behavior Tree (BT_MyFlyer)
           ↳ Set Value as Vector (Key: GoalLocation,
                                   Value: GetActorOfClass(TargetPoint).GetActorLocation)
```

4. Compile and save.
5. Go back to **BP_MyFlyer** → **Class Defaults** → set **AI Controller Class** to **BP_MyFlyerAI**.

3.6 Step 5: Fly

1. Drag a **TargetPoint** into your level, somewhere your flyer has to climb or route around geometry to reach. Put it up high: that's the whole point.
2. Drag **BP_MyFlyer** into the level, somewhere with open air around it.
3. Press **Play**.

Your pawn should take off and fly to the target point, routing around geometry in full 3D.

3.7 If it doesn't move

Work down this list in order. These are the actual causes, in the order they occur.

Symptom	Cause	Fix
Output Log: "No SVO Manager found in the level!"	No manager actor	Step 1
Nothing happens, no log errors	AI never possessed the pawn	Auto Possess AI = <i>Placed in World or Spawned</i> , and AI Controller Class is set
Task fails instantly	Goal is outside the navigation volume	Increase World Extent, or move the target inside it
Task fails instantly	Goal (or start) is inside geometry	Move them into open air
Agent flies but ignores walls	Draw Debug Volumes shows no detail	Min Voxel Size too large
Agent refuses an obvious doorway	Opening sealed by padding	Lower Clearance Padding (see Tutorial 02 §5)
Agent jitters or sticks	Physics fighting navigation	Set mesh collision to No Collision for the test

Full list: Troubleshooting.

3.8 Step 6: Make it look good

Defaults are functional, not beautiful. Two changes transform the motion:

On the LCM Fly To (SVO) task:

Field	Try	Effect
Flight Style	Physics (Drones/Ships)	Momentum and inertia instead of instant direction changes. <i>(This is already the default.)</i>
Flight Style	Linear (Biological/Magic)	Instant, weightless. Good for spirits, insects, magic
Banking Amount	4.0	Rolls into turns like an aircraft
Max Banking Angle	45	How far it's allowed to roll
Deceleration Distance	800	Starts slowing this far out instead of stopping dead
Max Acceleration	2000	Lower = heavier, more sluggish feel

Path shape: a raw path is blocky. In Tutorial 02 we met the smoothing modes; **Curved Spline** is what makes flight look natural rather than like a series of ruler lines.

3.9 Step 7: Several agents at once

Duplicate BP_MyFlyer a few times and press Play. They'll fly the same route and crowd each other. Two settings fix that, both on the Fly To task:

Field	Default	Purpose
Separation Radius	150	How close before they push apart
Separation Weight	2.0	How hard they push

And **Avoidance Style**:

Style	Behaviour
None (Ghost/Ignore)	They fly through each other
Reactive (Boids/Push)	Push apart on contact: cheap, and the default
Predictive (ORCA/Steer)	Anticipate and steer around <i>before</i> colliding: smoother, costlier
Context-Based Steering (SVO)	Geometry-aware steering

For a handful of agents, **Reactive** is fine. For dozens sharing a corridor, **Predictive** looks markedly better.

Past a few hundred agents, stop adding pawns and move to Tutorial 06: Mass Entity.

3.10 Step 8: Chasing a moving target

To chase something instead of flying to a fixed point:

1. Change the blackboard key `GoalLocation` to type **Object**, with **Base Class = Actor**.
2. Set it to the actor you want chased.
3. On the Fly To task, tick **Enable Goal Prediction**.

With prediction on, the agent aims where the target *will be* rather than where it is, so it intercepts instead of trailing. `Lead Seconds Cap` (default 2.0) limits how far ahead it will extrapolate, and `Drift Replan Cm` (default 400) controls how far the target must move before the path is recomputed.

3.11 Recap

- One manager, finite world, sensible world extent.
- Pawn + Floating Pawn Movement + Auto Possess AI.
- Blackboard key → LCM Fly To (sv0) → done.
- Draw Debug Volumes is your best debugging tool.
- Tune feel with Flight Style, banking and deceleration; tune crowds with avoidance and separation.

Next: Tutorial 04: The Same Agent in StateTree, or jump to Tutorial 05: Dynamic Obstacles to make the world move.

CHAPTER 4

Tutorial 04: The Same Agent in StateTree

TL;DR: Identical flight behaviour to Tutorial 03, authored in StateTree instead of a Behavior Tree. Swap the AIController's RunBehaviorTree for a **StateTreeAIComponent**, and bind the **Fly To** task's TargetLocation instead of a blackboard key.

Time: ~15 minutes. **Do Tutorial 03 first.** This chapter only covers the differences.

4.1 Should you use StateTree or Behavior Tree?

Both are fully supported and produce the same flight. Pick on authoring preference, not capability.

	Behavior Tree	StateTree
Goal comes from	A blackboard key	Bound instance data on the task
Best at	Reactive priority ordering	Explicit states and transitions
Maturity	Stable	Epic's StateTree plugin is experimental on 5.2, stable from 5.3

The 5.2 caveat. Epic's StateTree plugin is experimental on 5.2. If you're shipping on 5.2 and want the most conservative option, use the Behavior Tree path. On 5.3+ this distinction disappears.

4.2 Step 1: The pawn

Exactly as Tutorial 03 Step 2: a Pawn with **Floating Pawn Movement**, a visible mesh, and **Auto Possess AI** = **Placed in World or Spawned**.

Nothing StateTree-specific here.

4.3 Step 2: Create the StateTree asset

1. **Content Browser** → **Add** → **Artificial Intelligence** → **StateTree**.
2. When it asks for a **schema**, choose **StateTree AI Component Schema**. This matters: the AI schema is what exposes the AI context (controller and pawn) the Fly To task needs.
3. Name it **ST_MyFlyer**.

4.4 Step 3: Add the Fly To task

1. Open **ST_MyFlyer**. You'll have a **Root** state.
2. Add a child state and name it **FlyToGoal**.
3. With that state selected, in the **Tasks** section click **+** and choose **Fly To** (listed under the **LCM Nav3D** category).
4. Set the task's properties:

Field	Value	Notes
Target Location	your goal	See binding below
Acceptance Radius	100	
Flight Speed	600	Match your Floating Pawn Movement Max Speed

4.4.1 Binding the goal

This is *the* structural difference from Behavior Trees. There's no blackboard; you bind the task's Target Location to a value.

The simplest approach is a **StateTree** parameter:

1. Select the **Root** state → in **Parameters**, add a parameter: **Name GoalLocation, Type Vector**.
2. Select the **Fly To** task → click the bind icon next to **Target Location** → choose **GoalLocation**.

Now anything that sets that parameter drives the agent.

Chasing instead. The task also has a **Target Actor** field. Set or bind that and the agent chases the actor, using the same prediction logic as the BT version. Note the ST chase path requires Target Actor to be non-null. If you want a fixed point, use Target Location.

4.4.2 Why Fly To has its own Target Actor field

This trips up everyone arriving from Behavior Trees, so it is worth stating plainly: **that field is not an alternative to Find Actor By Tag - it is what Find Actor By Tag plugs into.**

A Behavior Tree has a blackboard, so a BT task can name a key and read whatever wrote it. StateTree has no blackboard. Data moves by **property binding** from one node's output to another node's input, which means the *consuming* node must declare a field to receive the value. Delete Target Actor and there is nothing left to bind to.

This is the engine's own convention, not a plugin invention. Epic's Move To task carries exactly the same pair - a Destination vector plus an optional Target Actor, with the actor winning when both are set.

Fly To needs the live actor rather than a resolved position because it re-samples the target's location *and velocity* every tick to fly a lead intercept. A one-shot position would send the agent to where the target was when the state began.

4.4.3 Resolve the actor with the evaluator, not the task

Two nodes resolve a tag. Pick by whether you want a snapshot or a live reference:

	Find Actor By Tag (task)	Find Actor By Tag (evaluator)
Resolves	Once, on state entry	Continuously, on Refresh Interval
Found Actor Location	Frozen at entry	Refreshed every tick
Target destroyed / respawns	Stays stale until re-entry	Re-resolves immediately
Costs a state	Yes - unless Stay Running After Resolve lets it share one	No
Visible to	Nodes ordered after it	The whole subtree of the state it sits on
Can fail a state	Yes (returns Failed)	No - gate on its b Found Actor output

Prefer the evaluator for goal sources. Put it on the parent state, set Tag to Player, and bind Fly To's Target Actor to its Found Actor. One resolver then feeds Fly To, Look At, conditions and transitions at once, and it stays correct when the target dies or streams in late.

Only the *search* is throttled by Refresh Interval (0.5 s by default); the location output refreshes every tick off the cached pointer, so a moving target is never stale. Reach for the task form when a frozen-on-entry snapshot is the point - picking a patrol anchor and committing to it - or when you specifically want the state to *fail* if nothing carries the tag.

Roaming between tagged goals. The task form also has **Pick Random Match**: tag every goal actor in the level (say, Goal), put the task ahead of **Fly To** in one state, and tick **Stay Running After Resolve** - this switch is what lets the two tasks share a state. Off, the resolve *finishes on entry*, and a task that finishes finishes its **whole state**: Fly To is entered but never ticked, and the agent stands still. Bind Fly To's Target Location to Found Actor Location, then add an **On State Completed** transition back into the same state - as a **Go to State** transition targeting the state itself, *not Tree Succeeded*, which would end the whole run after the first leg. Each re-entry draws a fresh random goal - Avoid Repeat Pick (on by default) guarantees it is never the goal the agent is already sitting on - and every agent runs its own tree instance, so a crowd sharing one StateTree asset spreads across the goal set by itself. This is the tagged-actor twin of the **Pick Random Goal** task, which does the same loop with random *points* instead of placed actors and carries the same Stay Running After Resolve switch for the same reason.

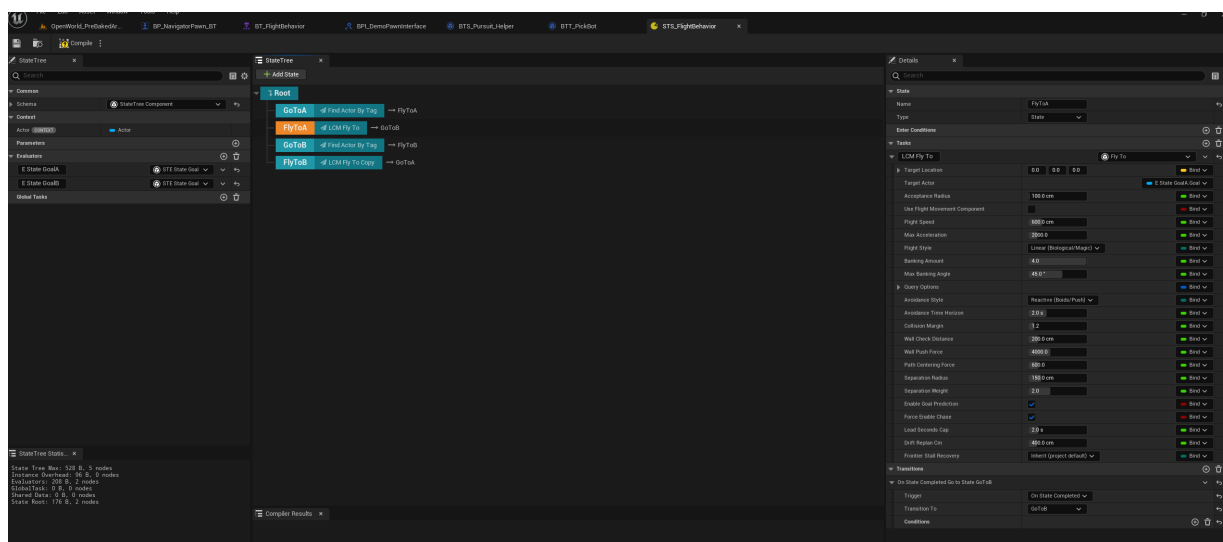


Figure 04.1 - A two-goal patrol, the StateTree way. This is the shipped STS_FlightBehavior. Each FlyTo* state runs one LCM Fly To task and transitions **On State Completed** to the next, so the patrol loop is expressed as states and transitions rather than as a Behavior Tree Loop decorator. The Find Actor By Tag states in between are what fill in each goal at runtime.

Note the **Evaluators** in the left panel - E State GoalA and E State GoalB. That is the StateTree replacement for a blackboard: an evaluator produces a value, and the task binds to it.

The screenshot shows the 'Details' panel of the StateTree configuration tool. It is set to the 'State' tab, showing a state named 'FlyToA' of type 'State'. Below this, the 'Tasks' section is expanded, showing a task named 'LCM Fly To'. The task has a 'Fly To' icon and a dropdown menu. The task parameters are listed in a table-like structure with a 'Bind' column on the right.

Parameter	Value	Bind
Target Location	0.0 0.0 0.0	Bind
Target Actor	E State GoalA.Goal	Bind
Acceptance Radius	100.0 cm	Bind
Use Flight Movement Component		Bind
Flight Speed	600.0 cm	Bind
Max Acceleration	2000.0	Bind
Flight Style	Linear (Biological/Magic)	Bind
Banking Amount	4.0	Bind
Max Banking Angle	45.0 °	Bind
Query Options		Bind
Avoidance Style	Reactive (Boids/Push)	Bind
Avoidance Time Horizon	2.0 s	Bind
Collision Margin	1.2	Bind
Wall Check Distance	200.0 cm	Bind
Wall Push Force	4000.0	Bind
Path Centering Force	600.0	Bind
Separation Radius	150.0 cm	Bind
Separation Weight	2.0	Bind
Enable Goal Prediction	☒	Bind
Force Enable Chase	☒	Bind
Lead Seconds Cap	2.0 s	Bind
Drift Replan Cm	400.0 cm	Bind
Frontier Stall Recovery	Inherit (project default)	Bind

Below the task parameters, the 'Transitions' section is expanded, showing a transition named 'On State Completed Go to State GoToB'. The transition has a 'Trigger' of 'On State Completed', a 'Transition To' of 'GoToB', and no conditions.

Figure 04.2 - The same task, in StateTree form. The properties are the ones you already met in Tutorial 03 - the difference is the **Bind** column down the right-hand edge. In StateTree every field can be driven by an evaluator, a parameter or another state's output, not just the goal.

In this shot Target Actor is bound to E State GoalA.Goal while everything else holds a literal, which is the normal arrangement: bind what changes, type what does not.

Frontier Stall Recovery reads Inherit (project default) here, and the task cannot tell you what that resolves to. A StateTree asset is not owned by a level, so it does not know which manager it will run under. The manager's 7. Runtime State ▶ Frontier Stall Recovery (auto) row is the one place that answer exists.

4.5 Step 4: The AI Controller

1. Create an **AIController** Blueprint, **BP_MyFlyerAI_ST**.
2. Add a **StateTree AI Component**.
3. Select it and set **State Tree** to **ST_MyFlyer**.
4. Set the goal parameter on **BeginPlay**. On the component, use **Set Parameter** (or set the value before the tree starts) with:
 - **Name:** `GoalLocation`
 - **Value:** e.g. `GetActorOfClass(TargetPoint) → GetActorLocation`
5. On **BP_MyFlyer** → **Class Defaults** → **AI Controller Class** = **BP_MyFlyerAI_ST**.

Unlike the BT path there is no **Run Behavior Tree** call: the **StateTreeAIComponent** starts the tree itself.

4.6 Step 5: Fly

Place a **TargetPoint** and your pawn, press **Play**. Same result as Tutorial 03: the pawn routes through 3D space to the goal.

4.7 The rest of the StateTree surface

The plugin ships a full StateTree task set, all under the **LCM Nav3D** category:

Task	Does
Fly To	The flagship: 3D navigation to a point or actor
Patrol	Cycle through a set of points
Orbit Target	Circle a target at a radius
Land	Descend and touch down
Look At / Turn In Place	Orientation without translation
Find Cover	Move to tactical cover
Evade CBS	Avoidance-driven evasion
Follow Flow Field	Follow a shared crowd steering field
Chunk Prewarm / Streaming Aware	Infinite-world streaming helpers
Find Actor By Tag / Pick Random Goal / Wait	Utility
Use Smart Object	Smart Object interaction

Plus conditions (**Path Exists**, **Is At Location**, **Chunk Loaded**, **Has Line Of Sight**, **Is In Cover**, **Health Below**, **Path Length Below**, **Threat Visible Via Perception**) and evaluators (**Path Status**, **Path Progress**, **Navigation Load**, **Threat Field**, **Squad**, **Memory**, **Perception Relay**, **Find Actor By Tag**).

Reading a node's outputs. Fields a node *produces* are grouped under an **Output** section and drawn as read-only OUT pins - those are the ones you bind *from*. Everything else is a parameter you either type a value into or bind *to*.

Every one is listed with its full property set in the API Reference.

4.8 Behaviour differences worth knowing

The BT and ST Fly To tasks are built for parity, but a few things genuinely differ:

	Behavior Tree	StateTree
Goal source	Blackboard key (Vector or Actor)	Target Location or Target Actor instance data
Chase	Works with a Vector or Actor key	Requires a non-null Target Actor
Arrival reported	Against the pre-move location that frame	Against the post-move location

The arrival difference is a sub-frame detail: it only matters if you're comparing the two side-by-side with a very small Acceptance Radius.

4.9 Troubleshooting

Symptom	Cause
Fly To isn't in the task list	Asset wasn't created with the StateTree AI Component Schema
Task fails immediately	No LcmNavigationManagerSV0 in the level, or the goal is outside/inside geometry
Agent never starts	StateTreeAIComponent has no State Tree assigned, or Auto Possess AI is Off
Goal is always (0,0,0)	The parameter isn't bound, or is set after the tree already ran

More: Troubleshooting.

Next: Tutorial 05: Dynamic Obstacles. Make the world move and watch agents reroute.

Tutorial 05: Dynamic Obstacles

TL;DR: Add an `LCM Nav3D Dynamic Obstacle` component to any actor that moves. It restamps the octree as the actor moves and agents reroute around it: closing doors, rising platforms, patrolling hazards, destructible cover.

Time: ~15 minutes. Assumes Tutorial 03.

5.1 Static vs dynamic

When the navigation volume is built, all static geometry is baked in. That bake is what makes queries fast, but it's a snapshot.

- **Never moves?** Do nothing. It's already in the octree.
- **Moves?** It needs a **Dynamic Obstacle** component, or the octree will keep believing the space is empty and agents will fly straight through it.

The common mistake: moving a static-mesh actor at runtime *without* the component. Nothing errors: agents ignore it, because as far as the navigation data is concerned it never moved.

5.2 Step 1: Make a moving obstacle

1. Create a Blueprint Actor, `BP_MovingWall`.
2. Add a **Static Mesh** component: a Cube, scaled to something that genuinely blocks a route (say $400 \times 100 \times 400$).
3. Make sure the mesh **has collision enabled**. The voxelizer finds obstacles via collision, so a mesh set to *No Collision* is invisible to navigation.

5.3 Step 2: Add the Dynamic Obstacle component

1. In `BP_MovingWall`, **Add Component** → search "`LCM Nav3D Dynamic Obstacle`".
2. That's the whole setup. It auto-registers with the navigation manager at `BeginPlay` and tracks its owner from then on.

5.3.1 The settings

Field	Default	What it does
Update Distance Threshold	50 cm	How far the actor must move before the octree is restamped. Larger = cheaper, but navigation lags reality.
Tracked Components	(empty)	Empty means the whole actor . Pick specific components to track only part of it.

Tracked Components is worth knowing about. On an actor where only one part obstructs (a gate inside a decorative frame, a swinging arm on a fixed base), pick just that component. Only its movement restamps the octree, and only its bounds block agents.

You can also manage this at runtime from Blueprint with **Add Tracked Component** and **Remove Tracked Component**.

5.4 Step 3: Move it

Give it motion. The simplest version, in the Event Graph:

Event Tick → SetActorLocation(StartLocation + (0, sin(GameTime) * 700, 0))

Or use a Timeline, a matinee/sequencer track, physics; it doesn't matter. The component watches the transform, however it changes.

5.5 Step 4: See it work

1. Place BP_MovingWall so it periodically **seals the only route** between your flyer and its goal.
2. Press **Play**.

The agent should path around the wall, and when the wall closes the route, re-plan and take a different way.

To watch it happen: tick **Draw Debug Volumes** on the navigation manager. You'll see the voxels under the wall flip to blocked as it moves. That's the restamp.

5.6 How agents notice

Agents re-plan automatically. The relevant setting is on the **navigation manager**:

Field	Default	Meaning
4. Performance → Finite Obstacle Update Interval	0.1 s	How often obstacle changes are folded into the finite-world octree

Lower = more reactive, more CPU. 0.1 is a good balance. For fast-moving hazards where agents must react instantly, try 0.05.

5.7 Ready-made examples

These demo maps show this working. Open any of them and read the setup:

Map	Shows
Demo/BehaviourTree/PreBaked_FiniteMaps/DynObs_BT	Obstacles sealing routes in a finite world
Demo/BehaviourTree/InfiniteMaps/DynObs_BT	The same rig in a streaming infinite world
Demo/BehaviourTree/InfiniteMaps/DynObsTunnel_BT	Movers sweeping a sealed tunnel: the wall-integrity test
Demo/StateTree/InfiniteMaps/DynObs_ST · DynObsTunnel_ST	The StateTree equivalents
Demo/MassEntity/PreBaked_FiniteMaps/DynObs_Mass	Dynamic obstacles against a Mass crowd

Map	Shows
Demo/MassEntity/InfinteMaps/DynObs_Tunnel_MassEntity	A crowd flowing through a tunnel as movers sweep it

The obstacles in these maps are driven along splines by the bundled **BP_SplineMove** Blueprint (Content/Blueprints/), so you can see a complete working rig rather than assembling one. For a simpler ping-pong mover the plugin also ships the **LCM Nav3D Obstacle Mover** component, below.

5.8 The bundled Obstacle Mover

The plugin ships a small helper so the demo maps move without Blueprint wiring, and it's useful in your own prototypes.

LCM Nav3D Obstacle Mover ping-pongs its owner between its start pose and an offset.

Field	Default	Meaning
Seal Offset	(400, 0, 0)	Vector from the open pose to the sealed pose
Period	4.0 s	Time for a full open → sealed → open cycle
Phase	0.0 s	Offset so several obstacles don't move in lockstep

Pair it with a Dynamic Obstacle component on the same actor and you have a moving obstacle with zero scripting. It moves the actor kinematically, which is exactly what the obstacle tracker expects.

There's a companion **LCM Nav3D Agent Driver** used by the demo maps to loop a pawn between two tagged points (LcmGoalA / LcmGoalB) using the real async navigation stack, handy for testing a level without authoring a full BT.

5.9 Performance notes

Dynamic obstacles are not free: each restamp re-voxelizes a region.

Keep it cheap:

- Prefer **fewer, larger** dynamic obstacles over many small ones.
- Raise **Update Distance Threshold** for things that move constantly but don't need precision.
- Use **Tracked Components** so only the genuinely blocking part is considered.
- If something moves but can never block a route (debris, VFX props, decoration), **don't give it the component at all.**

Don't put a Dynamic Obstacle component on your navigating agents. Agents avoid each other through the avoidance system (Tutorial 03 Step 7); making them obstacles too means each one is constantly re-voxelizing the world around itself.

5.10 Troubleshooting

Symptom	Cause
Agents fly straight through the obstacle	No Dynamic Obstacle component, or the mesh has collision disabled
Reaction is sluggish	Update Distance Threshold too high, or Finite Obstacle Update Interval too long

Symptom	Cause
Frame rate drops when things move	Too many dynamic obstacles, or they're too finely voxelized: raise <code>Min Voxel Size</code> or reduce obstacle count
Agent gets trapped when a route seals	Expected if there's genuinely no other route. Give the level a second path, or handle the failure in your BT/StateTree

More: Troubleshooting.

5.11 Where to go next

You now have the whole core loop: a navigation volume, agents that fly through it, and a world that can change underneath them. From here, pick what your project needs:

If you need...	Go to
Thousands of agents	Tutorial 06: Mass Entity
AI that picks tactical positions	Tutorial 07: Tactical EQS
Sight and awareness	Tutorial 08: Perception
Ground and flying AI together	Tutorial 09: Hybrid with Recast
Custom traversal or cost zones	Tutorial 10: Nav Links & Area Modifiers
To inspect it live in-game	Tutorial 11: Gameplay Debugger
To tune every setting	Settings Reference

Part II

Going Further

CHAPTER 6

Tutorial 06 - Mass Entity crowds (10K+ agents)

Add one trait to a Mass Entity Config and a crowd navigates in full 3D. No per-agent controller, no tick function, no code.

Time: ~20 minutes. **You'll end with:** thousands of agents flying to a goal.

A UActor AI agent costs roughly 50 μ s/tick once it has a controller, movement component and behaviour tree - a few hundred saturate the game thread. Mass represents agents as ECS fragments instead, so the same work runs in one processor sweep.

6.1 Prerequisites

- The MassEntity and MassGameplay plugins enabled. LCM Nav3D enables both; if you stripped them, re-enable both or the module fails to load.
- An LcmNavigationManagerSV0 in the level with navigation built.
- Epic's "MassEntity Quick Start" if you have never used Mass - this tutorial only adds a trait on top of the standard flow.

6.2 Step 1: Create the agent config

Add → Miscellaneous → Data Asset → MassEntityConfigAsset. Name it DA_MyCrowd.

Open it and add three traits - one from each category:

Trait	Category	Set
LCM Nav3D Agent	Movement - exactly one	Nav Mode, Agent Radius, Max Speed
LCM Nav3D Goal	Goal	Goal Mode, and the location or tag
LCM Nav3D ISM Render	Rendering	Mesh

6.2.1 Choosing a Nav Mode

Nav Mode	Use it for	Cost
Path Following	Agents with <i>different</i> goals, precise routes, infinite worlds	One path per agent
Flow Field	A crowd sharing <i>one</i> goal - the 10K-scale option	One field, shared by everyone

⚠ **Add a rendering trait or your agents are invisible.** They spawn and navigate correctly, they just draw nothing - the navigation trait deliberately owns navigation only. This is the most common first-run surprise.

6.2.2 Movement settings worth knowing

Setting	Default	What it does
Max Turn Rate Deg	360	The knob that turns square corners into arcs. 0 = instant snap
Max Acceleration	2000	cm/s ² . With turn rate, this is what makes movement look weighted
Flight Style	Physics	Physics (inertia) vs Linear (magic / biological, instant)
Avoidance Style	Reactive	Reactive (boids) / Predictive (ORCA, smooth mutual passing) / None
Lane Bias	0.6	Spreads a crowd across a corridor instead of balling up at turns

Defaults give natural movement out of the box.

6.3 Step 2: Spawn them

1. Place a **MassSpawner** actor in the level.
2. In **Entity Types**, reference DA_MyCrowd.
3. Set **Spawn Count** - start at 100 while you wire it up.
4. Give it a spawn-distribution asset so agents start at distinct positions **inside your navigation volume**.

6.4 Step 3: Give them a goal

The LCM Nav3D Goal trait does this with no code:

Goal Mode	Set	Behaviour
Fixed Location	Goal Location	Everyone paths to that point
Tagged Actor	Target Actor Tag (e.g. ApexGoal)	Everyone paths to the tagged actor, re-pathing if it moves

Repath Tolerance Cm decides how far a moving target drifts before agents re-path; Arrival Tolerance Cm is where they stop.

Write the goal and flip the status; the processors own every transition from there.

```
auto& Req = EntityManager.GetFragmentDataChecked<FLcmMassPathRequestFragment>(Handle);
Req.Goal = TargetPos;
Req.Status = ELcmMassPathStatus::Pending;
```

Status moves Pending → InFlight → Complete (Or Failed) and is readable at any time, so your own logic can branch on it.

6.5 Step 4: Check it works

1. Press **Validate Entity Config** on the asset. It catches the trait mistakes that are silent at runtime and all present as "my agents don't move".
2. Press Play with `r.LcmNav.Mass.DebugDraw 1` - agents draw as status-coloured spheres (grey idle, yellow pending, green complete, red failed) with a line to their waypoint.
3. Scale up gradually: 100, then 1,000, then 10,000, watching frame time.

6.6 If nothing moves

Symptom	Cause	Fix
Nothing visible at all	No rendering trait	Add LCM Nav3D ISM Render
Spheres are red	No path to the goal	Goal is outside the navigation volume, or inside geometry
Spheres stay grey	No goal set	Add the Goal trait, or write the request fragment
Validation error on save	Two movement traits, or Goal on a Flow Field agent	See Mass Trait Reference
Crowd vanishes when you walk away	World Partition streamed the spawner out	See the World Partition section of the trait reference

6.7 Tuning

Kill-switches and global overrides. Production authoring is the trait, not the console.

CVar	Default	What it does
<code>r.LcmNav.Mass.MaxPathsPerTick</code>	64	Spreads a large spawn wave's path requests across frames
<code>r.LcmNav.Mass.Avoidance</code>	1	Master switch for inter-agent avoidance
<code>r.LcmNav.Mass.DebugDraw</code>	0	Status-coloured spheres and waypoint lines

6.8 Scope

Mass agents resolve both endpoints **within the agent's own chunk**. Keep crowd goals in the same chunk as the crowd, or use a Behaviour Tree agent for long cross-chunk routes.

6.9 Related

- Mass Trait Reference - every trait, the combination matrix, and the World Partition trap.
- Tutorial 04 - StateTree, the per-entity AI layer that pairs with Mass.

Tutorial 07 - Authoring true-3D tactical EQS

TL;DR: LCM Nav3D ships EQS generators and tests that work in **true 3D** - candidate points anywhere in the navigable volume, scored by volumetric SVO line-of-sight, cover, threat-exposure, LCM Nav3D-path-distance and height. Author a query from these nodes exactly like stock EQS; the only differences are the Apex: -prefixed generators/tests and that "distance" means *LCM Nav3D path*, not Euclidean or navmesh.

7.1 The LCM Nav3D EQS node catalog

Generators (UEnvQueryGenerator, produce 3D points): | Node | Produces | |---|---| | **LCM Nav3D | Points In SVO Volume** | uniform 3D points in an AABB, filtered to unblocked SVO leaves | | **LCM Nav3D | Hemispherical Ring 3D** | shells around a target (perch / orbit / high-ground); *Upper Hemisphere Only* option | | **LCM Nav3D | Points Along Path** | points sampled along the LCM Nav3D path between two contexts | | **LCM Nav3D | Portal Proximity** | points at macro-graph portal centres near a context (chokepoints) |

Tests (UEnvQueryTest, score/filter): | Node | Axis | Notes | |---|---|---| | **LCM Nav3D | Line Of Sight 3D** | visible to context (bool) | volumetric SVO LOS, not 2.5D | | **LCM Nav3D | Reachability 3D** | an LCM Nav3D path exists (bool) | filter unreachable candidates | | **LCM Nav3D | Path Distance 3D** | LCM Nav3D path length (float) | NOT Euclidean; Context = Querier | | **LCM Nav3D | Threat Exposure** | summed SweepPenalty toward threats (float) | lower = safer | | **LCM Nav3D | Cover Score 3D** | fraction of threats occluded-from (float) | higher = better cover | | **LCM Nav3D | Height Advantage** | Z over context (float) | high ground |

Contexts (UEnvQueryContext): | Node | Returns | |---|---| | **LCM Nav3D | Known Threats** | actors tagged ApexThreat | | **LCM Nav3D | Threats From AI Perception** | the querier's perceived threats (Tutorial 08) |

7.2 Step 1 - Author a sniper-perch query

Create an EQS query EQS_FindSniperPerch:

1. **Generator:** **LCM Nav3D | Hemispherical Ring 3D** - GenerateAround = the threat context (Known Threats / Threats From AI Perception), RadiusMin/Max = your engagement band, Upper Hemisphere only = true (perch above the threat).
2. **Test:** **LCM Nav3D | Line Of Sight 3D** (Context = threats) - *Score, prefer should-have-LOS* - a perch must see its target.
3. **Test:** **LCM Nav3D | Cover Score 3D** (Context = threats) - *Score↑* - partial cover is good.
4. **Test:** **LCM Nav3D | Height Advantage** (Context = threats) - *Score↑* - favour high ground.
5. **Test:** **LCM Nav3D | Reachability 3D** (Context = Querier) - *Filter* - the agent must be able to get there.

The winning item is a reachable, high, in-cover position that still has line of sight to the threat - a sniper perch, chosen in full 3D.

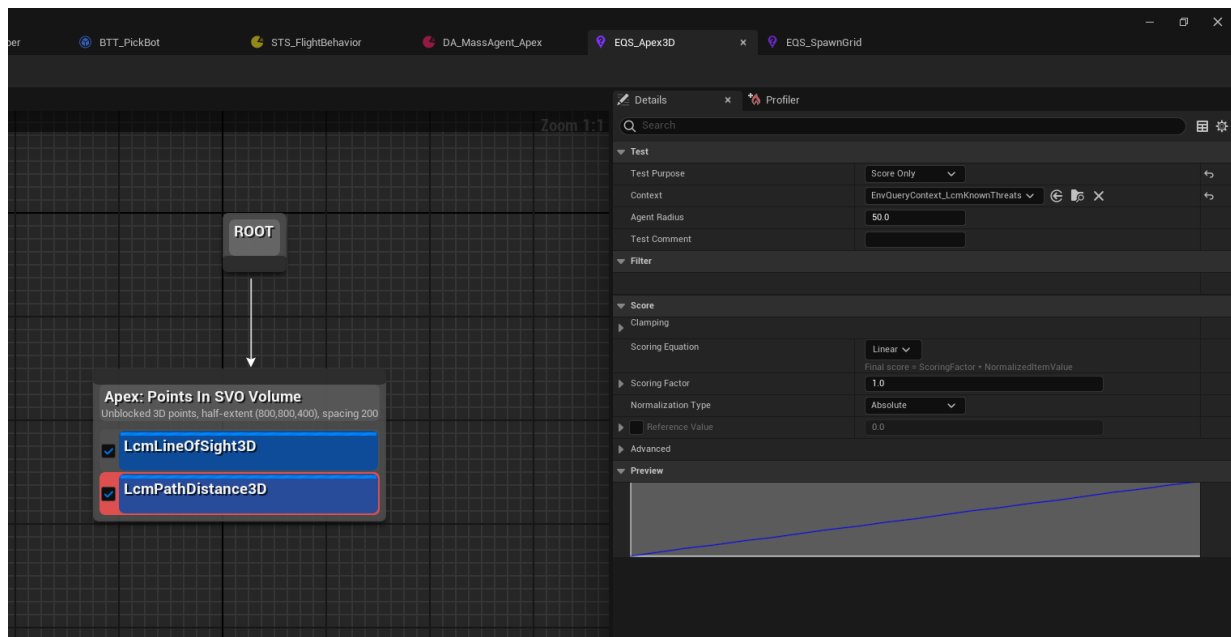


Figure 07.1 - The anatomy of a Nav3D query. One generator produces candidate points, then tests score or filter them, top to bottom. Here Apex: Points In SVO Volume is emitting unblocked 3D points across an 800×800×400 half-extent at 200 spacing - *unblocked* being the part a floor-plane generator cannot do, because it never had a third axis to be blocked in.

The two tests are colour-coded by role in the graph: a **filter** removes items, a **score** ranks them. The Details panel on the right belongs to the selected test:

- **Test Purpose** - Score Only, Filter Only, or both. This is the switch that decides whether a failing item is *penalised* or *deleted*.
- **Context** - what the test measures *against*. EnvQueryContext_LcmKnownThreats here, so line of sight is evaluated toward the threats the agent knows about.
- **Agent Radius** - how wide the thing that has to fit is. Leave this at your real agent size; a query that scores a perch a 50 cm drone can reach tells you nothing about a 200 cm gunship.
- **Scoring Equation** and the **Preview** graph - the curve mapping raw measurement to score. The preview updates live, so you can see a Linear ramp become a Square bias before you ever run the query.

7.3 Step 2 - The EQS Testing Pawn

Drop an **EQS Testing Pawn** in the level, set its QueryTemplate to EQS_FindSniperPerch, and place threats so the contexts resolve. The candidate items render as colour-graded balls; tweak generator/test parameters and watch the scores update live.

UE's stock EQSTestingPawn is all you need - the LCM Nav3D nodes render through it normally.

7.4 Step 3 - Wire the result into a Behavior Tree

1. A Run EQS Query task/service runs EQS_FindSniperPerch and writes the best item to a blackboard Vector key.
2. A MoveTo (or the LCM Nav3D FlyTo BT task) moves the agent to that key.
3. Re-run on an interval (service) so the perch updates as threats move.

(StateTree: run the query from a task that stores the location, then transition to your LCM Nav3D move task - see Tutorial 05.)

7.5 The five shipped example queries

Recipes you can build now from the catalog above:

Query	Recipe
EQS_FindSniperPerch	Hemispherical Ring 3D + Cover Score 3D + LOS 3D + Height Advantage
EQS_FindFlankingApproach	Points In SVO Volume + Threat Exposure↓ + Path Distance 3D
EQS_FindAmbushCorner	Portal Proximity + LOS 3D (prefer blocked) + Reachability 3D

7.6 Related

- Tutorial 08 - perception-driven threat sources (the auto-populated threat context).
- Tutorial 05 - StateTree integration (the move side).
- ULcmTacticalQueryLibrary - the same primitives as Blueprint nodes (Is Segment LOS Clear, Find Best Cover, Find Apex Path Sync, ...).

Tutorial 08 - Perception-driven tactical AI

TL;DR: Add an **LCM Nav3D AI Perception Bridge** next to your agent's **UAIPerceptionComponent** → its perceived threats auto-feed the **LCM Nav3D | Threats From AI Perception** EQS context → a true-3D LCM Nav3D EQS query (volumetric candidate points + 3D-LOS / cover / threat-exposure tests) picks a flanking or cover position → your BT/StateTree moves the agent there. Mass crowds get the same via the **LCM Nav3D Mass Perception Agent** trait, which writes a perceived-threats fragment.

8.1 Why this exists

Stock UE tactical AI flows through EQS, but the stock generators/tests are 2.5D (navmesh-projected). A flying or multi-level NPC can't reason about cover, line-of-sight, or flanking in true 3D with them. LCM Nav3D ships **true-3D EQS** - candidate points anywhere in the navigable volume, scored by **volumetric SVO** line-of-sight, cover, and threat exposure - and a **one-click bridge** so the threats those tests score against come straight from UE's own perception system.

8.2 Prerequisites

- LCMNav3D plugin enabled, an **ALcmNavigationManagerSVO** with a built SVO in the level.
- An AI agent with an **AAIController** and a **UAIPerceptionComponent** configured with an **AI_Sense_Sight** config (the standard UE setup - LCM Nav3D doesn't change this).
- Threats set up so perception can sense them (registered stimuli source / pawns on a hostile team), per your normal UE perception configuration.

8.3 Part A - UActor agent (BT / StateTree)

8.3.1 Step 1 - Add the perception bridge

On your agent (the pawn or its controller - wherever the **UAIPerceptionComponent** lives), **Add Component** → "**LCM Nav3D AI Perception Bridge**" (**ULcmAIPerceptionBridge**).

- It auto-finds the owner's **UAIPerceptionComponent** on **BeginPlay** and subscribes to **OnTargetPerception Updated**.
- **DecayWindowSeconds** (default 5) - how long a lost-sight threat stays queryable before it fades. Tune to taste.
- No other wiring: successfully-sensed actors are tracked; lost ones decay out.

That's the entire perception-to-LCM Nav3D link.

8.3.2 Step 2 - Build the tactical EQS query

Create an EQS query (e.g. **EQS_FindFlankingApproach**) and assemble it from these nodes:

Generator (pick one): | Node | Use it for | **LCM Nav3D | Points In SVO Volume** | general 3D candidate cloud around the agent | | **LCM Nav3D | Hemispherical Ring 3D** | sniper-perch / high-ground / orbit candidates (set *Upper Hemisphere Only*) | | **LCM Nav3D | Points Along Path** | candidates strung along the LCM Nav3D path to a goal |

Tests (add several; set each one's **Context** to **LCM Nav3D | Threats From AI Perception** where it scores against threats): | Node | Purpose | Typical setup | |---|---|---| | **LCM Nav3D | Threat Exposure** | prefer low summed danger toward threats | Score, prefer less | | **LCM Nav3D | Cover Score 3D** | prefer positions occluded from threats | Score, prefer greater | | **LCM Nav3D | Line Of Sight 3D** | filter to (or away from) visible-to-threat | Filter | | **LCM Nav3D | Reachability 3D** | drop positions the agent can't path to | Filter (Context = Querier) | | **LCM Nav3D | Path Distance 3D** | prefer near / far by *LCM Nav3D* path length | Score (Context = Querier) | | **LCM Nav3D | Height Advantage** | favour high ground over threats | Score, prefer greater |

A solid "flank to cover" query: **Points In SVO Volume** → **Reachability 3D** (filter) → **Cover Score 3D** (score↑) → **Threat Exposure** (score↓) → **Path Distance 3D** (score↓, mild weight).

The threats every threat-scoring test sees come from the bridge via **LCM Nav3D | Threats From AI Perception** - no manual tagging. (For non-perception workflows, **LCM Nav3D | Known Threats** returns actors tagged `ApexThreat` instead.)

8.3.3 Step 3 - Wire it into BT or StateTree, then verify

- **Behavior Tree:** add a `Run EQS Query` service/task that runs your query and writes the result `Vector/Actor` to the blackboard, then a `MoveTo` (or an `LCM Nav3D FlyTo`) to that key.
- **StateTree:** run the query from a task that stores the chosen location, then transition into your `LCM Nav3D move` task (see Tutorial 05).

Verify in PIE: the agent should pick positions that are reachable, in cover from, and low-exposure toward the threats it currently perceives - and should **stop reacting to a threat ~5 s after losing sight of it** (the decay window). Walk a threat out of the sight cone and watch the chosen position shift once it decays.

8.4 Part B - Mass crowd variant

For crowd-scale tactical perception, use the native `LCM Nav3D Mass perception` (Tutorial 06 covers Mass setup):

1. Add the **LCM Nav3D Mass Perception Agent** trait (`ULcmMassPerceptionTrait`) to your Mass entity config - set `FactionId`, `SightRadius`, `FOVDegrees`. Agents perceive different-faction agents within range + FOV + **true-3D SVO line of sight**.
2. The `ULcmMassPerceptionProcessor` writes each agent's closest visible hostile into its **FLcmMass PerceivedThreatsFragment** (`ClosestThreatPos`, `bHasThreat`, `VisibleCount`, `LastSeenSeconds`).
3. Read that fragment in your Mass logic (a `StateTree` task via `UMassStateTreeProcessor`, a steering processor, etc.) to drive evade / engage / regroup behaviour.

Mass agents consume the perceived-threats fragment **directly** - they don't issue EQS queries (there is no Mass-native EQS querier). EQS tactical queries are the `UActor` path (Part A). A hero-NPC-as-`UActor` querying with `Apex: Known Threats` can include crowd-relevant threats by tagging, giving a practical hybrid.

8.5 What you build by hand

The nodes above are all shipped and usable. What is **not** shipped is prebuilt query assets - you author your own from the generators and tests, which Tutorial 07 walks through end to end.

Four nodes need live flow-field or macro-graph state and are not available: the flow-field-directed generator, the flow-field-alignment test, the active-flow-field context, and the portal-proximity generator.

8.6 Related

- Tutorial 04 - StateTree, the move-task side of step 3.
- Tutorial 06 - Mass Entity, for the Part B agent setup.
- Tutorial 07 - Tactical EQS, authoring queries from these nodes.

- `ULcmTacticalQueryLibrary` - the Blueprint tactical primitives (Is Segment LOS Clear, Find Best Cover, ...) the EQS tests wrap.

CHAPTER 9

Tutorial 09 - Hybrid LCM Nav3D + Recast crowd avoidance

TL;DR: Add `ULcmCrowdAvoidanceBridge` to your LCM Nav3D flying pawn → stock UE5 `UCrowdManager` Detour RVO sees the LCM Nav3D pawn and steers Recast crowd agents around it. The Z-band filter (auto-applied by the bridge) prevents false-collision avoidance between agents at different vertical bands.

9.1 Prerequisites

- The third-person template (or any project with a Character + `AIController`).
- `LCMNav3D` plugin enabled.
- An `ALcmNavigationManagerSV0` in the level.
- An `ALcmFlyingPawn` (or your own LCM Nav3D flying pawn).

9.2 Step 1 - Add the bridge to your LCM Nav3D pawn

In the editor, select your LCM Nav3D pawn → Add Component → search "LCM Crowd Avoidance Bridge".

Defaults are sensible:

- `bAutoRegister` = `true` - registers with `UCrowdManager` on `BeginPlay` automatically
- `AgentRadius` = 50 cm - typical pawn radius
- `AgentHalfHeight` = 100 cm - cylinder extent for Detour
- `MaxSpeed` = 800 cm/s - matches `ULcmNavMovementComponent` default

That's it for the LCM Nav3D side.

9.3 Step 2 - Add Recast crowd agents (existing UE5 pattern)

Your Recast ground crowd agents should already use `UCrowdFollowingComponent` instead of the default `UPathFollowingComponent`. This is the standard UE5 setup for any project using crowd avoidance - LCM Nav3D doesn't change this. If you're not already on `UCrowdFollowingComponent`:

```
// In your AAIController subclass constructor  
GetCharacterMovement()->bUseAccelerationForPaths = true;  
//. and override the AIController to use UCrowdFollowingComponent
```

Or in Blueprint: the `AIController`'s `UCrowdFollowingComponent` is auto-registered as the agent.

9.4 Step 3 - Play

Hit Play. Your LCM Nav3D flying pawn and Recast ground crowd agents now avoid each other (per the Z-band filter rules below).

9.5 How the Z-band filter works

Unreal's UCrowdManager Detour RVO **assumes coplanar agents** - out of the box, a flying LCM Nav3D agent at Z=500 would be flagged as a "neighbour" by a Recast ground agent at Z=0, producing avoidance behaviour that looks like the ground agent dodging an obstacle 5 meters overhead.

The bridge fixes this at the **avoidance-group bitfield level**:

- Each agent gets an AvoidanceGroup bit derived from its world Z divided by `r.LcmNav.Crowd.ZBandHeightCm` (default 200 cm).
- Each agent's GroupsToAvoid is set to its own band + the ± 1 adjacent bands (so vertically close agents still see each other).
- Cross-band agents (e.g. ground Z=0 and aerial Z=500 with 200 cm bands $\rightarrow$ bands 0 and 2) **ignore each other** via Detour's standard bitfield mask.

Same engineering work, leverages stock Detour rather than fighting it.

9.6 Tuning `r.LcmNav.Crowd.ZBandHeightCm`

Use case	Recommended value
Default (typical pawn-height scenes)	200
Low-ceiling interior (1.5m floors)	150
Vertical skyscraper (need fine-grained per-floor separation)	100
Open sky (most LCM Nav3D traffic is far above ground)	400
Disable filter (legacy Detour coplanar behaviour)	0

Set the CVar in PIE to live-tune, then commit to your project's `DefaultEngine.ini` once you're happy.

9.7 Stacked multi-floor scenarios

UCrowdManager is one global instance per world. For stacked-floor scenarios (e.g. LCM Nav3D agents at Z=200, 500, 800 sharing the same UCrowdManager), the Z-band filter handles it automatically - each pair of agents whose Z-distance is > 1 band-height ignores each other.

For **truly independent** crowds (each floor should be its own crowd zone), you'd need to spawn separate UCrowdManager instances per zone via project-side subclassing. This is out of scope for v1.

9.8 Performance notes

- Each registered LCM Nav3D bridge contributes ~ 1 RegisterAgent call at BeginPlay + ~ 7 virtual-call evaluations per Detour tick (Detour reads the ICrowdAgentInterface getters once per query).
- The Z-band filter eliminates $\sim 99\%$ of cross-band RVO computation cost - only agents within ± 1 band of you contribute to your avoidance solve.
- Disabling the filter (`r.LcmNav.Crowd.ZBandHeightCm = 0`) restores legacy Detour coplanar behaviour for benchmarking.

9.9 Acceptance test (manual)

1. Spawn 5 Recast ground crowd agents in a circle at Z=0.
2. Spawn 1 LCM Nav3D flying pawn with ULcmCrowdAvoidanceBridge at Z=500.
3. Fly the LCM Nav3D pawn through the middle of the ground-agent circle.
4. **Expected:** ground agents continue their pathing without dodging the airborne pawn (Z-band separation $> 200\text{cm}$ default).

5. Now fly the LCM Nav3D pawn down to $Z=100$ - ground agents dodge it.

If step 4 fails (ground agents dodge the airborne pawn), check `r.LcmNav.Crowd.ZBandHeightCm` is not 0.

9.10 What about the LCM Nav3D pawn's own avoidance?

Two paths, both work:

1. **LCM Nav3D CBS/ORCA** (default, the FlyTo task's built-in steering): the LCM Nav3D pawn avoids Recast crowd agents via the existing CBS context. Detour Crowd is purely the *publishing* surface to the rest of the world.
2. **Use UCrowdFollowingComponent on the LCM Nav3D pawn**: replace the LCM Nav3D pawn's AIController's `UPathFollowingComponent` with `UCrowdFollowingComponent`. The pawn becomes a full Detour Crowd participant - receives the suggested-velocity from Detour and applies it through its movement component. Combine with `ULcmNavMovementComponent` for the locomotion side.

Path 1 is the default; Path 2 is for projects that want a uniform crowd avoidance solver across LCM Nav3D and Recast agents.

9.11 Mass variant

A Mass Entity equivalent is not available in this release.

9.12 Related LCM Nav3D surfaces

- `LCM_BTTask_FlyTo` - already has CBS+ORCA built-in for LCM Nav3D-internal avoidance
- `LCM_BTTask_EvadeCBS` - pure-defensive evade using the same CBS context
- `ULcmNavMovementComponent` - LCM Nav3D-aware flying-pawn movement component

CHAPTER 10

Tutorial 10 - Cost areas, query filters and 3D NavLinks

TL;DR: Drop an `ALcmNavAreaModifier` volume, give it a stock `NavArea` class, and every LCM `Nav3D` planner routes by it: `FlyTo` (BT and `StateTree`), `Fly To Location`, `Mass crowds`, `FindPath` and the stock `Move To`. A stock `NavigationQueryFilter` on the agent decides what each area costs **that** agent, or forbids it. `ULcmNavLinkComponent` places 3D ladders, drops and teleporters; link costs are queryable, and the planner does not route through links yet.

10.1 What an area does

A cost area is a volume (box, sphere or capsule) with a `NavArea` class. The planner reads three things from the class:

NavArea field	Effect on a route
Default Cost	Travel inside the area costs this many times the distance. 5.0 = five times dearer, 0.5 = a preferred lane.
Fixed Area Entering Cost	Paid once each time the route goes in. Use it for a toll or a doorway you want used sparingly.
Default Cost at <code>NavArea_Null</code> 's level (1e30 or more)	Impassable for everyone. No route crosses it.

A **query filter** on an agent overrides those per class, exactly as it does for `Recast`: a travel cost override, an entering cost override, or **excluded**. Two agents read the same volume differently. The prey avoids a predator's territory that the predator itself ignores.

Routes follow the cheapest total, not the shortest distance. An agent goes round a storm when going round is cheaper, and flies straight through when the detour costs more than the storm.

10.2 Step 1 - Place a cost area

1. Place an **LCM Nav3D Nav Area Modifier (Volumetric)** actor.
2. Set **Shape** and scale the box to cover the volume. A sphere reads its radius from the extent's X. A capsule runs along local Z.
3. Set **Area Class**. The plugin ships ready-made classes, and any stock or project `NavArea` subclass works:

Class	What it does
LCM Nav Area - Storm (x5)	Five times dearer inside.
LCM Nav Area - Preferred Lane (x0.5)	Half price: routes are drawn into it.
LCM Nav Area - Toll Gate (+2000 to enter)	Normal price inside, 2000 cm extra per entry.
LCM Nav Area - Territory (priced by filters)	Free for everyone, until a filter prices it.
LCM Nav Area - Restricted (excluded by filters)	Free, until a filter excludes it.

Class	What it does
LCM Nav Area - No-Fly (impassable)	Nobody crosses it, under any filter.

Leave **Area Class** empty to use **Cost Multiplier** on its own. That is the 3.3 behaviour, with no per-agent overrides.

Overlaps. Where volumes overlap, an area the agent's filter excludes always wins. Otherwise the higher **Priority** decides the cost. Costs do not multiply. (They did before 3.4.)

Moving areas. Attach the volume to anything that moves and tick **Movable**. Agents replan as it moves, and plan against where it will be over **Prediction Seconds**. **Patrol** moves a volume back and forth on its own, for demos and tests. After changing a class, shape, priority or size at runtime, call **Notify Area Changed**. Moving the actor needs no call.

10.3 Step 2 - Give an agent a filter

A filter is a stock `NavigationQueryFilter` Blueprint or class. Add **Areas** entries to override or exclude classes. The plugin ships two examples: `LCM Nav Filter - Prey` (Territory x10) and `LCM Nav Filter - Civilian` (Restricted excluded).

Agent	Where the filter goes
Pawn with an AI controller (BT, StateTree, Fly To Location)	The controller's Default Navigation Filter Class . <code>FlyTo</code> reads it on every request. No task property is needed.
A direct planner call	<code>FLcmNavigationQueryOptions::FilterClass</code> .
Mass crowd	Navigation Filter on the LCM Nav3D Agent trait.
Stock Move To	Its Filter Class pin, as with Recast.

With no filter, each area uses its own class defaults.

10.4 Step 3 - See what an agent sees

On the **LCM Nav3D Manager**, under **6. Debug > Cost Areas**:

- **Cost View:** Regions draws each volume and what it costs. Heat tints the octree by price.
- **Cost View Filter:** whose eyes the view looks through. Set it to the prey's filter to see the territory light up.

The console variable `r.LcmNav.CostView 0|1|2` overrides the property while playing. The Gameplay Debugger's LCM category shows the selected agent's route cost: length, priced length, entering costs paid, and the areas it crosses.

10.5 What happens at the edges

Situation	Result
The goal is inside an area the agent's filter excludes	The request fails: <code>Get Last Path Cost Failure</code> returns Goal In Excluded Area .
Every way to the goal crosses an excluded area	The request fails: No Route Under Filter .
The agent starts inside an excluded area	It is allowed out by the shortest way, then routes normally.

Situation	Result
An area moves across an agent's route	FlyTo replans when the route's cost changes enough to matter.

A FlyTo task reports these as **Failed**, with a Warning in the log, never an Error.

10.6 Step 4 - Query costs yourself

The query library prices a route the same way the planner does:

```
// The travel multiplier at a point, and a route's cost, as a given filter sees them:
bool bExcluded = false;
const float Here = ULcmNavAreaQueryLibrary::GetCostMultiplierAtForFilter(this, Location, UMyPreyFilter::StaticClass(), bExcluded);
float LengthCm, PricedCm, EnteringCost; bool bCrossesExcluded;
ULcmNavAreaQueryLibrary::EvaluatePathCostForFilter(this, Waypoints, UMyPreyFilter::StaticClass(),
    LengthCm, PricedCm, EnteringCost, bCrossesExcluded);
```

The filter-less versions (GetCostMultiplierAt, GetCostMultiplierAlongPath) read each area's own defaults. All of them are Blueprint-callable.

10.7 3D NavLinks

UE's stock UNavLinkCustomComponent projects its ends onto a Recast navmesh. A flying pawn can use any two points in space, so LCM Nav3D ships ULcmNavLinkComponent for vertical ladders, drops and teleporters.

1. Add the **LCM Nav Link (3D Vertical)** component to an actor.
2. Set LinkStart and LinkEnd in world space, e.g. (0, 0, 0) and (0, 0, 800).
3. bBidirectional: climb up and slide down, or one way only.
4. TraversalCostCm: the extra cost of using the link, e.g. 400 to favour a nearby ramp.

The link registers with UE's navigation system (it implements INavLinkCustomInterface) and with LCM Nav3D's registry. **The LCM Nav3D planner does not route through links yet.** Find and price them yourself:

```
ULcmNavLinkComponent* Nearest = ULcmNavAreaQueryLibrary::FindNearestNavLink(AgentLocation, /*MaxDist*/ 500.0f);
const float LinkCost = ULcmNavAreaQueryLibrary::GetNavLinkTraversalCostAlongPath(Waypoints);
```

A common pattern: FlyTo to LinkStart, play the traversal, then continue from LinkEnd.

10.8 Try it

Open the **Cost Areas** tile in the demo menu (NavCost_Showcase maps, in BT, StateTree and Mass versions). Six labelled stations: a storm the route bends round, a preferred lane off the direct line, a no-fly goal beside a storm goal (impassable is not the same as expensive), one territory that only the prey pays for, a toll gate, and two craft of different sizes making the same decision.

10.9 Acceptance test (manual)

1. Place a Storm (x5) box between two points, with open space around it.
2. Fly an agent across. It goes round.
3. Stretch the storm wall to wall so there is no way round. The agent flies through it and pays the storm.
4. Give the agent the Civilian filter and put a Restricted box across the only corridor to its goal. The request fails with **No Route Under Filter**.
5. GetRegisteredLinkCount counts your ULcmNavLinkComponents. GetNavLinkTraversalCostAlongPath([(0,0,0),(0,0,500)]) returns the link's TraversalCostCm.

CHAPTER 11

Tutorial 11 - LCMNav3D in the Gameplay Debugger

TL;DR: LCMNav3D ships an LCMNav3D category in the stock UE5 Gameplay Debugger. In PIE, press the standard Gameplay Debugger key (usually apostrophe '), select an agent → the category renders per-agent + plugin-wide state alongside Recast / BT / EQS / Perception. Zero project setup beyond the plugin being loaded.

11.1 What it shows

11.1.1 Plugin-wide block (always)

```
== LCMNav3D ==  
  Manager: resolved  
  Loaded chunks: 7  
  NavLink registry: 3  
  NavArea registry: 1  
  NavSystem bridge: active
```

Field	Meaning
Manager	Green when ALcmNavigationManagerSV0 is in the world; red if missing
Loaded chunks	ActiveVirtualChunks.Num for infinite worlds; 1 for finite
NavLink registry	Count of ULcmNavLinkComponent currently registered
NavArea registry	Count of ALcmNavAreaModifier currently registered
NavSystem bridge	Green when the navigation-system proxy is registered with UNavigationSystemV1

11.1.2 Per-agent block (when an actor is selected)

```
== Agent: BP_FlyingPawn_C_0 ==  
  Pos: X=1234.5 Y=678.9 Z=500.0  
  Speed: 624 cm/s  
  Chunk: (0,0,0) full  
  SweepPenalty: 0  
  Perceived: 2 (closest: BP_Energy_C_1)
```

Field	Meaning
Pos	Pawn world location
Speed	Pawn->GetVelocity.Size in cm/s
Chunk	(IntVec3) resolution - full, coarse (low-resolution stub), or none (streaming gap)
SweepPenalty	Risk-field byte at the agent's location (0 to 255)
Perceived	Count of currently-perceived actors + closest threat name

11.2 How to toggle

1. In PIE, press the standard Gameplay Debugger key (usually apostrophe ' - check Project Settings → Engine → Gameplay Debugger if you've rebound it).
2. The Gameplay Debugger HUD appears at the top of the viewport.
3. Look for the LCMNav3D category - it should be enabled by default (state set to EnabledInGameAndSimulate at module init).
4. Click an actor in the level to select it as the debug target - per-agent block populates.

If the category isn't showing:

- Confirm you're in PIE (Gameplay Debugger only runs in PIE/Standalone, not the editor itself).
- Confirm the plugin is loaded (check Output Log for [LcmNavBridge] messages on map load).
- Confirm you're not in a Shipping/Test build (the category compiles out when WITH_GAMEPLAY_DEBUGGER=0).

11.3 What survives PIE → editor → PIE reloads

- **CVars** (debug toggles, chase tuning, OTEL NDJSON, etc.) - persist across PIE sessions, reset on editor restart.
- **Profile loader state** (ULcmNavProfileSubsystem) - re-applied on each PIE init if the profile path is in your project's startup.
- **Telemetry buffer** - flushed at module shutdown; survives PIE-stop but not editor-close.
- **NavLink + NavArea registry** - populated at each actor's BeginPlay; PIE-stop clears via EndPlay.
- **NavSystem bridge proxy** - spawned per-world; new PIE session = new proxy.

The Gameplay Debugger category itself is **always-on after module load** - it doesn't lose its registration across PIE sessions because the category lives in the GameplayDebugger module's category map, not in the world.

11.4 Adding new fields (v2 design hook)

To extend the category in your project subclass:

```
#if WITH_GAMEPLAY_DEBUGGER
class FProjectGameplayDebuggerCategory_Apex: public FGameplayDebuggerCategory_LcmNav3D
{
public:
    virtual void CollectData(APlayerController* PC, AActor* DebugActor) override
    {
        Super::CollectData(PC, DebugActor);
        // Gather project-specific state here
    }
    virtual void DrawData(APlayerController* PC, FGameplayDebuggerCanvasContext& Canvas) override
    {
        Super::DrawData(PC, Canvas);
        Canvas.Printf(TEXT(" ProjectField: %s"), *MyProjectState);
    }
};
#endif
```

Register your subclass instead of the stock one in your project's module startup.

11.5 Performance

- CollectData is called once per tick per active category. Cost ≈ 10 μs (one GetActorOfClass + a few accessors + a perception query when an actor is selected).
- DrawData runs only when the HUD is visible. Cost ≈ negligible (text rendering).
- Shipping builds: **zero cost** (entire category compiles out via WITH_GAMEPLAY_DEBUGGER gate).

11.6 Related LCM Nav3D debug surfaces

- `r.LcmNav.Debug.StateTreeOverlay` - on-screen pawn state DrawDebugString
- `r.LcmNav.Debug.CoverViz` - cover-spot DrawDebugSphere overlay
- `r.LcmNav.Debug.FlowFieldViz` - flow-field arrow grid
- `LcmNavStress.SpawnAgents <N>` - stress-spawn helper for scale testing

CHAPTER 12

Tutorial 12 - Small worlds (centimetre-scale navigation)

TL;DR: the plugin voxelizes in Unreal units (1 uu = 1 cm). A 120 × 70 × 40 cm aquarium with 5 cm cave openings navigates at a 1 cm leaf either way you build it: at 100:1 on any release, or at 1:1 with the *World Scale* profile set to *Small*. Both were measured on the same test tank.

12.1 The two rules that are not obvious

1. **Set half the leaf you want.** The octree halves a half-extent until it drops below `Min Voxel Size`, so the finest leaf lands at 2-4× the value. For a 1 cm leaf set 0.5.
2. **Make the world edge a power-of-two multiple of the leaf.** `World Extent` is a cube half-extent centred on the world origin (the manager actor's location is ignored). With `World Extent` 64 and `Min Voxel Size` 0.5 the halving chain lands exactly on 1 cm; with 60 it lands on 0.94.

12.2 Option A - build at 100:1 (works on every release)

	Real	In Unreal at 100:1
Tank	120 × 70 × 40 cm	12000 × 7000 × 4000 uu
Manager: World Extent	-	6400 (finite)
Manager: Min Voxel Size	-	50
Clearance Padding	fish half-width	half-width × 100
Fish speed	6 cm/s	600 uu/s (the shipped default)

Measured: a 5 cm cave mouth resolved and routed, a plugged cave correctly unreachable, an agent crossing the tank in 3 s.

12.3 Option B - build at 1:1 with World Scale = Small

Set `World Scale` to `Small`, `World Extent` to 64, `Min Voxel Size` to 0.5. `Clearance Padding` is derived for you (half the leaf); the start-rescue distances and the large-node cost threshold scale with the leaf as well. The `Details` panel will note that the world is only 128 leaves across - that is expected for a small world, not a fault. The line-of-sight sampler, the neighbour probe and the route executor's sampling floors follow the leaf on their own, on every profile.

Agents follow too. The **Fly To** task (BT and StateTree) and the flight movement component carry an `Agent Scale` that defaults to 0 = follow the manager's `World Scale`, so under `Small` a fish authored with the shipped drone values (600 cm/s, 100 cm acceptance, 120 cm agent size) flies at 6 cm/s with a 1 cm acceptance and a 1.2 cm agent size, along the same path shape, in the same time. Scale the pawn actor itself (mesh and collision) to match - the plugin scales motion, not geometry. Set `Agent Scale` explicitly on a task when an agent should not follow the world (a large creature in a small world, or the reverse).

One engine-side limit remains for very small bodies: Unreal's swept movement pulls every blocking hit back by an absolute 0.1-1 cm and treats gaps inside the physics contact offset (2 cm minimum) as an initial overlap, so a body whose collision is itself a centimetre or less meets a wall under a different regime than its metre-scale twin. Measured with the bird pawn at agent scale 0.01: the flight is identical to the 100:1 flight until the first wall contact (and, on this tank, arrives sooner after it). Bodies that keep clear of walls, such as the navigator drones, are exact end to end. If wall-hugging behaviour must match a metre-scale reference exactly, build at 10:1 (agent scale 0.1), where the engine tolerances are still small against the body.

12.4 Stones with caves: collision, not the mesh

Solidity is read from **collision**. Set stone meshes to *Use Complex Collision As Simple*; a convex hull or box spanning a cave mouth is solid as far as the plugin is concerned, however fine the leaf. (Measured: the same tank with the stones' collision removed routes straight through them; with complex-as-simple it is byte-identical to the baseline.)

12.5 Props that must not be solid

Grass, backdrops and other dressing a body should pass through need the collision **profile** *NoCollision*, not only *Collision Enabled = No Collision*: a component that keeps the *BlockAll* profile gets its collision back when the map loads, and is then voxelized like stone. (Measured: seagrass saved that way walled two of the demo tank's goals off from a 4 cm body.)

12.6 Mass crowds at small scale

A Mass trait carries no reference-agent constants, so author its values in world centimetres: the demo school uses a 2 cm Agent Radius, 25 cm/s Max Speed and a 6 cm Arrive Radius. The flow field's own centimetre constants (the arrival stop, the goal cell, the obstacle skin and the slide's push cap) follow the World Scale automatically. The *LCM Nav3D ISM Render* trait has *Mesh Rotation* and *Mesh Offset* for a mesh not modelled facing +X; a fish modelled nose-along +Y needs Yaw -90.

12.7 The demo tanks

Demo/BehaviourTree/PreBaked_FiniteMaps/FishTank_BT, Demo/StateTree/.../FishTank_ST and Demo/Mass Entity/.../FishTank_MassEntity are a 120 x 70 x 40 cm aquarium built this way (option B, a 1 cm leaf). Ten 8 cm fish on the Fish archetype roam through a rock arch and round a predator's lair their Prey filter prices x10; in the Mass tank a school of 40 follows a leader fish and parts round a predator that carries its own Territory.

12.8 What to expect at small scale

- **Cost trends volumetric.** In a world a few dozen leaves across nothing is far from geometry *in leaves*, so the octree cannot stay coarse anywhere; node count grew 7.3-7.9× per halving of the leaf on the test tank. The voxel budget (64 M cells per volume) still applies.
- **Unreachable goals are cheap.** Free-space components are labelled at build time, so a goal behind sealed stone fails before any search runs.
- **Bakes land in the plugin folder.** Bake to Asset writes under /LCMNav3D/BakedData/ even for a map in your project. Remove test bakes before packaging; the manager logs a warning when it bakes a map that lives outside the plugin.
- **Infinite mode is not for this.** Its per-chunk budget forces $\text{Chunk Size} \leq 400 \times \text{Min Voxel Size}$; the panel says so if you try.

CHAPTER 13

Tutorial 13 - LCM Fly To Location (no Behavior Tree, no StateTree)

TL;DR: LCM Fly To Location is the flying counterpart of the engine's **AI Move To** node. Call it from any Blueprint - a level script, a pawn's Event Graph, a widget - with a pawn and a destination, and it fires **On Success** when the pawn arrives or **On Fail** with the reason it did not. The pawn needs no AI Controller, no Behavior Tree and no StateTree.

13.1 What you need

- An **LCM Nav3D Manager** in the level, with navigation data built or baked (as for any agent).
- A **pawn**. Either kind works:
 - With an enabled **LCM Flight Movement Component**, the pawn flies with its 6-DOF dynamics - banking, braking, a fish's tail - exactly as it does under the Fly To tasks. The component's *Agent Scale* carries over, so a fish at 0.08 uses its own acceptance, speeds and clearances.
 - Without one, the pawn is moved kinematically at *Flight Speed*.

13.2 Using the node

1. Right-click in a Blueprint graph and search for **LCM Fly To Location** (category *LCM Nav3D | Movement*).
2. Connect **Pawn** and **Destination**.
3. Handle the two exec pins:
 - **On Success** - the pawn has arrived.
 - **On Fail** - it has not; the **Result** pin says why.

Both pins also give **Pawn Location**, where the pawn was when the request ended.

Input	Default	Meaning
Pawn	-	The pawn to fly.
Destination	-	Where to go. Ignored when Target Actor is set.
Target Actor	none	Fly to this actor instead, re-planning as it moves.
Acceptance Radius	-1	Arrival distance in cm. ≤ 0 uses the Fly To default (100 cm, times the agent scale).
Filter Class (<i>advanced</i>)	none	A navigation query filter: which cost areas this pawn avoids or prefers (Tutorial 10).
Chase Moving Target (<i>advanced</i>)	off	Aim where a moving Target Actor is going, not where it is.
Flight Speed (<i>advanced</i>)	-1	Kinematic pawns only: cruise speed in cm/s. ≤ 0 uses the default.
Timeout Seconds (<i>advanced</i>)	0	Give up after this long. ≤ 0 : no limit.

Result	Pin	When
Success	On Success	Arrived.
No Path	On Fail	The destination cannot be reached from where the pawn is: sealed, outside the navigable volume, or excluded by the filter.
Failed	On Fail	The pawn set off but could not complete the route.
Aborted	On Fail	Cancelled, stopped, or replaced by a newer request for the same pawn.
Timed Out	On Fail	Ran longer than Timeout Seconds.
Invalid Request	On Fail	No pawn, or no LCM Nav3D Manager in the pawn's world. Fires at once.

13.3 Stopping and replacing

- **One request per pawn.** A new LCM Fly To Location on a pawn that is already flying ends the old one with *Aborted* and takes over, as AI Move To does.
- **Cancel** (on the node's return value) or **Stop LCM Fly To** (on the pawn) ends the request with *Aborted*.
- **Is LCM Fly To Active** tells you whether a pawn is flying one.

A stopped 6-DOF pawn **holds where it was stopped** rather than cruising on its last command. A body that cannot stop dead - a fish, a bird - glides its stopping distance and comes back. Measured on the demo fish: at most 4-7 cm past the cancel point, and settled within 2 cm of it 4 s later.

13.4 How arrival is judged

- A pawn arrives within **Acceptance Radius** of the destination. If the body reaches its last waypoint's neighbourhood and cannot close the final gap, it settles and completes within **twice** the radius. This is the Fly To tasks' rule.
- With a **Target Actor**, the pawn must reach the **actor**. With *Chase Moving Target* on, the pawn steers at a predicted intercept point, but reaching that point does not count as arriving; it keeps flying until it is within range of the actor itself.
- A path that is still being computed, or a streaming gap in an infinite world, is **not** a failure. The request waits.
- The request pauses with the game.

13.5 One Fly To, not three

The node has no movement code of its own. It runs the StateTree Fly To task's own code - the same planner, cost areas, re-planning, streaming waits, avoidance, speed governor, arrival and flight-component hand-off. A pawn flies the same whether a tree or this node sends it.

Do not drive one pawn from two places at once. A pawn running a Fly To task in a Behavior Tree or StateTree and an LCM Fly To Location at the same time has two sources of guidance; the last one each frame wins. Stop the tree's task (or leave the pawn without a tree) before using the node on it.

13.6 From C++

```
#include "Agent/LCM_FlyToAsyncAction.h"

ULcmFlyToLocationAsyncAction* Fly = ULcmFlyToLocationAsyncAction::FlyToLocation(
    this, MyPawn, FVector(4500, -2000, 800));
Fly->OnSuccess.AddDynamic(this, &AMyActor::HandleArrived); // UFUNCTION(ELcmFlyToResult, FVector)
```

```
Fly->OnFail.AddDynamic(this, &AMyActor::HandleNotArrived);  
Fly->Activate();
```

In C++, bind the delegates **before** calling `Activate()`: an invalid request fires On Fail inside `Activate()`. The Blueprint node does this for you.

13.7 Verified

The battery gate `g35_flyto_node` (`-run=LcmNavCostTest -Group=FlyToNode`) flies a pawn with no controller through the demo fish tank at a 1 cm leaf and the stock navigator pawn across the metre-scale cost showcase. It checks every case above: an invalid request, arrival and a single On Success, never inside rock or glass, No Path, replacement, Cancel and hold, a chase to a moving actor.

CHAPTER 14

Tutorial 14 - Stock Move To and EQS with LCM Nav3D

Goal: make UE's own navigation nodes (**Move To**, **AI Move To**, **Simple Move To**, **Find Path To Location Synchronously**, EQS projection) plan through LCM Nav3D, and check that they do.

14.1 How it works

When a level has an **LCM Nav3D Manager**, the plugin spawns a navigation-data proxy (`LcmNavigationDataProxy`). It then asks UE's navigation system to register the proxy. Once registered, stock queries for the proxy's agent go through LCM Nav3D, and LCM Nav3D plans them in 3D.

UE registers a navigation-data actor only for a **Supported Agent** whose **Navigation Data Class** is that actor's class. The plugin picks that agent for you where it can, and reports what really happened. (Before 3.4 the proxy was refused on every map while the log said "registered".)

14.2 Step 1: pick your setup

Your project	What to do
Only flying agents; no Recast navmesh in the level	Nothing. The proxy claims the default agent on this world's navigation system (<code>r.LcmNav.Bridge.ClaimDefaultAgent 1</code> , the default). Project Settings are never written.
Flying agents and ground characters on a Recast navmesh	Add a Supported Agent for LCM Nav3D (Step 2). The navmesh keeps the default agent.
You want the stock nodes to ignore LCM Nav3D	<code>r.LcmNav.Bridge.AutoRegister 0</code> .

14.3 Step 2: add a Supported Agent (mixed projects only)

1. **Project Settings** → **Engine** → **Navigation System** → **Agents** → **Supported Agents** → +.
2. **Name:** anything, e.g. `Flyer`.
3. **Navigation Data Class:** `LcmNavigationDataProxy`.
4. **Agent Radius / Height:** your flyer's size. UE uses them to pick the agent for a pawn, so make them differ from the ground agent's.
5. On the flying pawn's **Character/Pawn Movement** → **Nav Agent Properties**, use the same radius and height, or set **Preferred Nav Data** to `LcmNavigationDataProxy`.

14.4 Step 3: check it worked

The bridge reports one of these states, in the log and from Blueprint or C++ (`ULcmNavBridgeSubsystem::GetBridgeState() / GetBridgeStatusText()`):

State	Meaning
Registered	Stock queries for the bridged agent go through LCM Nav3D.
NotClaimed	Another navigation data (a Recast navmesh) owns the default agent, and no Supported Agent names the proxy. Do Step 2.
Failed	The navigation system refused the proxy. The status text gives UE's reason.
Waiting / Requested	Not settled yet (the navigation system registers on a later tick), or auto-registration is off.

A warning appears once if the bridge ends in `NotClaimed` or `Failed`. It names the setting to change.

14.5 Cost areas with the stock nodes

A **Filter Class** on Move To (a `NavigationQueryFilter`) works as it does with Recast. Its area costs and exclusions are applied by the LCM Nav3D planner, the same as on the LCM FlyTo nodes. Tutorial 10 covers cost-area volumes and filters.

14.6 Limits

- The stock nodes follow the route with the pawn's own movement component. For 6-DOF flight dynamics (bank, thrust, archetypes), use the **LCM Fly To** BT or StateTree task, or the **LCM Fly To Location** Blueprint node.
- Move To path following in UE is ground-oriented: it projects points and treats acceptance in 2D unless the movement component flies. Give the pawn a flying movement mode, or use LCM Fly To.

Part III

Behaviour Tree Stock Patterns

CHAPTER 15

BT Wait - use stock `UBTTask_Wait`

TL;DR: Unreal Engine ships `UBTTask_Wait`. Use it.

15.1 Pattern

1. In your BT graph: right-click → Add Task → "Wait".
2. Set `WaitTime` (seconds) + optional `RandomDeviation`.
3. Task returns `Succeeded` after the timer.

For a wait that depends on a blackboard-stored time, use `UBTTask_WaitBlackboardTime` instead.

15.2 Why we didn't ship `LCM_BTTask_Wait`

Bit-identical functionality. The LCM Nav3D StateTree variant `FLcmSTTask_Wait` is shipped because StateTree on 5.2 doesn't include a stock Wait task - but BT has one since 4.x. No additive value from a custom LCM Nav3D variant.

15.3 LCM Nav3D-specific wait variant

If you want to wait for a **specific LCM Nav3D condition** (e.g. "wait until the chunk at position X is loaded"), use the LCM Nav3D-unique:

- `LCM_BTTask_ChunkPrewarm` - waits for chunk readiness with a configurable timeout, proactively enqueues the chunk for generation

CHAPTER 16

BT IsAtLocation - use the stock UE5 decorator

TL;DR: Unreal Engine already ships `UBTDecorator_IsAtLocation`. Use it directly on LCM Nav3D pawns. No LCM Nav3D-specific variant exists or is needed.

16.1 Why we didn't ship `LCM_BTDecorator_IsAtLocation`

This audit found that `UBTDecorator_IsAtLocation` already does what an LCM Nav3D-side variant would do:

- Reads a Vector or Object blackboard key
- Computes distance from the pawn to the key
- Returns true if within `AcceptableRadius`
- Optional `bPathFindingBasedTest` reuses the active path-following result

This is the canonical "did the AI reach the location" predicate. Re-shipping it on the LCM Nav3D side would be redundant and force you to pick between two identical nodes.

16.2 Three-step usage on an LCM Nav3D pawn

1. **Drop the stock decorator on a Sequence / Selector node** in your BT graph. Right-click → Add Decorator → "Is At Location".
2. **Set the blackboard key.** Vector key (a goal position) or Object key (an `AActor` whose location is the goal - `bUseNavAgentGoalLocation` projects to nav-agent location). For LCM Nav3D flying pawns where you want the literal world position, use Vector.
3. **Set `AcceptableRadius`** to your "we count this as arrived" distance in cm. Typical: 50-150 cm.

That's it. The decorator works on every LCM Nav3D pawn that has an `AIController` - no LCM Nav3D-specific configuration needed.

16.3 When to use the path-finding variant

`UBTDecorator_IsAtLocation::bPathFindingBasedTest` makes the decorator consistent with whatever was used to reach the location - useful when the move was via `UBTTask_MoveTo` (Recast) so the "arrived" check matches.

For LCM Nav3D moves, this option will fall through to Recast even if your goal was inside an LCM Nav3D volume - leave it OFF and use straight geometric distance.

16.4 Related LCM Nav3D-unique decorators

For predicates that **don't** have a stock equivalent, see the LCM Nav3D-unique decorators we did ship:

- `LCM_BTDecorator_PathLengthBelow` - true-3D LCM Nav3D path-length check (stock `DoesPathExist` is binary + Recast)
- `LCM_BTDecorator_HasLineOfSight` - voxel-aware Lazy Theta* LOS (stock uses raycasts only)

-
- LCM_BTDecorator_IsInCover - composite SweepPenalty + LOS predicate
 - LCM_BTDecorator_ChunkLoaded - SVO streaming probe
 - LCM_BTDecorator_ThreatVisibleViaPerception - AI Perception + LCM Nav3D LOS composite

These are genuinely additive surfaces; "IsAtLocation" is not - use stock.

BT LookAt - choosing between the stock task and the LCM Nav3D one

TL;DR: UE ships `UBTTask_RotateToFaceBBEntry`, and for a ground agent it is the right answer. For a flying agent it is not, because both its success test and its applied rotation ignore the vertical axis. LCM Nav3D now ships `LCM_BTTask_LookAt` and `LCM_BTTask_TurnInPlace` for that case.

17.1 Which one to use

Situation	Use
Ground agent, target roughly level with it	Stock <code>UBTTask_RotateToFaceBBEntry</code>
You want facing to persist across several tasks	Stock task plus <code>UBTService_DefaultFocus</code>
Target can be above or below the agent	<code>LCM_BTTask_LookAt</code>
You need a constant turn rate rather than an eased one	<code>LCM_BTTask_TurnInPlace</code>
You need the turn to hold position while it happens	Either LCM task, with <code>bHoldStation</code>

17.2 The stock task, and what it does with the vertical axis

The stock task is well suited to walking agents and integrates with the `AIController` focus system, which is why it is the default recommendation in most UE projects. Two details matter once the agent can fly.

Its success test is two dimensional. The task measures alignment with `FVector::CosineAngle2D`, which projects both directions onto the XY plane before comparing them. A target directly above the agent therefore has an angle difference of zero: the task reports success immediately, with the agent still pointing level.

Its applied rotation drops pitch by default. The task rotates the pawn through the controller focus system, and `APawn::FaceRotation` keeps the pawn's existing pitch unless `bUseControllerRotationPitch` is set. On a default pawn that flag is false.

Together those mean that, for a target overhead, the stock task can neither point the agent at it nor notice that it has not.

None of this is a defect in the stock task. It is built for agents that move on a surface, where yaw is the only axis that matters.

17.3 The LCM Nav3D tasks

`LCM_BTTask_LookAt` eases toward the target; `LCM_BTTask_TurnInPlace` turns at a constant rate. They are otherwise the same node with a different rate mode, and both:

- measure the real 3D angle between the agent's forward vector and the target direction, so a pure pitch error reads as a pitch error;
- apply pitch as well as yaw;

- optionally hold position during the turn (`bHoldStation`), so a body with momentum does not drift while it rotates;
- optionally require line of sight, and **wait** rather than fail while navigation data is still streaming in.

Property lists are in the API Reference.

17.4 Pattern: face a target, then act

Sequence

```
LCM LookAt      (TargetKey = ThreatActor)
<your attack task>
```

`bFailOnTimeout` defaults to false, so a turn that runs out of time still succeeds and the sequence continues. Set it true if a failed turn should abort the sequence.

17.5 Pattern: keep facing while doing other things

This is still the stock pattern and LCM Nav3D does not replace it:

1. Add `UBTService_DefaultFocus` to the parent Selector or Sequence.
2. Set its focus blackboard key.
3. The controller holds focus for the whole subtree.

⚠ **Do not run a focus service and a rotation task at the same time.** The `AIController` re-asserts focus rotation every tick, so it will fight any task that writes rotation itself, including the LCM tasks. Pick one: a focus service for continuous facing, or a rotation task for a discrete turn. Running both has been observed to stall the agent outright.

17.6 On 6-DOF flight bodies

If the pawn carries `LcmFlightMovementComponent` with the flight dynamics locomotion model, that component owns the body's orientation and derives it from its own dynamics every tick. A rotation written by any task, stock or LCM, is overwritten in the same frame.

The LCM tasks detect this and report the facing as unsupported rather than pretending to have turned; the task still succeeds so that a sequence containing it continues to work. Agents using the default kinematic locomotion model are unaffected and rotate normally.

17.7 See also

- `LCM_BTTask_MoveToAndFace` - travel to a point and turn to face something on arrival, as one task.
- `LCM_BTTask_Hover` - hold position while facing a target, for a fixed time or indefinitely.

CHAPTER 18

BT TurnInPlace - constant-rate turning, and the stock focus pattern

TL;DR: For continuous “keep facing while doing other things”, use the stock `UBTService_DefaultFocus`. For a discrete turn at a constant rate, use `LCM_BTTask_TurnInPlace`, which ships with the plugin.

See `BT_LookAt_StockPattern.md` for the full comparison between the stock rotation task and the LCM Nav3D ones. The short version: the stock task's success test projects onto the XY plane and its applied rotation drops pitch, so on a flying agent with a target overhead it can report success while still pointing level.

18.1 Pattern: continuous facing during other tasks

If you want the agent to keep facing a target while executing other tasks in a parent sequence, this is the stock pattern and LCM Nav3D does not replace it:

1. Add `UBTService_DefaultFocus` to the parent Selector or Sequence.
2. Set the focus blackboard key (Vector or Object).
3. The `AIController`'s focus stays set for the entire subtree, so the pawn faces the target through `SetFocus` while other tasks execute.

This is the canonical UE pattern for “shoot while strafing” combat AI.

⚠ **A focus service and a rotation task will fight each other.** The controller re-asserts focus rotation every tick, overwriting anything a task writes, and this has been observed to stall an agent outright. Use a focus service for continuous facing or a rotation task for a discrete turn, not both on the same subtree.

18.2 Pattern: a discrete constant-rate turn

`LCM_BTTask_TurnInPlace` turns toward the target at `MaxRotationRateDegPerSec` and succeeds once it is pointing within `AcceptableErrorDegrees`.

Sequence

```
LCM TurnInPlace    (TargetKey = ThreatActor, MaxRotationRateDegPerSec = 180)
<your next task>
```

Constant rate is the difference from `LCM_BTTask_LookAt`, which eases toward the target instead. Use the constant rate when the turn speed is a character trait you want to be visible and consistent, for instance a heavy gun platform that traverses slowly. Both tasks share the same underlying implementation and the same properties otherwise.

`bHoldStation` defaults on, so the agent holds its position through the turn rather than drifting.

18.3 Property list

See the API Reference entry for `LcmBTTask_TurnInPlace`.

18.4 On 6-DOF flight bodies

Where `LcmFlightMovementComponent` is driving the pawn with the flight dynamics locomotion model, that component owns orientation and derives it from its own dynamics. The task reports the facing as unsupported rather than pretending, and still succeeds so surrounding sequences continue to work. Agents on the default kinematic locomotion model rotate normally.

CHAPTER 19

BT HealthBelow - use the stock blackboard + decorator pattern

TL;DR: Unreal Engine ships everything you need: a Float blackboard key + a Blackboard-Compare decorator. No LCM Nav3D variant.

19.1 Pattern

1. In your **Blackboard asset**: add a Float key `CurrentHealth`.
2. In your **AI controller / game code**: write `CurrentHealth` whenever damage is applied.
3. In your **BT graph**: add a Blackboard decorator to the "retreat" / "panic" sequence. Set:
 - Key: `CurrentHealth`
 - Notify Observer: On Value Change (so the decorator re-evaluates immediately on damage)
 - Operator: Is Less Than
 - Value: `30.0` (or whatever threshold)

That's the canonical UE5 way of doing "if health below X, do Y."

19.2 If you want a GameplayTag-based pattern

For projects using GAS where health states are tags (e.g. `Status.Bleeding`, `Status.LowHealth`), use the stock `UBTDecorator_CheckGameplayTagsOnActor`. Same semantics, tag-driven.

19.3 Why we didn't ship `LCM_BTDecorator_HealthBelow`

The stock blackboard-compare decorator covers the exact same functionality with one extra dropdown (the comparison operator) - which actually gives you more flexibility (`>` `<` `<=` `>=` `==`) than a fixed "below threshold" LCM Nav3D decorator. Use the stock decorator.

The LCM Nav3D StateTree variant `FLcmSTCondition_HealthBelow` is kept for ergonomic parity (named clearly in the StateTree node picker), but its underlying behaviour is also "float compare" - stock-equivalent.

19.4 Related LCM Nav3D-unique decorators

If your "should I retreat" gate isn't health-based but **threat-field-based**, the LCM Nav3D-unique surface gives you better options:

- `LCM_BTDecorator_IsInCover` - composite `SweepPenalty` + LOS predicate
- `LCM_BTService_ThreatFieldSampler` - writes `CurrentThreatLevel` + `bInThreatZone` into BB every 0.2s; then a stock blackboard decorator can gate on `bInThreatZone`

CHAPTER 20

BT PerceptionRelay - bind OnTargetPerception Updated directly

TL;DR: Unreal Engine's `UAIPerceptionComponent` already emits perception updates as a delegate. You don't need a custom BT service - just bind the delegate in your AI controller and write to a blackboard key.

20.1 Pattern (in your AIController subclass)

```
// In your AIController.h
UPROPERTY()
TObjectPtr<UAIPerceptionComponent> PerceptionComp;

UFUNCTION()
void OnPerceptionUpdate(AActor* Actor, FAIStimulus Stimulus);

// In your AIController.cpp BeginPlay
PerceptionComp = FindComponentByClass<UAIPerceptionComponent>();
if (PerceptionComp)
{
    PerceptionComp->OnTargetPerceptionUpdated.AddDynamic(
        this, &AYourAIController::OnPerceptionUpdate);
}

void AYourAIController::OnPerceptionUpdate(AActor* Actor, FAIStimulus Stimulus)
{
    if (UBlackboardComponent* BB = GetBlackboardComponent())
    {
        if (Stimulus.WasSuccessfullySensed())
        {
            BB->SetValueAsObject("ClosestThreat", Actor);
            BB->SetValueAsBool("bHasAnyTarget", true);
        }
        else
        {
            // Lost sight - Leave ClosestThreat for memory pattern (see below)
            BB->SetValueAsBool("bHasAnyTarget", false);
        }
    }
}
```

That's the canonical UE5 way. Three lines in your AIController + a Blackboard decorator in the BT.

20.2 Why we didn't ship `LCM_BTService_PerceptionRelay`

A custom service that ticks each frame and re-queries `GetCurrentlyPerceivedActors` is less efficient than binding the **event-driven** `OnTargetPerceptionUpdated` delegate. The stock pattern is also more idiomatic - every UE5 AI tutorial uses this pattern.

The LCM Nav3D StateTree variant `FLcmSTEval_PerceptionRelay` is kept because StateTree has no per-state perception delegate (its evaluators are tick-based by design). On the BT side, delegate binding is the standard.

20.3 Memory pattern (last-known-location with age)

UE5's perception component already tracks last-known location + age per perceived actor via `FActorPerceptionInfo::GetLastStimulusLocation(float* OutAge)`. To use it:

```
if (const FActorPerceptionInfo* Info = PerceptionComp->GetActorInfo(*Actor))
{
    float Age = 0.f;
    FVector LastKnown = Info->GetLastStimulusLocation(&Age);
    if (Age < 8.f) // 8 sec memory window
    {
        BB->SetValueAsVector("LastKnownThreatLoc", LastKnown);
    }
}
```

See `BT_MemoryDecay_StockPattern.md` for the full pattern. **Don't** roll your own decay timer - perception's age tracking already does it for you.

20.4 Related LCM Nav3D-unique services

For perception-aware predicates that need **LCM Nav3D LOS** (volumetric, not raycast):

- `LCM_BTDecorator_ThreatVisibleViaPerception` - composite: perception sees + LCM Nav3D Lazy Theta* LOS is clear
- `LCM_BTService_SquadSnapshot` - aggregates perceived allies into squad centroid / cohesion radius

CHAPTER 21

BT MemoryDecay - use FActorPerceptionInfo::GetLastStimulusLocation(&OutAge)

TL;DR: Unreal Engine's UAIPerceptionComponent already tracks per-actor last-known stimulus location with age. No need to roll your own decay timer.

21.1 Pattern (in your AIController or a small custom service)

```
UAIPerceptionComponent* Perception = GetAIPerceptionComponent();
if (!Perception) return;

// Pull the last-known info for a specific actor (e.g. the threat you're tracking).
if (const FActorPerceptionInfo* Info = Perception->GetActorInfo(*ThreatActor))
{
    float Age = 0.0f;
    FVector LastKnown = Info->GetLastStimulusLocation(&Age);

    // Apply your project's memory window.
    if (Age < MemoryWindowSec)
    {
        Blackboard->SetValueAsVector("LastKnownThreatLoc", LastKnown);
        Blackboard->SetValueAsFloat("MemoryAge", Age);
        Blackboard->SetValueAsBool("bHasMemory", true);
    }
    else
    {
        // Memory expired.
        Blackboard->SetValueAsBool("bHasMemory", false);
    }
}
```

That's it. Perception's age tracking handles refresh-on-stimulus + decay automatically.

21.2 How to integrate as a BT service

Wrap the snippet above in a UBTService_BlueprintBase subclass (Blueprint-only - no C++ needed):

1. Create a BP service. Override Receive Tick.
2. Inputs: ThreatActor (Object BB key) + MemoryWindowSec (float).
3. Inputs to write: LastKnownThreatLoc (Vector BB key) + MemoryAge (Float BB key) + bHasMemory (Bool BB key).
4. Tick body: get perception component → GetActorInfo → GetLastStimulusLocation(&Age) → branch on Age vs window → write BB keys.

Five nodes total. No C++ required.

21.3 Why we didn't ship LCM_BTService_MemoryDecay

The stock `FActorPerceptionInfo::GetLastStimulusLocation(&OutAge)` provides everything our planned MemoryDecay service would - location + age tracked **per sense, per actor**, automatically refreshed on stimulus and aged each frame.

Rolling our own decay timer would:

- Duplicate the perception component's bookkeeping
- Miss the per-sense granularity (perception tracks separately for Sight / Hearing / Damage / etc.)
- Force you to maintain two parallel "last seen" caches

The LCM Nav3D StateTree variant `FLcmSTEva1_Memory`. Same conclusion on both sides.

21.4 Related LCM Nav3D-unique services

If you want **squad-level memory** (where multiple allies' perceived-actor lists are aggregated), use `LCM_BTService_SquadSnapshot` - that's a genuinely LCM Nav3D-additive surface.

Part IV

Reference

LCM Nav3D - Editor Tools

Everything here lives behind the **Nav3D** button in the Level Editor toolbar. The panels are also under **Window ▸ Level Editor**.

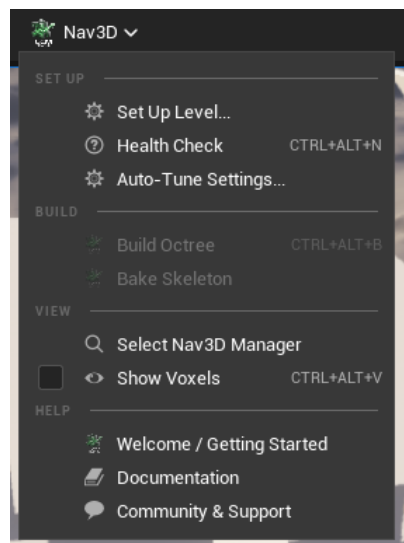


Figure 1 - The Nav3D toolbar menu. Four groups, in the order you need them. Greyed entries are not broken: **Build Octree** is finite-only and **Bake Skeleton** appears only when something will actually read a bake, so the menu never offers you an action that would be an expensive no-op.

22.1 Quick start

1. **Nav3D ▸ Set Up Level...** - creates and configures the manager, picks finite or streamed navigation to match your map, attaches navigation invokers, and builds.
2. **Nav3D ▸ Health Check** - dock it and leave it open while you work.
3. Add the **LCM Nav3D FlyTo** task to a Behavior Tree or StateTree, or drop the Mass crowd traits on a spawner.

Nav3D actors are also in the **Place Actors** panel under **LCM Nav3D**.

22.1.1 Keyboard shortcuts

Shortcut	Action
Ctrl+Alt+N	Health Check
Ctrl+Alt+B	Build Octree
Ctrl+Alt+V	Show Voxels

Rebind or clear them under **Editor Preferences ▸ Keyboard Shortcuts**.

22.2 The Welcome screen

Opens once per plugin version, and any time from Nav3D ► **Welcome / Getting Started.**



Figure 2 - Welcome screen. The part worth reading is **REQUIRED PLUGINS**: it checks the dependencies declared in `LCMNav3D.uplugin` against what your project actually has enabled, and offers an **Enable** button on any that are off. If one is missing, the plugin's modules will fail to load - this panel is where you find that out, rather than from a startup error.

Links under **HELP & COMMUNITY** are read from the `.uplugin` descriptor, so the UI and the store listing cannot drift apart. A greyed link means that field is empty in the descriptor, not that the link is broken.

22.3 The Manager panel

Select the **LCM Nav3D Manager** in your level. Above the numbered settings sections you get a header card with live state pills (world model, residency, voxelizer), the context-appropriate build actions, and two readouts worth knowing about.

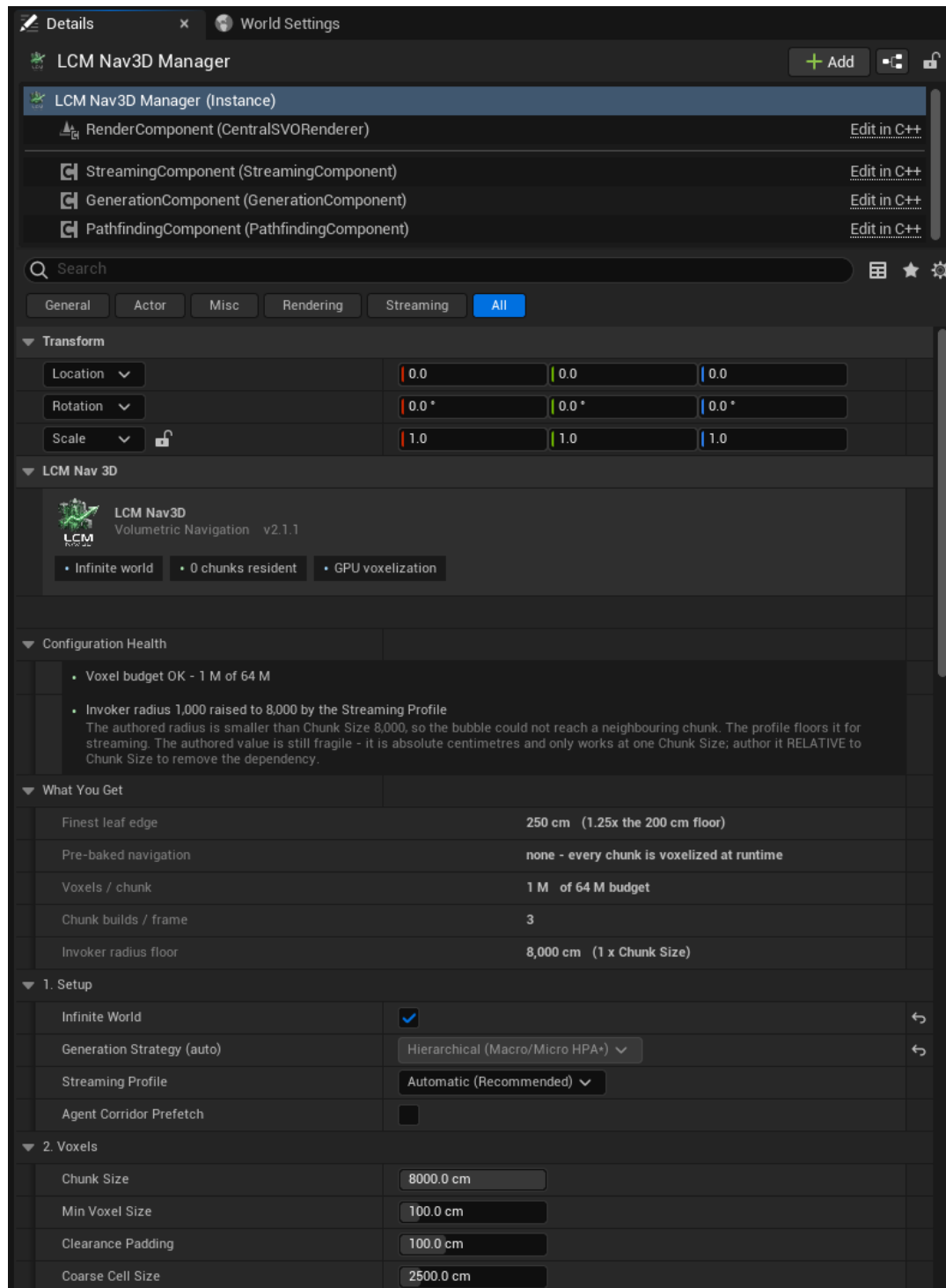


Figure 3 - Manager Details, top half. Read it downwards:

- **The pills** - world model, chunks resident, voxelizer. Three facts at a glance.
- **Configuration Health** - expands only while something is wrong. In this shot it is reporting that the authored invoker radius of 1,000 cm was *raised to 8,000* by the Streaming Profile, because a radius below one Chunk Size leaves an agent with no neighbouring chunk and therefore no route at all.
- **What You Get** - the derived numbers you would otherwise have to know a rule to predict. **Finest leaf edge 250 cm** with Min Voxel Size at 100 is not a bug: the octree halves a chunk until the child

drops below your voxel size, so the leaf is $\text{chunkSize} / 2^k$ and lands between $2\times$ and $4\times$ the value you typed.

22.3.1 The build actions

Only the buttons that mean something in your world model are shown - a finite world has no chunks to skeletonize, and a streamed world has no single SVO to save.

Button	World	What it does
Build SVO (Editor Preview)	Finite	Voxelizes now, into memory. EQS queries and the debug draw resolve without entering Play. Thrown away when you close the editor - it is a preview, not a build product
Bake to Asset	Finite	Writes the SVO to a <code>ULcmSVODataAsset</code> and attaches a streaming proxy that loads it. The shipped game stops voxelizing the world at every launch
Bake Skeleton	Infinite, and only when Use Baked Skeleton is on	Whole-map coarse connectivity, so long-range routing is complete from frame 0

Bake to Asset uses the manager's own `World Extent`, `Min Voxel Size`, `Clearance Padding` and collision filters, so the bake cannot describe a different world than the runtime expects. It records what it was baked *against*, which is what lets the health section tell you later that the bake has gone out of date. On success it also turns **Auto Build On Play** off: leaving both on is not belt-and-braces, because the proxy and the manager each write the navigation data during `BeginPlay` with no ordering guarantee between actors - which one wins is undefined, and when the rebuild wins you pay the full load-time stall the bake exists to remove.

Re-bake after moving level geometry or changing the voxel settings.

Save the level after baking. The asset is written to disk immediately, but the streaming proxy that *loads* it is a level actor. Close without saving and you are left with a bake nothing reads - and `Auto Build On Play` cleared on an actor that was discarded, so the level has no navigation at all.

The LCM Nav3D Bake Volume actor is the other way to bake, and it is for a different job - see Bake Volumes below. For a single finite world, the manager button is simpler and cannot disagree with the runtime settings. Both write the same asset format through the same code path.

22.3.2 Configuration Health

Setting *combinations* that no per-property clamp can express, checked live and shown where the settings are. Rows come in three tiers: $\triangle$ a real hazard, i a property worth knowing, $\checkmark$ an all-clear.

$\triangle$ Warning	Why it matters
Voxel budget	Exceeding it does not fail - generation silently coarsens <code>Min Voxel Size</code> until it fits, so you get less precision than you configured
Clearance Padding vs Min Voxel Size	Padding inflates every solid surface, so padding $\geq 2\times$ voxel size seals narrow openings. The right value is your agent radius , which only you know
Invoker radius vs Chunk Size	A radius under one chunk size leaves an agent with no neighbouring chunks, so no route can be planned and nothing further streams

⚠ Warning	Why it matters
Residency demand	An invoker bubble is a cube: chunks scale as $(2R / \text{ChunkSize})^3$, so halving Chunk Size costs $8\times$ the chunks
Baked asset out of date	A bake is a frozen snapshot and nothing at runtime can tell it stopped matching the level. It still loads - you just get agents avoiding open space or routing through geometry, with nothing logged
Auto Build On Play overrides the bake	Both write the navigation data during <code>BeginPlay</code> with no ordering guarantee, so which one the level uses is undefined
i Information	What it tells you
Finest leaf vs the floor	The finest leaf is $\text{ChunkSize} / 2^k$, landing anywhere in $[2\times, 4\times] \text{Min Voxel Size}$ non-monotonically . Not a defect - see below

Why the leaf ratio is information, not a warning. It depends only on `Chunk Size` (it is identical at every `Min Voxel Size`), so it really asks "is `Chunk Size` a power-of-two multiple of the voxel floor" - and the Auto-Tuner's own candidate grid fails that test on most of its entries. Whether a coarse leaf actually *hurts* - a sealed opening, an unroutable pair - cannot be decided from the settings, and is exactly what the tuner measures. So the panel states the fact and lets the measurement carry the verdict.

What the staleness check can and cannot see. It compares the voxel settings the bake ran with, and an order-independent hash of every collidable actor inside the baked bounds - identity, transform, scale and local bounds. So it catches geometry **moved, added, removed or rescaled**, which is nearly always what "the level changed" means. It does **not** walk source triangles, so a mesh edited in place - same actor, same transform, new geometry - hashes identically. Re-bake after editing a mesh. Assets baked before this signature existed report as unverifiable rather than stale.

When a ⚠ row is present the section offers one action: **Tune Settings...**, which opens the Auto-Tuner. There are deliberately no per-warning "apply this value" buttons - such a value would be *derived*, and this system's search space is non-monotonic, so only measurement can answer it.

These are the same checks that have always been written to the Output Log - the log is now a consumer of the same rules, so the two can never disagree.

22.3.3 What You Get

The values your settings actually resolve to, rather than the ones you typed:

Row	Note
Navigable volume	Finite only. The real world-space bounds - see the note below
Finest leaf edge	The real smallest voxel, with its ratio to the achievable floor ($2 \times \text{Min Voxel Size}$)
Voxels / chunk (or / world)	Against the hard budget
Chunk builds / frame	Derived from the Streaming Profile
Invoker radius floor	What the profile raises an invoker's radius to, if anything

The finite volume is centred on the world origin, not on the manager actor. `World Extent` is a half-extent, so the volume spans $[-\text{WorldExtent}, +\text{WorldExtent}]$ on each axis around $(0,0,0)$. The

manager is a logical singleton with no root component, so **moving it does not move the navigable volume**. Build your playable space around the origin, or use an infinite world.

22.3.4 Why the panel is short

The manager has no geometry, no collision and no transform, so the inherited Actor sections that cannot apply - Rendering, Collision, Physics, LOD, Cooking, Input, HLOD, Mobile, Ray Tracing, Texture Streaming, Virtual Texture - are hidden from this panel. They still exist on the class; they are just noise here.

Replication, **Actor** (tags) and **World Partition** are deliberately kept: the manager hosts multicast RPCs, tags are a legitimate way to find it from gameplay, and `Is Spatially Loaded` genuinely matters for a singleton that must not stream out.

22.4 Bake Volumes

Drop an **LCM Nav3D Bake Volume** from the Place Actors panel and resize the brush to cover the area you want baked. Its panel uses the same language as the manager's - header pills, one action, Configuration Health, What You Get.

22.4.1 I want two separate areas navigable. Which setup?

Grid mode + Infinite World. Nothing else does it. This is the single most common wrong turn, so it is worth being blunt about why:

A **finite** world has exactly **one** navigation volume - one slot, in memory. A single-volume Bake Volume writes it. The manager's **Bake to Asset** writes it. Both write it at the same chunk coordinate, so whichever ran **last** wins and the other area has no navigation at all. **Overlapping the two does not join them** - there is nothing in a finite world that could hold both. Baking area A, then baking area B, and finding only B navigable is the system working exactly as a finite world is defined, not a bug in the bake.

What you want	Setup
One area	Either producer - a single-volume Bake Volume or the manager's Bake to Asset. Not both
One bigger area	Enlarge the volume, or raise the manager's <code>World Extent</code>
Two or more separate areas	<code>Snap To Global Grid</code> on each volume + <code>Infinite World</code> on the manager

The manager's Bake to Asset now **refuses** to run while a single-volume Bake Volume is in the level, rather than silently replacing its output, and both Details panels flag the pair.

22.4.2 One volume or many?

This is the `Snap To Global Grid` switch, and the answer differs because the two settings feed **different runtime containers**:

<code>Snap To Global Grid</code>	Produces	Manager must be	How many volumes
Off	One chunk, sized to this volume	Finite world	Exactly one
On	One chunk per grid cell	Infinite world	As many as you like

Off feeds the manager's single navigation slot, so a second single-volume bake has nowhere to go: both write the same asset name at the same chunk coordinate, and only the last one baked survives. To cover more ground with one volume, enlarge it - or raise the manager's `World Extent`.

On feeds a coordinate-keyed container that holds any number of chunks. This is the multi-region workflow, and it needs *Infinite World* on the manager. With a *finite* manager every proxy is routed into the one slot instead, so each chunk silently overwrites the last. The header pill turns amber when the two disagree, and Configuration Health says which way to resolve it.

"Infinite World" names how navigation is *stored and streamed*, not whether the data is generated at runtime. A fully baked, grid-aligned world is still an infinite-mode world - the chunks just arrive from disk instead of the voxelizer.

22.4.3 Do several connected volumes route across their seams?

Yes. Chunks are keyed by grid coordinate, so which volume produced a chunk does not matter; adjacent chunks connect the same way whether they came from one volume or four. What matters is that every volume uses the **same** `Chunk Size`, `Min Voxel Size` and `Clearance Padding` (the panel checks all three), and that they are grid-aligned, which `Snap To Global Grid` guarantees.

Seams are **scanned at bake time** and shipped with each chunk, so long-range routing across a chunk boundary is correct from the first frame.

Assets baked before 2026-08-03 are different, and the panel will tell you. They carry no seam data, so each boundary loaded as *one full-face portal* - a portal asserting the whole shared face was passable without checking the voxels, corrected only once an agent physically entered the chunk. A walled seam therefore looked *open* to long-range routing. The failure was a route that exists on the macro graph but not in the world, which is why it survived casual testing. **Re-bake to fix it;** Configuration Health reports how many chunks are still in that state.

Very large grids are the one exception. Scanning seams needs the whole grid resident at once (a portal is a property of the boundary *between* two chunks), so past `r.LcmNav.Bake.MaxMacroChunks` (default 1024) the bake falls back to writing chunks one at a time with no seam data. That is logged, and the completion dialog says so. Raise the cap if you have the memory, or bake in smaller volumes.

22.4.4 What the volume panel checks

Every rule here is about **two actors disagreeing**. The bake volume carries its own copy of the voxel settings and nothing reconciles them at runtime, so a mismatch does not fail - it bakes, it loads, and navigation is computed against a world that is not there.

⚠ Warning	Why it matters
Bake mode vs manager world model	Decides whether the chunks land in the single slot or the keyed container - the table above
<code>Chunk Size</code> ≠ the manager's	Baked chunks are placed at <code>coordinate × the runtime Chunk Size</code> , so a mismatch puts every chunk in the wrong place <i>and</i> makes the streamer regenerate the map at load
<code>Min Voxel Size / Clearance Padding</code> ≠ the manager's	The bake describes a differently-shaped world, and the staleness check compares against the <i>manager's</i> values - so the bake reads as out of date the moment it is written
Two single-volume bake volumes	Same asset name, same coordinate: silent overwrite
Voxel budget per chunk	Exceeding it coarsens <code>Min Voxel Size</code> silently, which is what voxelizes narrow openings shut
More than 512 chunks	The bake is synchronous on the game thread with no cancel, and writes one asset each

| Baked chunks without precomputed seams | Assets from before 2026-08-03, or past the macro-bake cap - see the seam section above. Re-bake | | The manager also bakes this volume | Two producers, one finite

slot - see the top of this section | | Oblong brush in single-volume mode | An octree root is a **cube**, so the bake inflates to the largest axis on all three and pays voxels for the remainder |

22.4.5 Sizing the brush, and why the numbers don't match it

Three things surprise people here, so the panel states all three under **What You Get**:

- **Brush Settings size × Transform scale.** Both scale the volume, and the readout shows the product. It is one box, controlled from two places.
- **Single-volume mode bakes a cube.** An octree root cannot be oblong, so the bake uses the **largest** half-extent on every axis: a 100 × 100 × 10 m brush bakes a 100 m cube, and the voxel budget is charged for all of it. That is why the budget row can dwarf the box you drew. Keep the brush roughly cubic, or switch to grid mode.
- **Grid mode snaps outward.** The brush is sliced on the *global* chunk lattice, so the baked region grows to whole chunk boundaries - straddling one by a centimetre costs an entire extra row of chunks. The **Actually voxelized** row shows the real region and count.

Chunk Size is hidden unless Snap To Global Grid is on, because it is not read otherwise - resolution comes from Min Voxel Size in both modes. When it *is* shown, it is snapped to a whole multiple of Min Voxel Size as you type (chunk edges must land on voxel boundaries), so the panel reports the value actually in use, and flags it if the manager disagrees.

i Information	What it tells you
No obstacle channels set	Empty falls back to WorldStatic only ; anything solid on another channel bakes as open air

There is deliberately **no fix button** here. Every warning is two settings disagreeing, and either side may be the one that is wrong - that is a choice only you can make.

22.4.6 After baking, the debug view shows the result

Baking from a volume used to draw nothing while the manager's own build drew immediately, which looked like the bake had failed. It had not - the debug renderer reads the manager's *in-memory* navigation, and a bake writes assets and spawns proxies that inject nothing until BeginPlay. The bake now loads its own output back into the editor manager, so both paths look the same. Tick Draw Debug Volumes on the manager to see it.

22.5 Setup Wizard

One page, because every control feeds the same live readout.

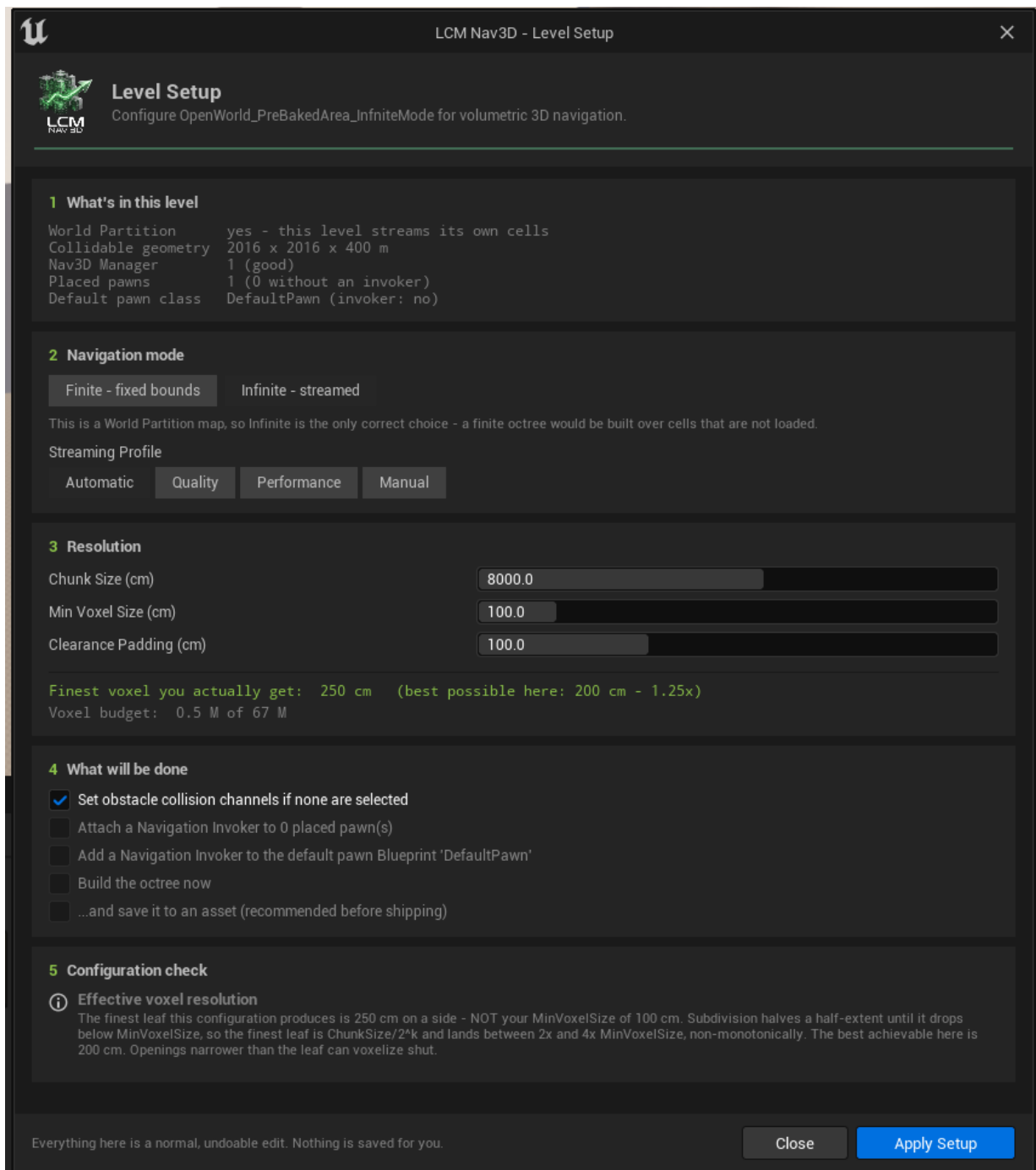


Figure 4 - Setup Wizard. Note what section 1 has already worked out on its own: this is a World Partition map, so section 2 tells you Infinite is the only correct choice and says *why* - a finite octree would be built over cells that are not loaded. Section 4 is an explicit checklist, so nothing happens that is not written on screen, and section 5 re-runs the configuration check against the settings you are *proposing* rather than the ones currently saved.

Section	What it does
What's in this level	World Partition, geometry size, manager count, pawns, the GameMode's default pawn
Navigation mode	Finite vs Infinite, plus the Streaming Profile
Resolution	Chunk Size / World Extent, Min Voxel Size, Clearance Padding
What will be done	An explicit checklist - nothing happens that isn't listed

Section	What it does
Configuration check	Live problems, updating as you change settings

Under **What will be done**, Build the octree now has a companion: ...**and save it to an asset**. That is the same voxelization with its result *saved* rather than discarded - the wizard's route to the Bake to Asset flow. It is off by default, unlike the other actions: those are reversible edits to settings, while this one writes content to /Game/LcmNavigation/BakedData and adds an actor to your level. Save the level afterwards.

22.5.1 The resolution readout

Finest voxel you actually get: 250 cm (best possible here: 200 cm - 1.25x)

This is the most important line in the wizard. The finest leaf is $\text{ChunkSize} / 2^k$, so it lands between **2x** and **4x** your Min Voxel Size and moves **non-monotonically** with Chunk Size. Measured at Min Voxel Size 100:

Chunk Size	Finest leaf
3000	375 cm
5000	312 cm
7000	219 cm
8000	250 cm

A smaller chunk can give you a **coarser** voxel. Openings narrower than the leaf voxelize shut, which is why "it works at one Chunk Size and not another" is a real effect and not a bug in your level.

Everything the wizard does is one undo step. Nothing is saved for you.

22.6 Health Check

A dockable panel that reports anything which would stop agents moving, with a **Fix** button on each row.

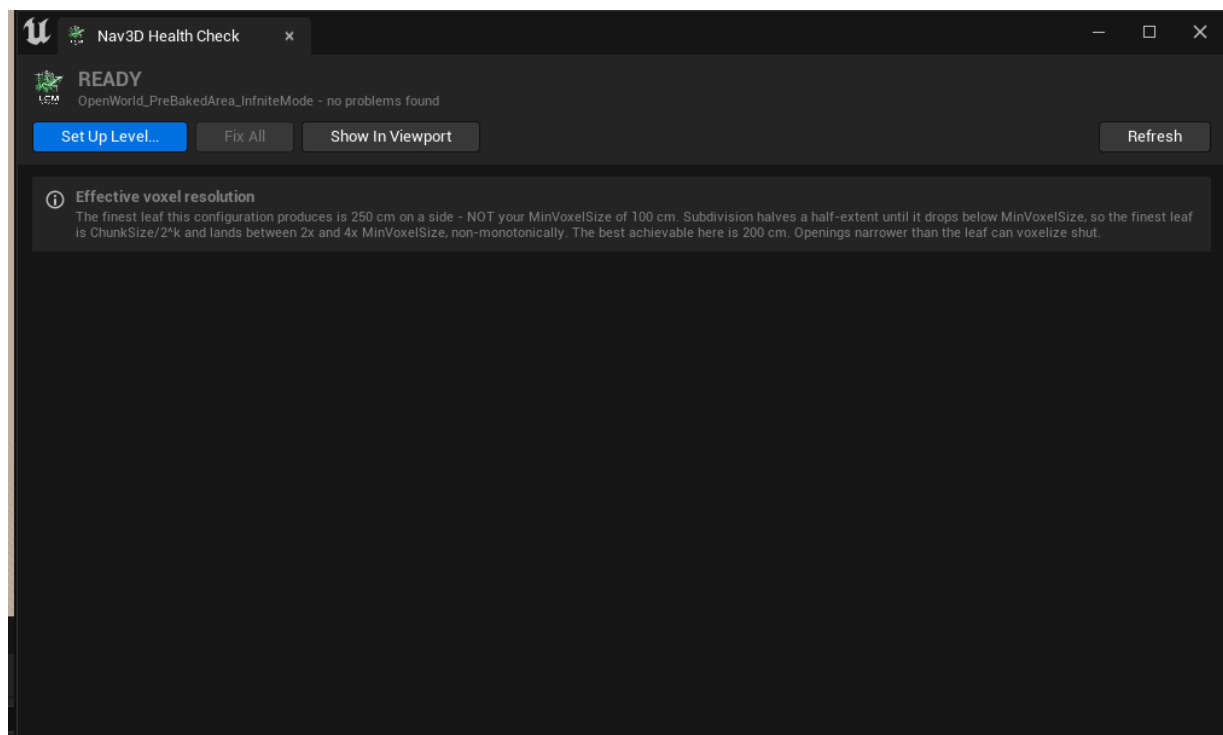


Figure 5 - Health Check, on a level that passes. The verdict is one word - **READY, WORKS, WITH WARNINGS** or **NOT READY** - coloured and sized so it reads from across the room. **Fix All** greys out when there is nothing to bulk-fix, which is the state you want to end up in and then stop thinking about. Dock this panel and leave it open; it re-inspects itself whenever the map, its actors or any Nav3D setting changes.

- **Fix All** applies every automatic repair, *except* ones that edit a Blueprint asset - those keep their own button, so changing your content is always a deliberate click.
- **Show In Viewport** draws the findings that have a *position*, for 30 seconds: the finite navigable volume in blue, your level's geometry bounds in red when they extend past it, and a marker over every pawn missing a Navigation Invoker. Read-only; it changes nothing.
- Refresh is automatic on map open, actor add/delete, and Nav3D setting changes.

★ **The volume box is worth looking at once.** World Extent is a *half* extent and the box is centred on the **world origin**, not on the Manager actor - the Manager has no root component, so dragging it 500 m away moves nothing. Everyone reads that in the tooltip; nobody believes it until they see the box sitting somewhere other than around their level.

Typical findings: no manager, more than one manager, a World Partition map set to Finite, no navigation invoker anywhere, over the voxel budget, Clearance Padding wide enough to seal openings.

Invoker radius below one Chunk Size is the classic one. The agent has no neighbouring chunks, so no route can be planned, so nothing further streams - and it looks exactly like "the AI refuses to move". The Automatic, Quality and Performance streaming profiles floor it for you; Manual does not.

22.7 Auto-Tuner

Builds the octree at each candidate setting, runs **your** routes through the real planner, and ranks what it measured.

It runs as three steps, shown as a rail across the top:

1 Routes to test → 2 What to measure → 3 Measured results

- **Back is always available.** The loop is measure → look → adjust → measure again, so going back to fix a route is never punished.
- **Forward is gated,** and a disabled **Next** always states why in the footer: no routes yet, no Nav3D Manager in the level, or Play is running.
- A step is **ticked** in the rail once it is done. Step 3 only ticks when a configuration was actually *recommended* - a sweep where nothing completed every route is a finished run and an unfinished job.
- On step 2 the footer button is the run: **Run Measurement**, which advances to the results when it finishes. On step 3 it becomes **Measure Again**.

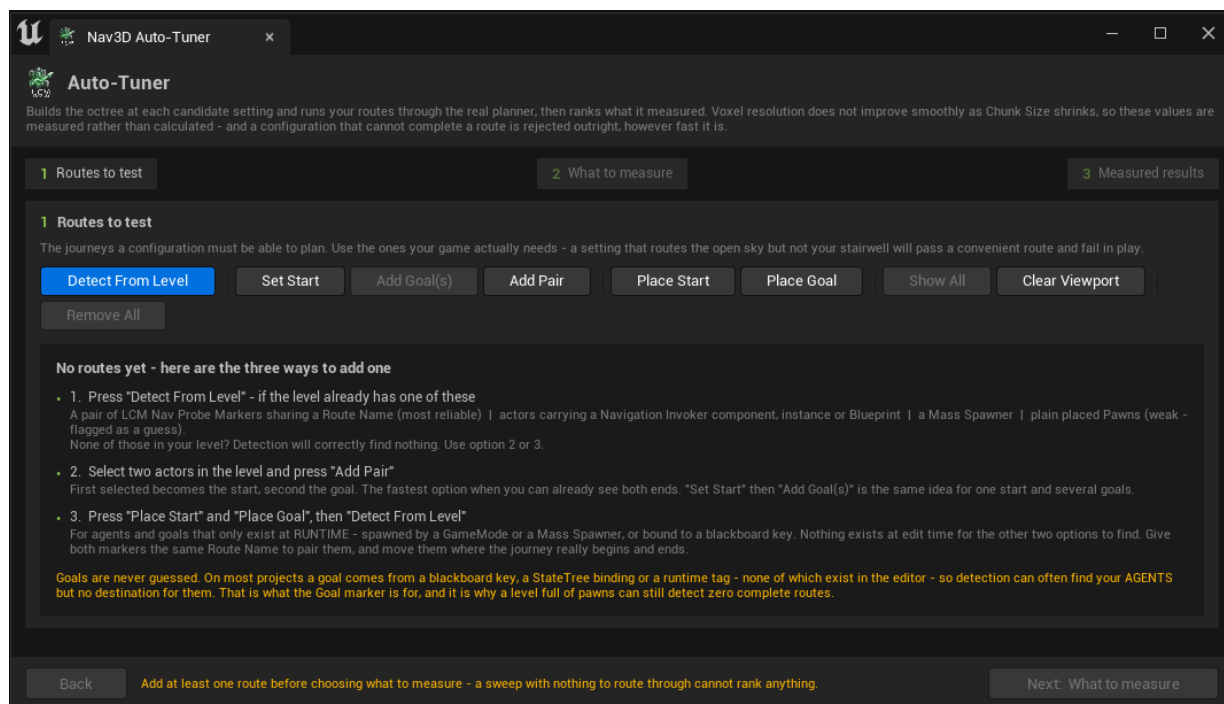


Figure 6 - Step 1, before you have added anything. The empty state is the screen every user sees first, so it does the teaching: three numbered ways in, each naming *what must already be in your level* for it to work. Note the footer - the **Next** button is disabled and the reason is printed beside it in amber rather than hidden in a tooltip.

22.7.1 Choosing routes

A route is a journey your agents actually make. **Every one must complete** for a configuration to be considered, so use the journeys your game depends on - a setting that routes the open sky but not your stairwell will pass a convenient route and fail in play.

Start with **Detect From Level**. The toolbar above the list is grouped by intent, left to right:

Group	Buttons	When
Find them for me	Detect From Level	Always try this first. Appends; never replaces hand-built routes, and skips duplicates.
Build from selection	Set Start, Add Goal(s), Add Pair	You can see the endpoints in the level.
Place by hand	Place Start, Place Goal	Agents or goals only exist at runtime.
View	Show All, Clear Viewport	Check the endpoints before spending minutes on a sweep.
Discard	Remove All	-

22.7.2 What "Detect From Level" needs

Detection reads specific things out of the level. If none of them are there it correctly finds nothing - so this is the checklist for making it work, in descending order of confidence:

It looks for	You get
A pair of LCM Nav Probe Markers sharing a Route Name	The most reliable result - you stated the route explicitly
Actors carrying a Navigation Invoker component (instance <i>or</i> Blueprint)	Start points, from the actual Nav3D agent signal
An AMassSpawner	A start at the spawner's location
Plain placed Pawns	Starts, flagged as a <i>weak</i> guess

★ **Goals are never guessed, and this is the usual reason detection "does nothing".** On most projects a goal comes from a blackboard key, a StateTree binding or a runtime tag - none of which exist at edit time. So a level full of pawns can detect plenty of *agents* and still produce zero complete routes. That is exactly what the **Goal marker** is for.

When detection adds nothing, the panel says which of those cases you are in and what to do about it, rather than reporting "0 routes added".

Detection **appends** - it never replaces routes you built by hand, and it skips duplicates by endpoint.

22.7.3 Building routes by hand

- **Add Pair** - select exactly two actors; the first becomes the start, the second the goal. The fastest option when you can already see both ends.
- **Set Start / Add Goal(s)** - pin one start, then add many goals against it. While a start is pinned the panel shows a chip naming it, with an **Unpin** button; Add Goal(s) is disabled until one is pinned.
- **Probe markers** - place a **Start** and a **Goal** marker and give both the same **Route Name**, then press Detect. This is how you tune projects whose agents and goals only exist at runtime. An unpaired Goal marker pairs with every detected agent.

Each route carries its **own agent width** (diameter, cm), editable on its row - a gap a drone fits through may be unusable for a gunship, so one sweep can answer for a mixed fleet. 0 applies no size filtering.

Press **Show** on any route to highlight it in the viewport with the others drawn faintly for context.

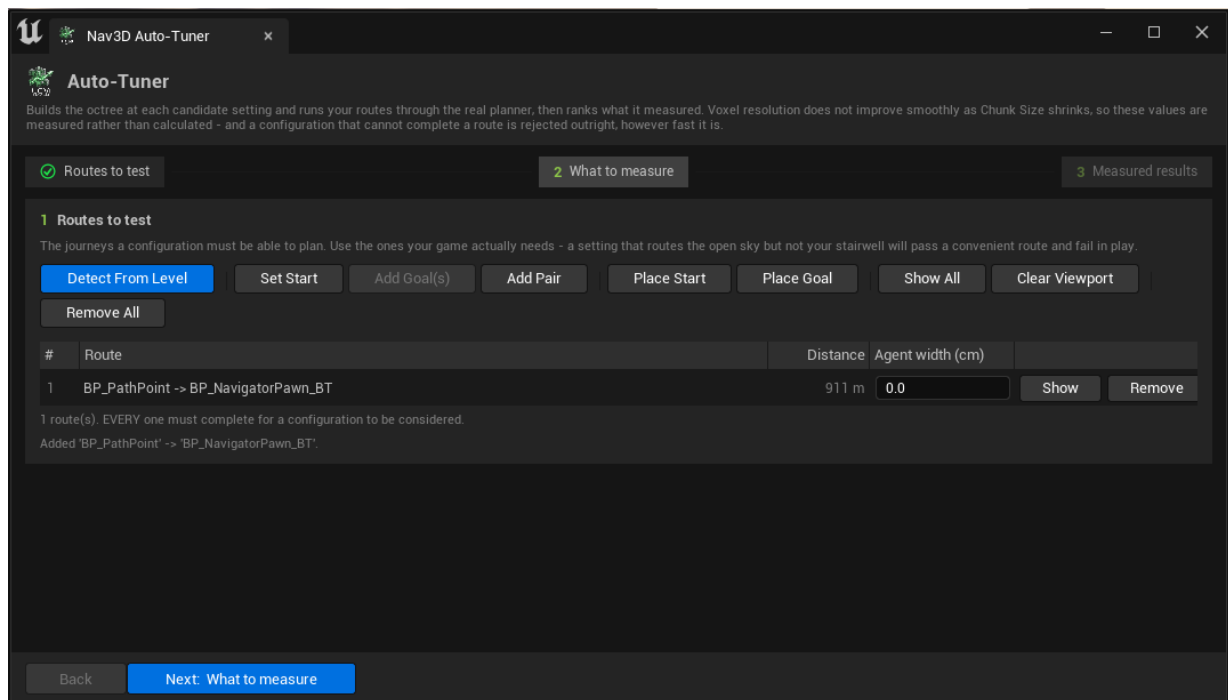


Figure 7 - Step 1, with a route added. Add Pair named it after the two actors it came from, in order, so a reversed route is obvious rather than something you debug later. Two things changed in the chrome the moment the route existed: step 1 in the rail gained a **tick**, and the footer's **Next** turned live.

22.7.4 Step 2 - What to measure

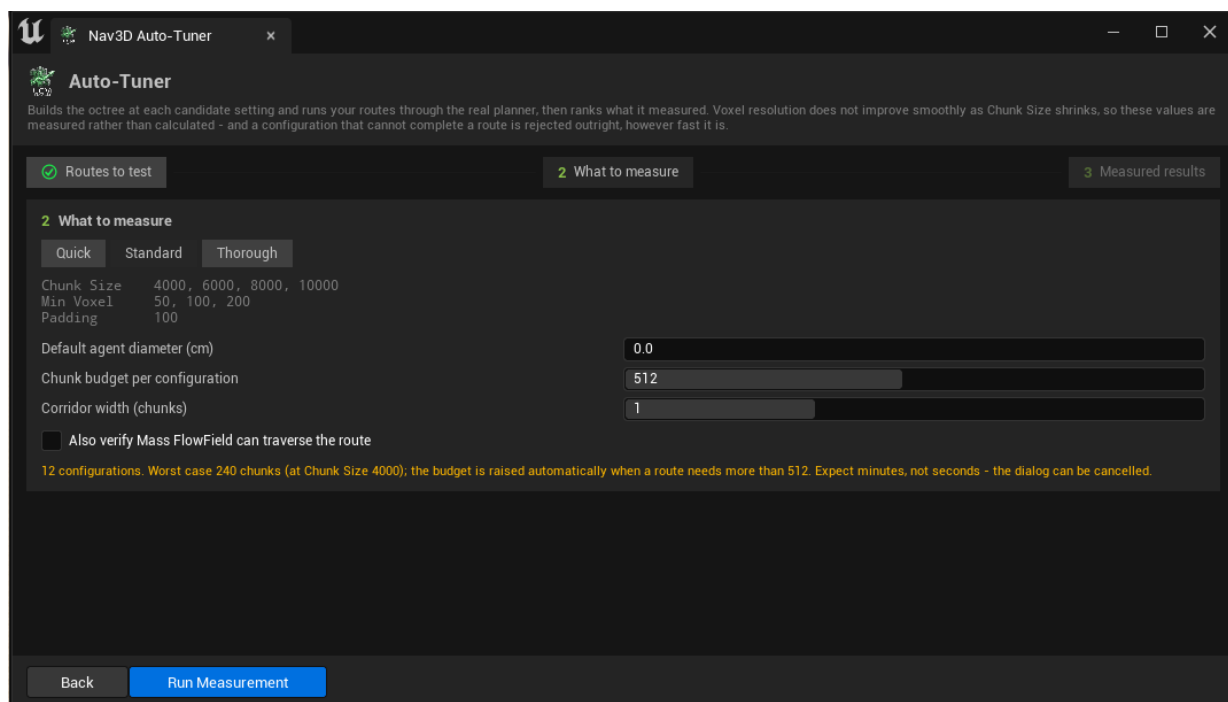


Figure 8 - Step 2. A preset never leaves you guessing what it means: the exact Chunk Size, Min Voxel and Padding values it will try are printed underneath it. The amber line is the honest cost estimate - configuration count, worst-case chunk count and *which* Chunk Size produces that worst case, because a smaller chunk needs more of them for the same route.

Leave **Also verify Mass FlowField can traverse the route** off unless you ship a FlowField crowd. Behaviour Tree, StateTree and Mass *PathFollowing* all dispatch through the same planner the tuner already queries,

so they are covered either way; FlowField is a different solver and costs an extra field solve per chunk per route.

22.7.5 How it ranks

1. **Completeness is a hard gate.** A configuration that cannot plan a route is rejected however fast it is. "Complete" means the path *reaches the goal*, not merely that the query returned success.
2. Among the survivors you get three picks - **Best Quality** (finest measured voxel), **Fastest** (cheapest to build and hold) and **Recommended** (the finest voxel within twice the cheapest one's memory).

The full matrix is always shown, so you can disagree on the merits. Click any column header to sort by it - the three picks keep their badges wherever they land, and sorting back to no column restores the ranked order. **Copy Table** puts every measured row on the clipboard as tab-separated text, which is a far better thing to attach to a support thread than a screenshot.

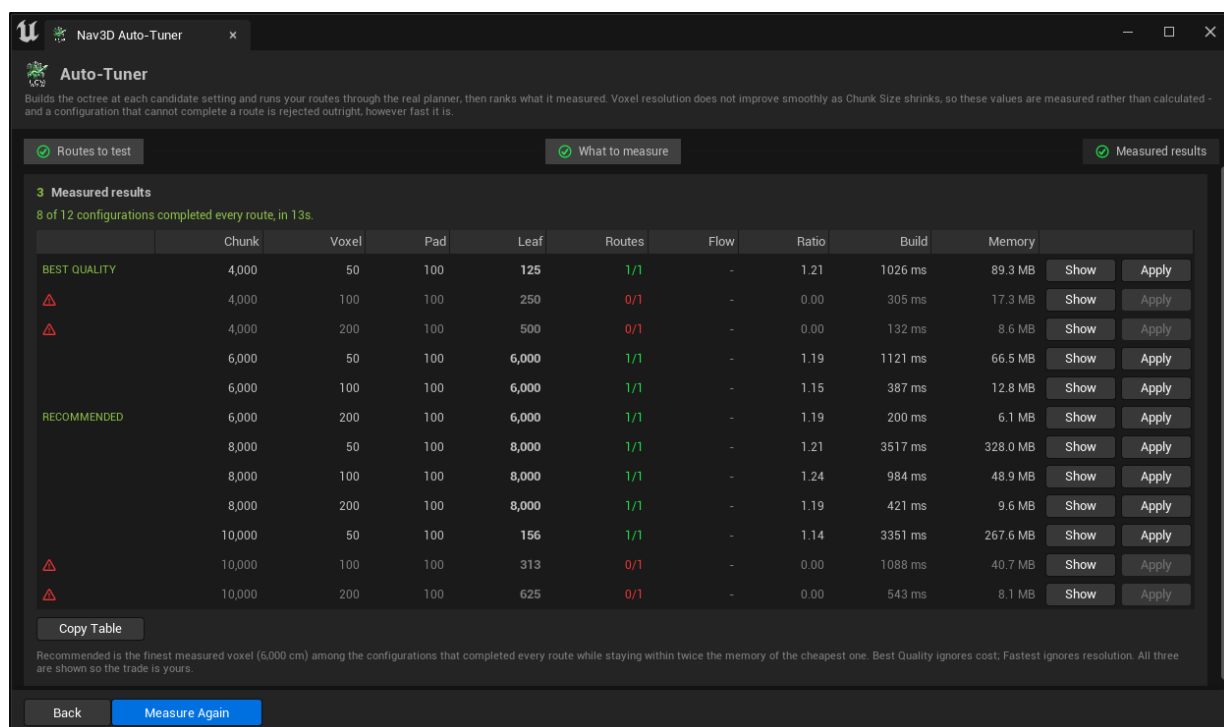


Figure 9 - Step 3, measured results. How to read a row:

Column	Read it as
Routes	The hard gate. Green 1/1 survived; red 0/1 is out no matter how good the rest of the row looks.
Leaf	The measured finest voxel - the number that decides what fits through a gap. Sort by it to rank quality.
Ratio	Path length ÷ straight-line distance. 1.00 is a straight line; nothing can be lower.
Build / Memory	What the configuration costs to produce and to hold. Sort by Memory to rank cost.
⚠ icon	This row could not route, or its build hit the chunk budget. Hover the row for the reason.

Show is enabled on failed rows too, where **Apply** is not. That asymmetry is deliberate: applying a broken configuration would be wrong, but *looking* at one is the whole point - the red rings it draws mark where each route gave up, and that position is usually the answer.

22.7.6 Seeing the routes

A route you cannot see is a route you cannot check, and the single mistake that invalidates a whole sweep - an endpoint sitting inside a building or under the terrain - measures as *"no configuration can route this level"*, which looks exactly like a real finding.

- **Show In Viewport** (routes card) draws every route for 30 seconds: **sphere = start**, **box = goal**, thin grey line between them. The line is the straight line, not a path - it makes no claim that a route exists.
- **Show** (on any measured row) draws the waypoints that configuration *actually* produced. Completed routes are green.
- A route that stopped short is marked with **red rings at the last waypoint**, and a thin line to the goal it never reached.

★ **The red rings are usually the whole answer.** A route that gives up against a doorway, vent or stairwell was not blocked by your level - it was blocked by **Clearance Padding**, which inflates every solid surface and seals openings narrower than the leaf voxel. Lower the padding and re-measure before concluding the level cannot be navigated.

Show is deliberately enabled on failed rows too, where Apply is not: applying a broken configuration would be wrong, but *looking* at why it broke is the point.

22.7.7 What it does and does not cover

Behavior Tree, StateTree, Mass PathFollowing	Covered - all use the same planner
Mass FlowField	Covered only with "Also verify Mass FlowField..." ticked - it is a different solver
Streaming (invoker radius, builds per frame)	Not tuned - that is the Streaming Profile's job
Dynamic obstacles	Voxelized where they currently sit; their motion is not simulated

Park moving obstacles in the positions that most constrain navigation before tuning, and set each route's **agent diameter** to whatever actually flies it.

A sweep voxelizes the level once per configuration. Expect minutes, not seconds. The progress dialog can be cancelled, and your settings are restored either way - nothing changes until you press **Apply** on a row.

22.8 Project Settings

Project Settings ▶ **Plugins** ▶ **LCM Nav3D**. Project-wide only - anything that can differ between two levels lives on the Manager actor, where it is saved with the level.

22.8.1 Content Locations

Setting	Default
Baked Navigation Path	/Game/LcmNavigation/BakedData
Baked Skeleton Path	/Game/LcmNavSkeletons

Changing a path does **not** move an existing bake, but it does not orphan one either: the loaders probe the configured path first and the shipped default after, so an already-baked level keeps loading. Re-bake to migrate it properly.

22.8.2 New Level Defaults

What the **Setup Wizard** proposes for a level with no manager yet - Chunk Size, World Extent, Min Voxel Size, Clearance Padding, Streaming Profile and the obstacle collision channels. Set these once and "the way we set up a level here" stops being a convention someone has to remember.

They are *proposals*. Measured level geometry still overrides the world extent, an existing manager's authored values still win, and nothing here rewrites a level that is already configured.

If your project puts level geometry on a **custom collision channel**, adding it here is the difference between a one-time setup step and a bug on every new level. An empty channel list voxelizes the world as open air, and that failure is silent - open air passes every clearance filter and every quality metric.

22.8.3 Console Variable Overrides

A map of `r.LcmNav.*` names to values, applied at startup. This is the escape hatch for the ~345 console variables the page deliberately does *not* curate: pinning one here puts it in `DefaultGame.ini`, so it is version-controlled, survives a restart, and is visible to the whole team.

- Applied at **project-setting priority** - they beat a variable's built-in default and *lose* to the command line or anything typed into the console, so a local debug override is never stamped back over.
- A name that is not a registered variable is **warned about** in the log, not silently ignored.

Most of these variables are diagnostic or advanced switches, and their correct shipping value is the default. Check first whether the behaviour you want is already a real property on the Manager or on the FlyTo task - it usually is, and a property is saved with the asset that needs it instead of applied to everything.

22.9 Keyboard shortcuts

Shortcut	Action
Ctrl+Alt+N	Health Check
Ctrl+Alt+B	Build Octree
Ctrl+Alt+V	Show Voxels

Rebind or clear them under **Editor Preferences ▶ Keyboard Shortcuts**.

22.10 Where things are configured

Setting	Where
World type, voxel sizes, collision channels	Nav3D Manager, Details panel
Streaming aggressiveness	Manager ▶ Streaming Profile
Per-agent size, smoothing, solver	The FlyTo task / Mass trait
Output paths, new-level defaults, pinned CVars	Project Settings ▶ Plugins ▶ LCM Nav3D
Everything else	<code>r.LcmNav.*</code> console variables

Settings Reference: LcmNavigationManager SVO

Every field on the navigation manager, in the order the Details panel presents them. Defaults and limits below are the shipping values.

If you only read one thing: the four settings that decide whether the plugin works well in your level are **Infinite World**, **World Extent / Chunk Size**, **Min Voxel Size**, and **Clearance Padding**. Everything else is refinement.

Concepts behind these are explained in Tutorial 02: Core Concepts.

23.1 The panel at a glance

The manager's Details panel is one **LCM Nav3D** group floated above the inherited Actor sections, then seven numbered sections in the order below.

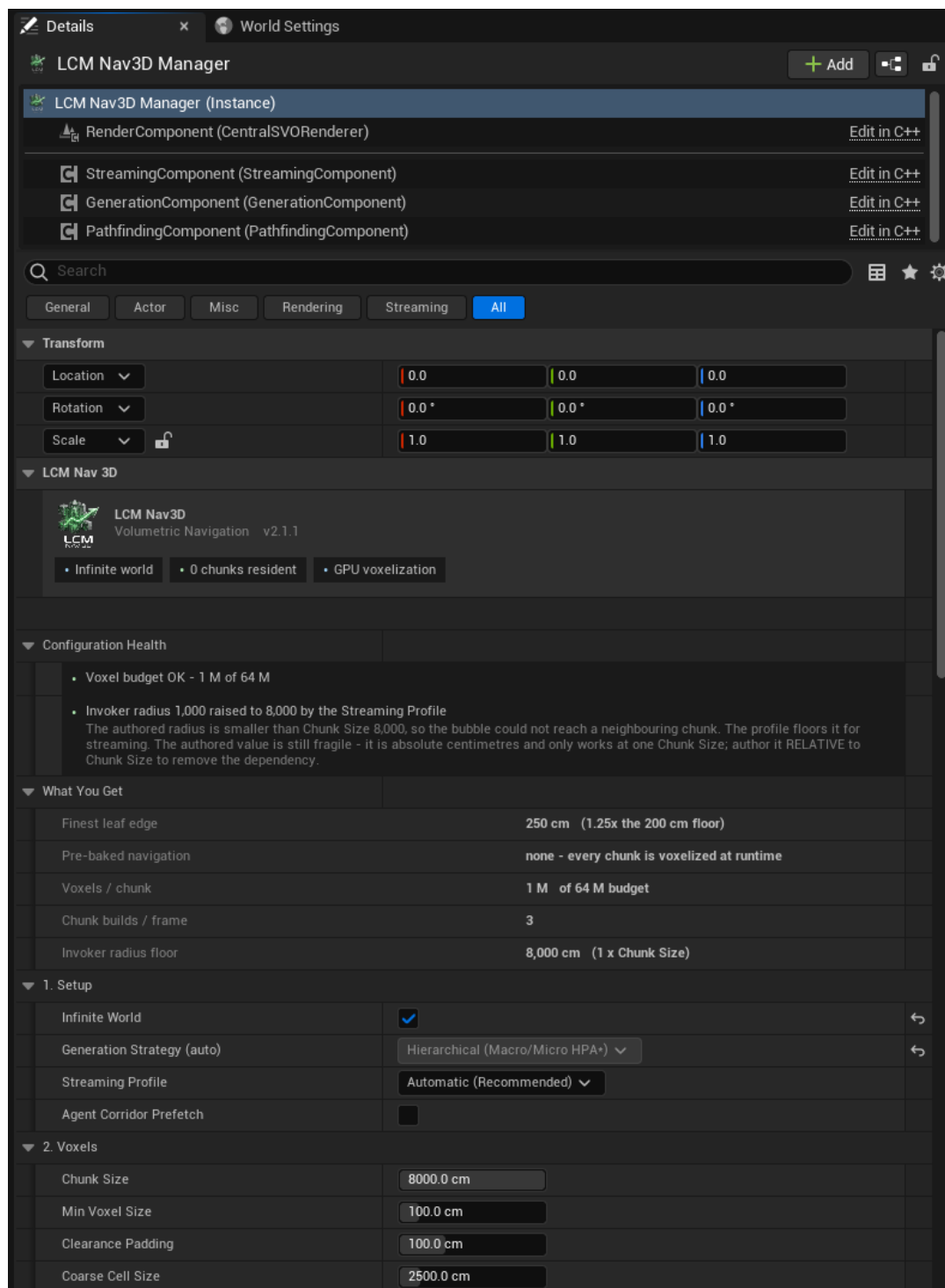


Figure 1 - Sections 1-2, plus the three blocks above them. *Configuration Health* and *What You Get* are not settings; they are what the settings below *resolve to*. They are worth reading before you change anything - the health block in this shot is reporting a raised invoker radius, and the *What You Get* block is reporting a 250 cm finest leaf from a 100 cm Min Voxel Size, which is the single most misread number in the plugin.

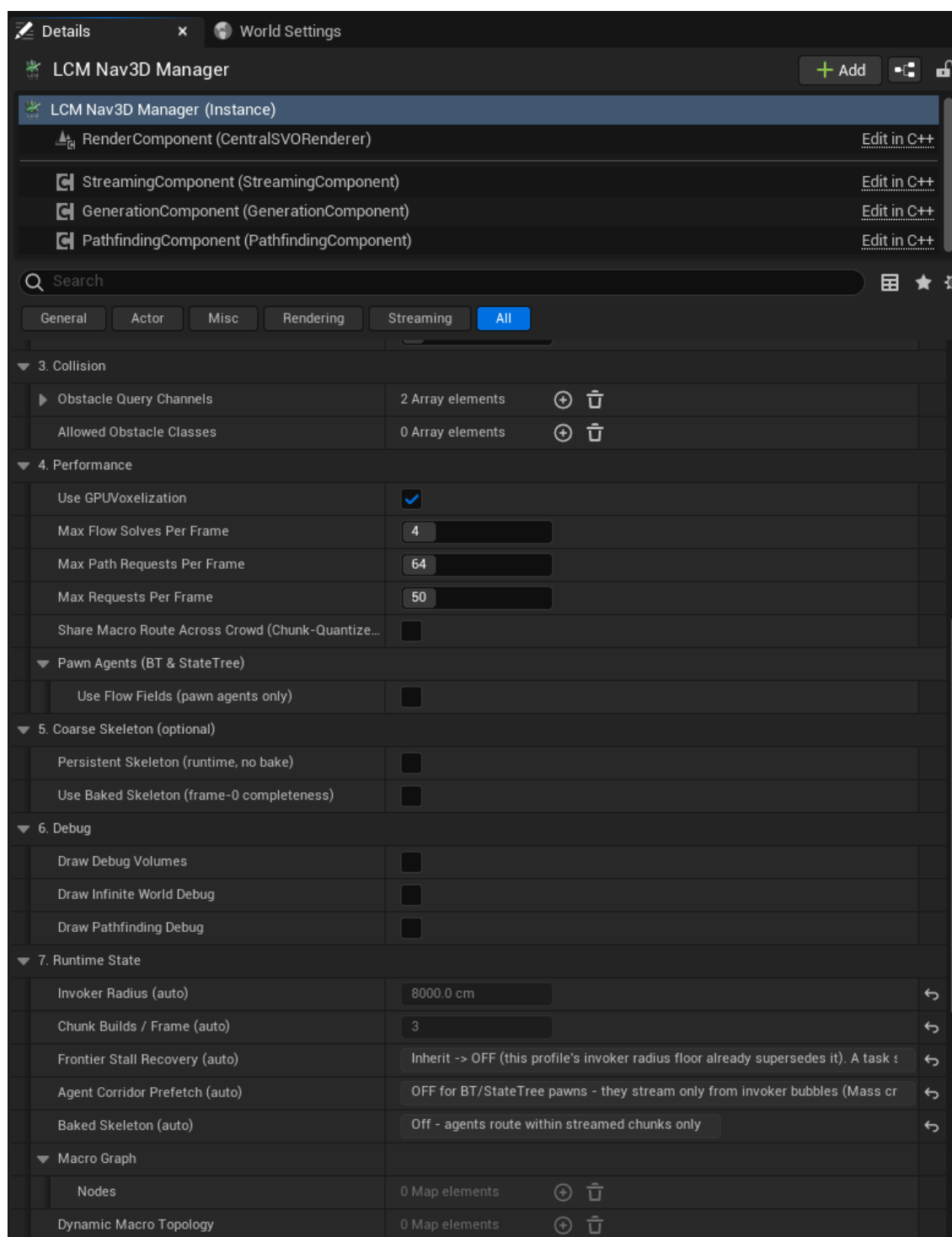


Figure 2 - Sections 3-7. Two things to notice:

- **Section 7 is read-only.** Every (auto) row is a value the Streaming Profile *derived*, shown so the profile is auditable rather than magic. Frontier Stall Recovery (auto) is the only place a BehaviorTree task's Inherit can be resolved for you, because a BT asset is not owned by a level and cannot know which manager it will run under.
- **Sections hide what does not apply.** A finite world shows no Coarse Skeleton options, and Max Generations Per Frame disappears unless you are finite or on the Manual profile - the plugin does not show you a property it is going to ignore.

23.2 1. Setup

23.2.1 Infinite World: default off

The biggest decision. Off = one fixed navigation volume built up front. On = the world is divided into chunks generated around agents as they move.

Leave it off unless you have a genuinely unbounded or streaming world. Finite is simpler, faster and more predictable.

Ticking this hides `World Extent` and reveals `Chunk Size` and the skeleton options.

Levels that stream in late. When a streaming level loads after the navigation around it was already built, the chunks it overlaps are rebuilt automatically, so its geometry does not stay open air. Three console variables control this:

Console variable	Default	Meaning
<code>r.LcmNav.Streaming.RevoxelizeOnLevelStreamIn</code>	1	Rebuild the chunks a newly streamed level overlaps. 0 turns it off
<code>r.LcmNav.Streaming.RevoxelizeGraceSeconds</code>	2	Chunks built within this many seconds after the level arrived are rebuilt too
<code>r.LcmNav.Streaming.RevoxelizeMaxPerTick</code>	8	Most chunks released for rebuild per frame, so the work is spread out

23.2.2 Auto Build On Play: default on (*finite only*)

Builds the navigation volume at `BeginPlay` - voxelizing the whole world, on every launch, on every player's machine. Turn it off if you want to trigger `BuildSVO` yourself from Blueprint at a specific moment, or if you have baked the SVO to an asset.

Turn it off when you bake. The **Bake to Asset** button does this for you, and it is not a tidiness measure: the streaming proxy and the manager both write the navigation data during `BeginPlay`, and actor order is not guaranteed, so with both enabled which one the level ends up using is undefined. The Configuration Health section flags the combination.

23.2.3 Generation Strategy: read-only (*infinite only*)

Shows which streaming strategy was selected automatically.

23.3 2. Voxels

23.3.1 World Extent: default 10000 cm (*finite only*)

Half-width of the navigation volume, measured outward from the **world origin**. The default gives a 200 m cube spanning -10000 ... +10000 on each axis.

Range: 500-400000. Recommended UI range (slider): 2000-20000.

Anything outside this box has no navigation data. Paths to targets outside it fail. Size it to cover your play space.

How big can it actually be? It depends on Min Voxel Size, not on a fixed distance. The real limit is a voxel *count*: the world is $2 \times \text{World Extent}$ wide and holds $(2 \times \text{World Extent} / \text{Min Voxel Size})^3$ voxels against a 64 M ceiling - which is 400^3 . So the largest legal half-extent is $200 \times \text{Min Voxel Size}$.

Min Voxel Size	Largest World Extent	World size
50 cm	10 000	200 m cube
100 cm (default)	20 000	400 m cube
200 cm	40 000	800 m cube
500 cm	100 000	2 km cube

The property clamp used to be a flat **20000**, which is the row for the *default* Min Voxel Size and wrong everywhere else - it blocked a perfectly legal 40 000 at 200 cm voxels, and allowed 8× over budget at 50 cm. A per-property clamp cannot express a rule that depends on another property, so the clamp is now just a sanity ceiling and **Configuration Health reports the real budget live** at whatever Min Voxel Size you are using. Exceeding it does not fail - the generator silently coarsens Min Voxel Size to fit, so you get less precision than you configured.

Trading voxel precision for area is a real choice, not a workaround: a 2 km flying-combat arena at 500 cm voxels is a sensible configuration. Just make it deliberately.

⚠ **The volume does NOT follow the manager actor.** The manager is a logical singleton with no root component, so its position is ignored - moving it does not move the navigation volume. Build your playable space around the world origin, or switch to an infinite world. The manager's Details panel reports the resolved bounds under **What You Get** so you can see them directly.

23.3.2 Chunk Size: default 8000 cm (*infinite only*)

Edge length of one streaming chunk.

Range: 1000-800000. Recommended UI range (slider): 3000-8000. The clamp carries the same caveat as World Extent above - it is a sanity ceiling, not the real budget, which depends on Min Voxel Size and is reported live in Configuration Health. **Here the slider range is the number that matters:** it is the measured-reliable band, and going outside it degrades long-range routing regardless of the budget.

⚠ **Do not treat this as a free dial.** It affects memory, generation cost, *and* whether long routes can be found at all. **8000 is the tested default.** If you change it, re-test long-range routing specifically: a value that looks fine for short hops can break distant paths.

⚠ **Chunk Size changes navigation *precision*, and not in the direction you expect**

This is the single most surprising thing about the setting, so it is worth stating plainly.

Subdivision stops when a child's half-extent falls below Min Voxel Size, starting from Chunk Size / 2. The finest leaf edge you actually get is therefore $\text{Chunk Size} / 2^k$ - it lands somewhere between **2× and 4× Min Voxel Size**, and it does **not** improve as Chunk Size shrinks. Measured on a real level at Min Voxel Size = 100:

Chunk Size	finest leaf edge	Chunk Size	finest leaf edge
3000	375 cm (3.75×)	7000	219 cm (2.19×)
5000	312 cm (3.12×)	8000	250 cm (2.50×)
6000	234 cm	9000	281 cm

So a smaller Chunk Size can give you a **coarser** octree. At Chunk Size 3000 you get 375 cm leaves from a 100 cm setting - and **an opening that stays navigable at 250 cm can voxelize shut at 375 cm**. That is felt as *"the agent refuses to path"*, which reads like a bug rather than a setting.

What to do: leave it at 8000 unless you have a measured reason. If you must change it, use the **Auto-Tuner** (Editor Tools) - it *measures* the leaf edge from the built octree for each candidate rather than deriving it, and gates on whether a route exists at all before comparing quality or cost.

23.3.3 Min Voxel Size: default 100 cm

The finest resolution the octree will subdivide to. **The dominant quality/cost control.**

Range: 10-2000. Recommended UI range: 25-500.

- Smaller → finds tighter gaps; more memory; longer builds.
- Larger → cheaper; narrow openings disappear.

Rule of thumb: about half the narrowest gap agents must pass through.

It's a floor, not an exact size. The octree halves down from the world/chunk size, so effective resolution snaps to powers of two. 100 → 90 may change nothing; 100 → 60 may double memory. Tune by testing.

23.3.4 World Scale: default Standard

Which set of centimetre constants the manager uses. *Standard* is the shipped tuning for metre-scale worlds at a 100 cm leaf and is byte-identical to earlier releases. *Small* derives those constants from the finest leaf this manager produces - Clearance Padding becomes half the leaf (written into the property, which turns read-only), and the start-rescue probe distances and the large-node cost threshold scale with it - so a centimetre-scale world such as a 120 × 70 × 40 cm aquarium at 1 cm voxels gets defaults that mean the same thing. *Custom* keeps whatever you set. Change *Min Voxel Size*, *World Extent* or *Chunk Size* under *Small* and the padding is re-derived; it is re-derived again at *BeginPlay* so config-driven setups match the editor.

Min Voxel Size now accepts down to **0.5 cm** and *World Extent* down to **50 cm**. The finest leaf is 2-4× *Min Voxel Size*, so set half the leaf you want and make the world edge a power-of-two multiple of it. Small worlds are small in *leaves* too: below ~32 leaves across, node count grows nearly with volume as the leaf shrinks (the panel says so). See *Small Worlds*.

A few centimetre constants follow the leaf on every profile, because they live in the generator and the solver rather than on the manager: the neighbour-probe offset, the voxel line-of-sight sampling step, the waypoint-merge tolerance and the route executor's sampling floors are bounded by the finest leaf of the navigation data itself. They keep their previous values at production leaf sizes; there is nothing to set.

23.3.5 Padding Mode: default Centimetres

How *Clearance Padding* is expressed. *Centimetres* keeps the value you enter. *Leaves* derives it as *Clearance Padding Leaves* × the finest leaf whenever the leaf changes (and again at *BeginPlay*), so the blocked band around every surface stays the same number of leaves thick at every *Min Voxel Size*. That is the condition under which halving the leaf grows the octree by the surface law (about 4× per halving) instead of by a band that thickens, in leaves, as the leaf shrinks, which keeps the growth per halving under about 4.5×. *World Scale* = *Small* already derives half a leaf and takes precedence.

23.3.6 Clearance Padding Leaves: default 0.5

The multiple used under *Padding Mode* = *Leaves*. 0.5 inflates every surface by half a leaf, which is what *Small* derives. *Range: 0-4*.

23.3.7 Clearance Padding: default 100 cm

Inflates obstacles during voxelization so agents keep clear of surfaces.

Range: 0-1000. Recommended UI range: 0-300.

⚠ **Keep this well below Min Voxel Size.** Padding that approaches voxel size fills in doorways and corridors completely, and the pathfinder reports no route through openings you can clearly see. This is the single most common cause of "it says there's no path but I can see the gap".

23.3.8 Coarse Cell Size: default 2500 cm

Cell size of the coarse long-range routing layer. Larger = cheaper long-range planning, less precise. The default suits most levels.

Range: 100-20000.

23.4 3. Collision

23.4.1 Obstacle Query Channels

Which collision channels count as obstacles. Anything on these channels is voxelized as solid. Defaults to WorldStatic + WorldDynamic.

Empty means *nothing*, not *everything*. Unlike Allowed Obstacle Classes below, clearing this list does not widen the query - it voxelizes an empty world, and you get navigation data that says every point is open air. That passes every metric and shows up only as agents flying through walls. **Bake to Asset** refuses to run on an empty list for exactly this reason.

If a wall isn't blocking agents, check that its collision channel is in this list **and** that the mesh actually has collision enabled.

23.4.2 Allowed Obstacle Classes

Optional filter. Leave **empty** to consider all actors (normal). Populate it to restrict voxelization to specific actor classes (useful when you want to ignore scenery entirely).

23.5 4. Performance

23.5.1 Voxelization Method: default Auto

Where the octree is built. Auto builds it on the GPU, which is substantially faster, and falls back to the CPU automatically if no suitable GPU is present. GPU asks for the GPU explicitly, and CPU always builds on the CPU. Choose CPU only to force deterministic, reproducible builds.

Levels saved with the older Use GPU Voxelization checkbox are converted to this setting when they load.

23.5.2 Max Generations Per Frame: default 1

Chunk builds allowed per frame. Higher fills the world faster but can cause hitches. *Range: 1+, UI up to 8.*

23.5.3 Max Path Requests Per Frame: default 64

Path queries dequeued per frame. Raise for many agents replanning at once; lower to protect frame time. *Range: 1-256.*

23.5.4 Max Requests Per Frame: default 50

General request throttle. *Range: 1-200.*

23.5.5 Max Flow Solves Per Frame: default 4

Flow-field solves per frame. *Range: 0-16.*

23.5.6 Finite Obstacle Update Interval: default 0.1 s (*finite only*)

How often dynamic obstacle changes are folded into the octree. Lower = more reactive, more CPU. *Range: 0-2 s.* See Tutorial 05.

23.5.7 Chunk Quantized Macro Cache: default off

Caches long-range routes quantized to chunk boundaries. Helps when many agents share similar long-range routes (large crowds spawning together).

23.5.8 Pawn Agents (BT & StateTree) subcategory

Field	Default	Meaning
Use Flow Fields (pawn agents only)	off	Opt-in: pawn agents heading to a shared goal use one steering field instead of individual paths
Flow Field Threshold	16	Agent count at which a shared field is built (<i>range 2-512</i>)
Flow Field Cluster Cell Size	1000 cm	Grouping granularity for shared goals (<i>range 100-10000</i>)

Flow fields are for **swarms heading to the same place**. For a handful of agents with individual goals they are pure overhead, so leave it off.

23.6 5. Coarse Skeleton (optional)

Two independent, composable ways to keep **long-range** macro routing complete in a streamed world. **Both are opt-in and default off.** They are not part of the base configuration - turn one on only when an agent must route to somewhere it cannot currently see. Agents operating inside their streaming bubble need neither.

23.6.1 Persistent Skeleton (runtime, no bake): default off (*infinite only*)

Keeps a lightweight long-range connectivity graph in memory so distant routes can be planned before the detailed chunks exist. No bake required; works on any world, including procedural ones.

Turn it on when an agent **returns across ground it has already covered** and must not lose routability over it. The cost is memory that grows with how much ground has been covered - one compact node per visited-then-evicted chunk, bounded by `r.LcmNav.MaxSkeletonNodes`, not by the size of the resident set.

23.6.2 Use Baked Skeleton (frame-0 completeness): default off (*infinite only*)

Loads a pre-baked skeleton so long-range routing is complete from the very first frame, including into **never-visited** space.

Tick this on first. The **Bake Skeleton** button only appears in the banner once it is on - nothing else reads a bake, so the button stays hidden until you have asked for one. Then click it **Rebake** after changing the level or the voxel settings - a stale bake is refused at load, and the **Baked Skeleton (auto)** line under **7. Runtime State** will say so.

Baked Skeleton (auto) under **7. Runtime State** always reports which of the two, if either, is actually active.

23.7 6. Debug

23.7.1 Draw Debug Volumes: default off

Draws the octree. **The most useful diagnostic in the plugin:** if a gap shows no fine voxels, the pathfinder can't route through it either. Expensive; turn it off when you're done.

Also available from Blueprint via `SetDrawDebugVolumes` / `ToggleDrawDebugVolumes`.

23.7.2 Draw Pathfinding Debug: default off

Draws computed paths.

23.7.3 Draw Infinite World Debug: default off (*infinite only*)

Draws chunk boundaries and streaming state.

23.7.4 Debug Redraw Coalesce Seconds: default 0.25 s

How long debug redraws are batched. Raise it if debug drawing itself is hurting frame rate.

23.7.5 Cost Areas subcategory

Field	Default	Meaning
Cost View	<i>Off</i>	Draws the cost areas while playing. <i>Regions</i> : each volume and what it costs. <i>Heat</i> : the octree tinted by price. The console variable <code>r.LcmNav.CostView</code> 0/1/2 overrides it; -1 hands control back
Cost View Filter	<i>none</i>	Whose eyes the view looks through: a stock Navigation Query Filter class. Empty = each area's own defaults

See Tutorial 10 for cost areas themselves.

Force Redraw SVO was removed. It was a checkbox that instantly unchecked itself, and it was destructive: on a level with no baked data it *deleted* the editor preview it was meant to refresh. Reloading a bake into the preview now happens automatically when you bake, and the manual action is the **Refresh Preview** button, which appears on the manager only when the level actually has baked navigation to reload.

23.8 7. Runtime State

Read-only diagnostics showing live counts of resident chunks, skeleton state and staleness. Useful when tuning an infinite world; nothing to set.

23.9 Build & Tools (Blueprint-callable)

Function	Purpose
<code>BuildSVO</code>	Build the navigation volume now
<code>BuildSVOInEditor</code>	Build in-editor into memory , so EQS previews work without playing. Not saved

Function	Purpose
BakeFiniteSV0ToAsset	<i>(finite)</i> Save the SVO to an asset + streaming proxy, so the level stops voxelizing at launch. Turns Auto Build On Play off. Returns false and does nothing if a single-volume Bake Volume already owns this level's one finite volume
BakeSkeletonInEditor	<i>(infinite)</i> Pre-bake the long-range skeleton for Use Baked Skeleton
LoadBakedSkeleton	Load a previously baked skeleton
FindPath	Synchronous path query: start, end, options, out array of points
FindCover	Tactical cover query
UpdateDynamicObstacle	Manually stamp an obstacle by bounds
SetMacroTopologyCell	Manually mark a coarse cell blocked/free
ForceRedrawSV0, SetDrawDebug Volumes, ToggleDrawDebugVolumes, DebugDrawFlowField	Debug

23.10 Per-agent settings

These live on the **Fly To** BT task / **Fly To StateTree** task, not the manager. Both tasks expose the identical surface, in six numbered sections:

1. Goal

Field	Default	Meaning
Acceptance Radius	100 cm	Distance at which the goal counts as reached
Target Location	-	<i>(StateTree only)</i> world-space goal
Target Actor	<i>none</i>	<i>(StateTree only)</i> moving goal; overrides Target Location when chase is on
Timeout Seconds	0 (no limit)	<i>(advanced)</i> Report Failed after this long in flight. For a goal that keeps moving away, or one reached only by a very long route

2. Movement

Field	Default	Shown when	Meaning
Agent Scale	0 (= World Scale)	always	Multiplies every centimetre and speed value on the task (acceptance radius, speed, acceleration, deceleration distance, wall check, separation, drift replan, Minimum Agent Size and the follower's internal floors). 0 follows the manager's World Scale : 1 under <i>Standard</i> , finest leaf / 100 cm under <i>Small</i> (0.01 in a 1 cm world). A driving flight component adopts the same scale unless its own Agent Scale is set
Use Flight Movement Component	off	always	Delegate the body to a <code>ULcmFlightMovement</code> Component for 6-DOF drone / bird / fish dynamics
Flight Speed	600 cm/s	not delegating	Cruise speed

Field	Default	Shown when	Meaning
Max Acceleration	2000 cm/s ²	not delegating	Lower = heavier feel; also sets how hard the agent can brake into its arrival
Flight Style	<i>Physics (Drones/Ships)</i>	not delegating	vs <i>Linear (Biological/Magic)</i>
Deceleration Distance	800 cm	not delegating and Physics-weighted	Slow-down lead-in. LinearDirect has no inertia to bleed off, and the flight component brakes on $v = \sqrt{2 \cdot a \cdot d}$ against the real goal distance instead
Banking Amount	4.0	not delegating	Roll into turns (cosmetic)
Max Banking Angle	45°	not delegating	Roll limit (cosmetic)

Whoever owns the body owns its caps. Tick Use Flight Movement Component and this section collapses to that one checkbox: speed, acceleration, flight style and banking are set on the component's **Flight Config** instead, and the task reads them from there for its guidance, braking and chase-lead maths. Untick it and the task's own values apply. Exactly one place is ever live.

3. Pathfinding

Field	Default	Meaning
Query Options	-	Solver, heuristic, smoothing, agent size, wall-distance preference
Query Options > Filter Class	<i>none</i>	The stock Navigation Query Filter this agent sees cost areas through. Empty = the AI controller's Default Navigation Filter Class
Query Options > Require Complete Path	off	<i>(advanced)</i> Fail a request whose route stops short of the goal, instead of returning the part that can be planned now

4. Avoidance *(the reactive knobs hide when Avoidance Style is CBS)*

Field	Default	Meaning
Avoidance Style	<i>Reactive (Boids/Push)</i>	None / Reactive / Predictive (ORCA) / Context-Based
Avoidance Time Horizon	2.0 s	ORCA look-ahead (Predictive only)
Collision Margin	1.2	Safety multiplier on the avoidance radius
Wall Check Distance	200 cm	Wall probe range
Wall Push Force	4000	Wall push strength - keep it strong
Path Centering Force	600	0 = free drift, high = train-on-tracks
Separation Radius	150 cm	Neighbour spacing
Separation Weight	2.0	Spacing strength

5. Moving Goal (Chase)

Field	Default	Meaning
Enable Goal Prediction	off	Intercept a moving target
Force Enable Chase	off	Activate chase without also setting <code>r.LcmNav.Chase.Enable</code>
Lead Seconds Cap	2.0 s	Prediction limit
Drift Replan Cm	400 cm	Target movement before replanning

6. Streaming (infinite worlds only)

Field	Default	Meaning
Frontier Stall Recovery	<i>Inherit</i>	Coast / climb out / arrive un-quantized when stalled at a streaming frontier. <i>Inherit</i> defers to the manager's Streaming Profile

Frontier Stall Recovery is the **locomotion** half only. Pinning chunks *ahead* of the agent is a separate world-level setting - **Agent Corridor Prefetch** on the LCM Nav3D Manager. They used to be one checkbox named for the half it mostly wasn't.

Solver and smoothing choices are described in Tutorial 02 §6.

StateTree tasks that finish at once. On UE 5.2 a task that completes, completes its whole state, so a FlyTo beside it stops. Find Cover, Chunk Prewarm, Streaming Aware, Find Actor By Tag and Pick Random Goal have **Stay Running After Resolve**: tick it when the task shares a state with a FlyTo, and the state runs until the FlyTo completes.

23.11 Console variables new in 3.4

Variable	Default	Meaning
<code>r.LcmNav.CostView</code>	-1	Cost view override: 0 off, 1 regions, 2 heat. -1 = the manager's Cost View
<code>r.LcmNav.Bridge.AutoRegister</code>	1	Register the navigation proxy so stock Move To and EQS plan through LCM Nav3D
<code>r.LcmNav.Bridge.ClaimDefault Agent</code>	1	With no Recast navmesh in the world, the proxy serves the default agent. See Tutorial 14
<code>r.LcmNav.MacroPath.Known Fallback</code>	1	When the coarse search reaches its limit, route through the streamed chunks instead of failing
<code>r.LcmNav.MacroPath.CapRetry Sec</code>	5	After that happens, how long a route searches the streamed chunks only
<code>r.LcmNav.MacroPath.CapRetry MaxSec</code>	60	The wait doubles each time the full search reaches its limit again, up to this
<code>r.LcmNav.Mass.SerialPhases</code>	1	Sets <code>mass.FullyParallel</code> 0 at startup (see Troubleshooting). 0 leaves it alone
<code>r.LcmNav.Mass.FlowAsync Bootstrap</code>	1	A crowd's first flow field is built off the game thread. 0 = on the game thread, as before 3.4

23.12 Recommended starting points

Scenario	Infinite	Extent / Chunk	Min Voxel Size	Clearance Padding
Arena or single building	off	10000	100	100
Tight interiors, small agents	off	10000	50	25
Large outdoor level	off	20000	200	100
Open / streaming world	on	8000	100	100

Start here, then confirm with `Draw Debug Volumes` that the gaps you care about actually have voxels in them.

CHAPTER 24

Troubleshooting

Problems in the order people actually hit them. **Start with the Quick Triage table:** it resolves most issues in under a minute.

24.1 Quick triage

Symptom	Most likely cause	Jump to
Agent doesn't move at all	No navigation manager in the level	\$1
"No path" through a gap you can see	Clearance Padding sealed it	\$3
Agent flies through walls	Geometry not voxelized	\$4
Moving obstacles ignored	Missing Dynamic Obstacle component	\$5
Can't find the demo maps	<i>Show Plugin Content</i> is off	\$7
Frame rate drops	Voxel resolution too fine	\$6
Long paths fail, short ones work	Infinite-world streaming	\$3

24.2 The two diagnostics that answer almost everything

1. Turn on **Draw Debug Volumes** (navigation manager → 6. *Debug*).

This draws the octree. It tells you what the pathfinder actually sees, which is frequently not what you assume:

- **No small voxels around a doorway** → the pathfinder can't route through it either.
- **No voxels at all in a region** → it's outside `World Extent`, or the chunk hasn't generated.
- **Solid voxels where there's visibly open air** → `Clearance Padding` is too large.

2. Check the **Output Log** (Window → Output Log) and filter for `Lcm`. The plugin logs the specific reason a query failed.

24.3 1. Nothing moves

Work in this order.

24.3.1 Is there a navigation manager?

Output Log shows `LCMBTTask_FlyTo: No SVO Manager found in the level!` → drag an `LcmNavigationManagerSVO` into the level. One per level, required.

24.3.2 Did the AI possess the pawn?

- Pawn → Class Defaults → **Auto Possess AI = Placed in World or Spawned**
- Pawn → Class Defaults → **AI Controller Class** is set
- The controller actually runs the tree (Run `Behavior Tree`, Or a `StateTreeAIComponent` with a `State Tree` assigned)

24.3.3 Can the pawn move at all?

It needs a movement component that supports flight. **Floating Pawn Movement** is the simple choice. Check its **Max Speed** isn't 0.

24.3.4 Is the goal set?

Print the blackboard key (or StateTree parameter). A goal of $(0,0,0)$ means it was never set, or was set after the tree started.

24.4 2. Agent moves, but badly

Symptom	Cause	Fix
Jitters, vibrates in place	Physics fighting navigation	Set mesh collision to <i>No Collision</i> to confirm, then re-add carefully
Stops short of the goal	Acceptance Radius too large	Lower it, but not below one voxel
Overshoots and circles	Speed too high for turn rate	Lower Flight Speed or raise Deceleration Distance
Robotic square turns	Grid-locked solver, no smoothing	Use <i>Lazy Theta*</i> and <i>Curved Spline</i> smoothing
Sinks into the floor	Insufficient clearance below	Raise Clearance Padding slightly, or Min Voxel Size finer
Agents pile up on each other	No separation	Raise Separation Radius / Separation Weight , or use <i>Predictive (ORCA)</i> avoidance

24.5 3. "No path found"

24.5.1 First: is the goal reachable at all?

- **Inside the navigation volume?** Outside **World Extent** there is no data. Enlarge it, or move the goal in.
- **Inside geometry?** A goal (or start) buried in a wall has no free voxel. Move it into open air.
- **Genuinely enclosed?** If the destination is sealed off, failure is correct.

24.5.2 The classic: padding sealed the opening

This is the most common cause of a mystifying "no path".

If **Clearance Padding** is close to **Min Voxel Size**, obstacle inflation fills narrow openings completely. A 2 m doorway with 100 cm voxels and 100 cm padding can vanish entirely.

Fix: lower **Clearance Padding**, or lower **Min Voxel Size** so the opening survives the inflation. Confirm with **Draw Debug Volumes**: you should see free voxels *through* the opening.

24.5.3 Short paths work, long ones fail

Almost always an **infinite world** issue: the route crosses chunks that haven't generated yet.

- Turn on **Persistent Skeleton** under **5. Coarse Skeleton (optional)**. It is **off by default**
- it is what allows long-range planning before the detail exists, and it is the first thing to try when an agent loses routability over ground it has already crossed.
- For routing to be complete on the very first frame, including into space the agent has never visited, tick **Use Baked Skeleton** (also under **5. Coarse Skeleton (optional)**) and then click the **Bake Skeleton** button, which appears in the banner once that option is on.
- Tick **Assist Infinite Streaming** on the **Fly To** task for agents that must traverse the frontier.
- Leave **Chunk Size** at **8000** unless you have a specific reason and will re-test long routes.

24.5.4 Everything fails immediately

The volume never built. Check **Auto Build On Play** is on, or that you call `BuildSV0`. With `Draw Debug Volumes` on, you should see the octree at `BeginPlay`.

24.5.5 It fails only for some agents: cost areas

Call `Get Last Path Cost Failure` on the manager.

- **Goal In Excluded Area:** the goal is inside a volume this agent's filter excludes (or a No-Fly / Null area). Move the goal, or give the agent a filter that allows it.
- **No Route Under Filter:** every way to the goal crosses an excluded volume. Open a gap, or change the filter.

Set the manager's **Cost View** to *Regions* and its **Cost View Filter** to that agent's filter to see the world as it does. See Tutorial 10.

24.5.6 The route stops short of the goal

In a streamed world, part of the way may not be generated yet. `Find Path` returns that partial route on purpose, and the agent follows it while the rest streams in. **Find Path Detailed** tells you whether a route is Complete or Partial; **Require Complete Path** in the query options makes a partial route fail instead.

24.6 4. Agents ignore geometry

If agents fly through a wall, the wall isn't in the octree.

1. **Does the mesh have collision?** Voxelization uses collision, not visual geometry. *No Collision* = invisible to navigation.
2. **Is its channel in `Obstacle Query Channels`?** (3. *Collision*)
3. **Is `Allowed Obstacle Classes` filtering it out?** Empty = allow everything, which is what you normally want.
4. **Was it added after the build?** Static geometry is baked at build time. Something spawned later needs either a rebuild or a Dynamic Obstacle component. In an infinite world, a level that streams in after its area was built is picked up automatically: the chunks it overlaps are rebuilt (see Infinite World).
5. **Is it too thin?** Geometry thinner than a voxel can be missed. Lower `Min Voxel Size`, or thicken the collision.

24.7 5. Dynamic obstacles do nothing

1. **Does the actor have an LCM Nav3D Dynamic Obstacle component?** Moving an actor without one changes nothing as far as navigation is concerned, and produces no error.
2. **Does its mesh have collision enabled?**
3. **Is `Update Distance Threshold` too high?** It won't restamp until the actor moves that far (default 50 cm).
4. **Reaction too slow?** Lower `Finite Obstacle Update Interval` (default 0.1 s).
5. **Using `Tracked Components`?** If the list is non-empty, only those components are tracked, so make sure the blocking one is in it.

See Tutorial 05.

24.8 6. Performance

24.8.1 Build is slow / hitches when streaming

- Raise `Min Voxel Size` (the biggest lever by far).
- Keep `Voxelization Method` on Auto (GPU).
- Lower `Max Generations Per Frame` to spread work (world fills in more slowly, but smoothly).

24.8.2 Frame rate drops with many agents

- Lower **Max Path Requests Per Frame** to cap per-frame planning.
- Use cheaper avoidance: *Reactive* rather than *Predictive*.
- For agents sharing a goal, enable **Use Flow Fields**.
- **Past a few hundred agents, move to Mass Entity.** Actor ticks, not pathfinding, are the ceiling.

24.8.3 Memory too high

- Raise **Min Voxel Size** (memory scales steeply with resolution).
- Shrink **World Extent** to just your play space.
- In infinite worlds, keep **Chunk Size** at the default.

24.8.4 Debug drawing is itself slow

Expected: it's a debug tool. Raise **Debug Redraw Coalesce Seconds**, or turn it off.

24.9 7. Editor and install

Problem	Fix
Can't find demo maps	Content Browser → Settings → Show Plugin Content
"Modules are missing or built with a different engine version"	Rebuild; if it fails, your project is Blueprint-only. See Tutorial 01
Plugin absent from the Plugins window	Wrong folder layout: must be YourProject/Plugins/LCMNav3D/LCMNav3D.uplugin
Fails to load, <code>GetLastError 126</code>	A Mass plugin dependency got disabled; re-enable <code>MassEntity</code> , <code>MassGameplay</code> , <code>MassAI</code>
Compile errors about <code>StateTree</code> or <code>Mass</code> types	Same cause: an engine plugin dependency is off
EQS preview shows nothing in-editor	Call Build SVO In Editor on the manager; EQS previews need navigation data without playing
Stock Move To ignores LCM Nav3D	The log says NotClaimed : a Recast navmesh owns the default agent. Add a Supported Agent for LCM Nav3D (Tutorial 14)
Log line "mass.FullyParallel set to 0"	Expected. The plugin runs Mass phases serially to avoid a UE 5.2 phase-order ensure that froze the editor for seconds, and it was also faster at 10,000 agents. It never overrides a value you set. To opt out: <code>r.LcmNav.Mass.SerialPhases 0</code>

24.10 8. Determinism and multiplayer

If you need identical results across runs (replays, lockstep, automated tests):

- Set **Voxelization Method** to CPU.

Path solving already runs on the CPU and is deterministic by design. Voxelization is the part that can vary: GPU execution order isn't guaranteed run-to-run, so force the CPU voxelizer when you need a bit-identical rebuild.

24.11 Still stuck?

Gather these before asking for help, since they identify most problems immediately:

1. **Draw Debug Volumes screenshot** of the area that misbehaves.
2. **Output Log** filtered to `Lcm`.
3. **Manager settings**: Infinite World, World Extent/Chunk Size, Min Voxel Size, Clearance Padding.

4. **Which path** you're using: Behavior Tree, StateTree, or Mass.
5. **Does the equivalent demo map work?** If yes it's a setup issue in your level; if no it's an install issue.

Then bring them to the community Discord: <https://discord.gg/nXkpD6uaBG>

Open a **Technical Support** ticket in #get-help and you get a private channel with the author. First reply within 24 hours on weekdays. #support-faq there carries this same triage list, and #known-issues tracks confirmed defects with their workarounds - worth checking before you write.

LCM Nav3D - Realistic Flight/Swim Locomotion Reference

Ultra-realistic 6-DOF locomotion for flying/swimming pawns (drones, birds, fish), layered on top of LCMNav3D navigation. The nav system provides **guidance** (where to go); this layer provides physically-plausible **dynamics** (how the body moves).

25.1 Concept - Guidance → Control → Dynamics

- **Nav (unchanged):** SVO path / flow field → pure-pursuit + CBS/ORCA avoidance → a *desired velocity*.
- **ULcmFlightMovementComponent (new):** consumes that desired velocity and runs a 6-DOF rigid-body flight/hydro model (thrust, lift/drag, gravity/buoyancy, attitude PID) per archetype, producing the pawn transform and an animation-drive struct.

25.2 Quick start

1. Add a **Lcm Flight Movement Component** to a pawn whose root is a collision primitive.
2. Set: `bEnabled = true`, `LocomotionModel = FlightDynamics`, `Archetype` (Multirotor / Bird / Fish / Generic6DOF), and tune `FlightConfig`.
3. Drive it one of two ways:
 - **Nav-driven (recommended):** on the pawn's **BT/StateTree FlyTo** task, tick **Use Flight Movement Component**. The task feeds guidance automatically.
 - **Manual:** call `RequestVelocity(WorldDesiredVelocity)` each tick from your own code/Blueprint.

Off by default: with `bEnabled=false` (or the task flag off) nothing changes - motion is the legacy kinematic `FlyTo`.

Already have a movement component on the pawn? Leave it there - nothing to remove. While the flight component is enabled it takes **sole movement authority**: any other movement component driving the same root (`FloatingPawnMovement`, `CharacterMovement`, `ULcmNavMovementComponent`) is automatically paused (velocity zeroed, deactivated, tick off) and restored exactly as found the moment flight stops driving (`bEnabled=false`, the component deactivated, or the pawn torn down). Without this, UE runs both integrators against one body and the last ticker overwrites the pawn's velocity readout - and a `CharacterMovement` keeps applying gravity against the flight dynamics. Opt out per-instance with **Pause Conflicting Movement Components**, or globally with `r.LcmNav.Flight.MovementAuthority 0`. Two notes: stock `MoveTo` path-following still binds the pawn's original movement component, so drive an enabled flight component with **FlyTo** (or `RequestVelocity`) - and the pawn's root primitive must not have **Simulate Physics** enabled (the component warns in the log if it does).

25.2.1 Stopping at the goal (Hold Station When Idle)

A movement component is told *what velocity to have*, so "stop" has always meant "target zero velocity" - and a body with momentum answers that by coasting until drag absorbs it, with nothing to bring it back. Measured settling error after arriving inside a 150 cm sphere and being told to stop: multirotor 267 cm past the goal, jet 936 cm, fish 2369 cm, and the bird 14,790 cm away and still descending.

Hold Station When Idle (on by default, `r.LcmNav.Flight.HoldStation`) makes an unguided component steer back to the point guidance handed it over at, on a gentle brake-shaped approach with a deadband

so a parked body is genuinely at rest. Any `RequestVelocity` call takes over immediately, so it only ever governs an otherwise-idle body. With it on, all four archetypes now settle within ~2 m of the goal and stay there.

25.3 Debugging the flight path - planned vs predicted vs flown

Turn on the navigation manager's `bDrawPathfindingDebug` (the master switch for all path debug) and a flight-component pawn draws three distinct lines:

line	what it shows	switch
Orange polyline + red spheres	the PLANNED path the planner delivered	manager <code>bDrawPathfindingDebug</code>
Magenta arc ahead of the pawn	the PREDICTED dynamic flight path - the component's real dynamics rolled forward along the remaining route (look-ahead, banking, braking included)	component <code>bDrawDebugPrediction</code> (default on) / <code>r.LcmNav.Flight.DebugPredict</code> , horizon <code>r.LcmNav.Flight.DebugPredictSec</code>
Green→red fading trail behind	the FLOWN trajectory (speed-coloured), plus yellow = velocity and cyan = guidance arrows	component <code>bDrawDebugTrail</code> (default off) / <code>r.LcmNav.Flight.DebugTrail</code> , lifetime <code>r.LcmNav.Flight.DebugTrailSec</code>

How to read them: the gap between orange and magenta is the body's *physics* (corner-cutting, brake distance - expected and correct); the gap between magenta and the trail is *live avoidance* (neighbours, walls - the prediction deliberately excludes what depends on other agents' futures). A lone agent whose trail leaves the magenta line is a tracking defect worth reporting. All three draw nothing in Shipping builds.

25.4 Archetypes (key `FlightConfig` params)

Archetype	Model	Notable params
Jet / Fixed-Wing (Generic6DOF)	Body-forward thrust, airspeed ² lift carrying the weight, coordinated bank-to-turn, thrust-vectored VTOL blend below <code>WingBorneSpeed</code>	<code>MaxThrust</code> , <code>ThrottleResponse</code> , <code>WingBorneSpeed</code> , <code>LiftCoeff</code> , <code>MaxLiftFactor</code> , <code>BankTurnGain</code> , <code>MaxBankAngleDeg</code> , <code>VelocityP</code> , <code>AttitudeP/D</code>
Multirotor	Tilt-to-translate, altitude hold, spool lag, wind/gust, hover jitter	<code>RotorSpinUpRate</code> , <code>WindVelocity</code> , <code>GustStrength/Frequency</code> , <code>HoverJitterStrength</code> , <code>bYawTracksVelocity</code> , <code>RotorMaxRPM</code>
Bird	Effort-throttled pulsed wingbeat, airspeed ² lift, flap↔glide, banked turns, flare	<code>FlapFrequency</code> , <code>FlapThrust</code> , <code>LiftCoeff</code> , <code>MaxLiftFactor</code> , <code>BankTurnGain</code> , <code>MaxBankAngleDeg</code> , <code>FlareSpeed</code> , <code>FlareDragScale</code>
Fish	Neutral buoyancy, tail-beat undulatory thrust + sway, anisotropic hydro drag, current	<code>bNeutralBuoyancy</code> , <code>TailBeatFrequency</code> , <code>TailBeatFreqGain</code> , <code>TailWiggleStrength</code> , <code>LateralDragMult</code> , <code>WaterCurrent</code>

Shared: `MaxSpeed`, `MaxAcceleration`, `MaxTiltAngleDeg` (Multirotor), `MaxTurnRateDeg`, `SubSteps` (integration sub-steps; higher = stiffer + smoother), `BankingAmount/MaxBankingAngle` (kinematic-mode roll).

25.4.1 Winged bodies (Jet, Bird) - how they fly, and how to tune them

Both share one wing model, and it behaves like an aircraft rather than a hovering body:

- **Turns are made by banking.** The heading error commands a bank; the banked lift supplies the centripetal force; the nose then follows the *velocity*. So a winged body has a genuine minimum turn radius, $v^2 / (g \cdot \tan(\text{MaxBankAngleDeg}))$ - 212 cm at 600 cm/s and 60°. Raise `MaxBankAngleDeg` (and `MaxLiftFactor` with it) for tighter turns; lower it for a heavy bomber.
- **Lift carries the weight, thrust only fights drag.** At cruise the throttle sits near `drag/MaxThrust` (~0.12 at defaults). `LiftCoeff` sets how much airspeed the wing needs: weight is balanced when $\text{LiftCoeff} \cdot v^2 = \text{Mass} \cdot 980$.
- **They cannot pinpoint-stop on aerodynamics alone.** Commanded to a hard stop, a wing has nothing to push against. The **Jet** answers this with a thrust-vectored blend below `wingBorneSpeed`; the **Bird** flares to shed speed and hover-flaps to hold. Both rely on the nav layer's arrival ramp - with a constant full-speed command any fast forward-flyer orbits its goal, which is expected.
- **Arriving is not the same as staying.** When a `FlyTo` task arrives it stops feeding guidance, and an unguided flight component **holds the position it was left at** rather than coasting to wherever momentum runs out (see *Hold Station When Idle* below). Without that a winged body sails past its goal and never comes back.
- **The Jet is Generic6DOF remodelled** (the enum name is unchanged so existing content keeps working). `r.LcmNav.Flight.JetGeneric 0` restores the previous hover-thruster behaviour exactly.

Notes:

- A fast forward-flyer (bird/fish) can't pinpoint-stop; the nav layer's **arrival slowdown** makes it flare to a perch. Without arrival-aware guidance a fast body orbits its goal (expected).
- Realistic dynamics *lag* guidance by design; raise `MaxAcceleration` for snappier, lower for heavier/driftier.
- **The two control loops are a cascade - tune them as a pair, never one alone.** `VelocityP` is the outer loop (time constant $1/\text{VelocityP}$); it commands a *tilt* that the inner `AttitudeP/AttitudeD` loop must then achieve. Keep the inner loop ~4× **faster** than the outer one, or the body wanders side to side and hunts back and forth around its goal:

	formula	shipped default
inner natural frequency	$\text{wn} = \text{sqrt}(\text{AttitudeP})$	7.75 rad/s
inner damping ratio	$z = (\text{AttitudeD} + \text{AngularDrag}) / (2 \cdot \text{wn})$	0.84
inner time constant	$1/(z \cdot \text{wn})$	0.15 s
outer time constant	$1/\text{VelocityP}$	0.67 s

△ `AngularDrag` adds to `AttitudeD` - the integrator applies $-\text{AttitudeD} \cdot w$ and then $-\text{AngularDrag} \cdot w$, so the loop's real damping is the **sum**. Raising `AttitudeP` for a snappier body without raising `AttitudeD` with it drops z below ~0.7 and the body visibly rings after every heading change. Damping must scale as $2 \cdot z \cdot \text{sqrt}(\text{AttitudeP})$.

25.4.2 Altitude while manoeuvring - `MaxTiltAngleDeg` and the thrust budget

A tilt-to-translate body (`Generic6DOF` / `Multirotor`) keeps only $\cos(\text{tilt})$ of its thrust opposing gravity, so holding height while tilted needs a thrust-to-weight ratio of $1/\cos(\text{tilt})$:

	tilt	lateral accel available	thrust-to-weight needed just to hold height
	30°	566 cm/s ²	1.15×
	45°	980 cm/s ²	1.41×
	60° (default)	1697 cm/s ²	2.0×
	75°	3657 cm/s ²	3.9×

MaxTiltAngleDeg caps this so the body cannot ask for a manoeuvre that costs it altitude, and when the thrust budget saturates the **vertical axis is served first** and the lateral axis takes the remainder. Δ Raising the tilt limit without raising MaxThrust to match makes the body descend under hard lateral commands - the harder you ask it to move, the faster it sinks. Keep $\text{MaxThrust} \geq \text{Mass} * 980 / \cos(\text{MaxTiltAngleDeg})$; at the shipped 1 kg / 3000 / 60° there is 2× margin.

25.4.3 Reaching the commanded speed - drag is fed forward

The velocity loop cancels the drag it is about to apply, exactly as it already cancels gravity. That matters more than it sounds: drag is a *constant disturbance* to a proportional-only loop, and a P-only loop answers one with a **permanent offset** of $|\text{drag}| / (\text{Mass} * \text{VelocityP})$. Left uncanceled at the shipped defaults that is 336 cm/s against a 600 cm/s command - the body would cruise at 44 % of what it was told, and no gain fixes it (raising VelocityP shrinks the error but costs the visible lean). Kill switch `r.LcmNav.Flight.DragFeedForward 0`.

25.4.4 Making a body *lean* (tilt-to-translate look)

A thrust-vector body leans because thrust must point along $\text{Mass} * \text{accel} + \text{weight}$. There is no cosmetic lean parameter in FlightDynamics mode - the attitude is physical, so you get lean by changing what the body has to *do*. Three separate mechanisms, commonly confused:

Want	Knob	Why
Forward lean while cruising	LinearDragCoeff.X/ Y $\uparrow$	At steady speed $\text{accel} \approx 0$, so lean is exactly $\text{atan}(\text{drag}/\text{weight})$. No control gain can change this. 0.5→1.4 takes it from 10°→28° at 600 cm/s.
Lean while accelerating/braking	VelocityP $\downarrow$	Peak lean is $\text{atan}(\text{VelocityP} * \text{MaxSpeed}/g)$ but it decays over $1/\text{VelocityP}$, and the area is fixed. High gain = a spike too brief to see; low gain = a shallower lean actually held.
Bank into turns	YawP $\downarrow$ (keep $\square$ AttitudeP)	The body banks only while its velocity error is <i>lateral</i> . Fast yaw whips the nose onto the new heading first, making the error forward instead - pivot-then-go, no bank.

Δ In Kinematic mode none of the above applies: attitude there is cosmetic, driven by BankingAmount and MaxBankingAngle.

25.4.5 BankingAmount is now a multiple of the physically correct bank

BankingAmount (Kinematic mode, and the matching field on BT/StateTree FlyTo) is a **multiplier on the coordinated-turn angle** $\tan(\text{bank}) = v * \text{yawRate} / g$. 1 is the true bank a real body holds in that turn; above 1 exaggerates it for readability; 0 flies wings-level.

Δ Its default changed from 4 to 1 and the old number does not carry over. The previous law dotted a *normalised* per-frame velocity delta with the right vector - normalising throws away how hard the turn actually is, so the roll command sat near ± 1 on ordinary velocity noise and the body flicked between $\pm 4^\circ$ every frame in straight flight. Multiplying the correct law by 4 instead saturates MaxBankingAngle in any ordinary turn. If you had authored a value here, re-tune it around 1. Legacy law: `r.LcmNav.Flight.CoordinatedBank 0`.

25.4.6 Flying straight up or down

Travel-referenced attitude is built by **parallel transport** of the previous frame, not from a world-up reference. This is not a refinement - the two obvious UE builders are *discontinuous* at vertical travel, and both were in use:

- `FVector::Rotation` yaws by $\text{atan2}(Y, X)$, which at $X = Y = 0$ is $\text{atan2}(0,0) = 0$: a body climbing straight up snaps its heading to world +X, then swings with any lateral noise.

- `FRotationMatrix::MakeFromXZ` **substitutes** $(1,0,0)$ for its up reference once $|Fwd.Z|$ passes 1 - `KINDA_SMALL_NUMBER` - the frame teleports rather than degrading.

Either one makes a climbing or diving agent roll hard for reasons invisible in its flight path. World-up levelling is still applied, weighted by how horizontal the travel is, so it fades out exactly where the reference stops being meaningful and the body simply holds its roll through vertical. Applies to Kinematic, Bird, Fish and both FlyTo tasks. Kill switch `r.LcmNav.Flight.ContinuousFrame 0`.

25.4.7 The vertical axis has its own gain - `VelocityPZ`

`velocityP` is held low on purpose: horizontally it commands a **tilt** the attitude loop then has to achieve, so it must stay $\sim 4\times$ slower than that loop. The vertical axis has no such constraint - thrust acts on it **directly**, with no attitude change in between - so making it share the slow gain costs bandwidth for nothing. Every production multirotor controller splits these (PX4 `MPC_Z_VEL_P` vs `MPC_XY_VEL_P`; ArduPilot `PSC_VELZ_P` vs `PSC_VELXY_P`).

Measured on a closed-loop rig flying a **dead-level** path, so any altitude motion is controller-generated:

<code>VelocityPZ</code>	settled altitude wander	settled vertical speed
legacy (shares <code>VelocityP</code> = 1.5)	40.3 cm	33.4 cm/s
2	26.1	30.3
4 (default)	6.0 cm	12.1 cm/s
8	5.4	9.9

Cross-track error and arrival time are unchanged ($173.8 \rightarrow 173.6$ cm, $10.85 \rightarrow 10.83$ s), so it costs nothing horizontally. **4 is derived, not chosen**: position-loop damping $\zeta = 0.5 \cdot \sqrt{(K_z \cdot L/v)}$ reaches 1.0 at $K_z=4$ when the look-ahead equals the speed, and the velocity loop's damping against the rotor spool, $\zeta = 1/(2\sqrt{(K_z \cdot \tau_{spool})})$, is 0.707 at exactly $K_z=4$ with `RotorSpinUpRate` 8. Raise `RotorSpinUpRate` too if you push it much past 4.

25.4.8 A path that ends short of the goal no longer orbits

If the delivered path's last waypoint is further than $2 \times \text{Acceptance Radius}$ from the goal, the task used to pin the index back to that waypoint *every tick* - and since the agent is already inside the acceptance radius of it, the next tick retired it and pinned again. Measured: **1750 pin events in 40 s**, with the task neither succeeding nor failing. It now pins once, holds position, replans twice, and then reports **Failed** with a log line naming the likely cause. Common causes of a short path: the goal quantises to a leaf centre, or `r.LcmNav.SeamSafe.TruncateUnprovable` (default ON) truncated the route at its first unprovable leg.

25.5 Troubleshooting - symptom to cause

Every row below is a *structural* cause with its own kill switch, so each can be toggled on its own. Work down the list; do not start by re-tuning gains, because none of these is a gain problem.

Symptom	Likely cause	Switch to toggle
Altitude swings up and down continuously, even in level flight	The vertical axis shared the (deliberately slow) horizontal <code>velocityP</code> , leaving the pure-pursuit position loop at $\zeta \approx 0.61$ - underdamped, with nothing to damp it	<code>velocityPZ</code> (default 4; set 0 for the legacy shared gain)
Never reaches a waypoint, circles it forever, task never Succeeds or Fails	End-of-path fallback re-pinned the index every tick when the path ended $> 2 \times \text{AcceptanceRadius}$ short of the goal	fixed unconditionally; look for the <code>[FlyTo]</code> giving up: warning

Symptom	Likely cause	Switch to toggle
Altitude rings up then sags whenever it starts a horizontal move	Thrust magnitude applied along the body's <i>current</i> axis instead of projected onto it	<code>r.LcmNav.Flight.ThrustProjection</code>
Sinks during hard turns or fast starts	Tilt unbounded / thrust saturation cutting the vertical component	<code>r.LcmNav.Flight.ThrustPriority</code> , then raise <code>MaxThrust</code> or lower <code>MaxTiltAngleDeg</code>
Never reaches its commanded speed; feels unresponsive	Drag not fed forward → permanent P-loop offset	<code>r.LcmNav.Flight.DragFeedForward</code>
Rolls violently when climbing or diving steeply	Discontinuous world-up attitude reference	<code>r.LcmNav.Flight.ContinuousFrame</code>
Twitches side to side in straight, level flight	Roll commanded from a <i>normalised</i> velocity delta	<code>r.LcmNav.Flight.CoordinatedBank</code>
Lurches when it passes a waypoint or an avoidance term flips	Discontinuous guidance command stepping the cascade	<code>r.LcmNav.Flight.GuidanceSlew</code>
Judders / grinds along a surface it is touching	Velocity not corrected after a blocking sweep	<code>r.LcmNav.Flight.SlideVelocity</code>
Weaves near geometry it is not even approaching	Wall whiskers cast off the <i>nose</i> (which lags travel), binary, no forward or vertical coverage	<code>r.LcmNav.WallAvoidV2</code> 1 (⚠ default 0 - see below)
Bird's wingbeat stutters on and off at cruise	Flap/glide threshold had no hysteresis	fixed unconditionally (Schmitt trigger on <code>GaitEffort</code>)
Fish rolls in time with its own tail beat	Wiggle force fed back into the attitude error	fixed unconditionally
Passes waypoints / overshoots the goal	Guidance solving stopping distance against <code>MaxAcceleration</code> , a ceiling the loop rarely reaches	<code>r.LcmNav.FlightGuidanceAccelFromVelocityP</code> (already default 1)
Weaves along a dense path	Pure-pursuit look-ahead shorter than the body's response distance	<code>r.LcmNav.PursuitLagFloorK</code> (already default 1.5)

⚠ `r.LcmNav.WallAvoidV2` defaults to 0 on purpose. Reducing this term's authority has been *measured harmful* once already - a bounded version cost a single agent its arrival (33 loop events, orbiting), because a planned route sometimes points into a wall and only a full-authority push clears it. V2 keeps full authority at contact and only ramps the far field, but that measurement has not been retaken against it. Turn it on if flyers judder near geometry, and re-check arrival rates.

25.6 Agent scale - flying a body in a centimetre world

`ULcmFlightMovementComponent::AgentScale` (default 0) makes a body authored for a metre-scale world fly in a centimetre-scale one without retuning anything. The integrator runs in **agent units**: every `FlightConfig` value and every internal constant (gravity, the airspeed thresholds, the station-hold deadband) keeps its production meaning, and positions and velocities are converted at the component's boundary - world = agent x scale. 0 adopts the scale of the `FlyTo` task that drives the body (its own `AgentScale`, or the navigation manager's **World Scale**); a positive value is explicit. At 0.01 a 1 m drone tuned for 600 cm/s flies as a 1 cm one at 6 cm/s, through the same path shape, in the same time - a similarity transform, not a retune. Two things follow from that: `FLcmFlightState` is reported in agent units (`Speed01` is unchanged, `Airspeed` and `VerticalSpeed` read 600 at full speed whatever the scale), and the pawn's own collision and mesh must be scaled by the user - the plugin scales motion, not geometry. In a 120 x 70 x 40 cm aquarium at a 1 cm leaf, a drone at actor scale 0.01 flies the cave route it flies at 100:1.

25.7 Animation contract - FLcmFlightState

Every tick the component produces FLcmFlightState (archetype-agnostic where possible):

Field	Meaning
Speed01	Normalized airspeed ($ velocity / MaxSpeed$) - blendspace axis
VerticalSpeed	cm/s, climb + / dive - - climb/dive pose blend
BankAngleDeg / PitchDeg	Body roll / pitch - additive lean/pitch
YawRateDeg	Turn rate - lean/tilt into turns
GaitPhase	0.1 cyclic driver: rotor spin / wingbeat / tail-beat
GaitAmplitude01	Throttle (drone) / flap-glide blend (bird) / undulation vigour (fish)
RotorRPM	Multirotor rotor RPM
Throttle01 / Airspeed	Commanded effort / raw speed

Consume it two ways:

- **Pull:** GetFlightState (BlueprintPure) on the component, e.g. from an AnimBP's Update.
- **Push:** implement `ILcmFlightAnimInterface::ReceiveFlightState` on your AnimBP (or pawn). The component calls it every tick (bBroadcastAnimState, default on). Cache the struct and drive:
 - GaitPhase → rotor/wing/tail cyclic pose (Sine/curve on the phase)
 - Speed01 + VerticalSpeed → locomotion blendspace
 - BankAngleDeg / PitchDeg → additive lean/pitch
 - GaitAmplitude01 → flap-vs-glide (bird) or effort (fish/drone) blend weight
 - RotorRPM → prop-spin material/WPO param (Multirotor)

25.8 Mass crowds & scale (P7-P8)

The LCM Nav3D Agent trait (NavMode = PathFollowing) carries the same flight options (LocomotionModel, FlightArchetype, FlightConfig) - set them and a whole crowd flies with the identical 6-DOF physics as a pawn, still pathing + avoiding + arriving. Default Kinematic → byte-identical to before.

Dynamics LOD (the scale knob). Full 6-DOF per agent is expensive; set FlightDynamicsLODDistance (cm) so only agents within that distance of the viewer run full dynamics - everyone else uses the cheap kinematic integrator (face-velocity, no roll). So a 10K crowd keeps only the on-screen agents on the expensive path. 0 = always full. Global kill switch: `r.LcmNav.Mass.FlightLOD` (1 default / 0 = full everywhere). LOD switching is seamless - a far agent's integrator state re-syncs from its live transform when it returns to range.

Deferred (documented) scale extensions:

- **GPU 6-DOF compute** for very large crowds is a designed extension, intentionally not shipped here: CPU dynamics-LOD already bounds cost to the visible set, which is the cheaper win. Revisit with engine uplift.
- **Shared flight config** - FlightConfig is currently per-entity; moving it to a const-shared fragment per archetype would cut crowd memory. Deferred (shared-fragment archetype plumbing).

25.9 Determinism

The dynamics are deterministic: fixed integration sub-steps, no RNG or wall-clock; gust/jitter are seeded per-agent sinusoidal noise. Same input sequence → identical trajectory.

25.10 Bird and Fish do not share the drone control cascade

⚠ **StepBird and StepFish are separate integrators - they never call RunStep6DOF.** That is deliberate (a bird does not translate sideways by tilting a rotor), but it meant they also never had its velocity-tracking loop: thrust acts along body-forward only, and until 2026-08-06 **nothing corrected a velocity error perpendicular to the nose**. They pointed at the guidance direction and waited for drag, so cross-track and altitude error could only decay passively - never be driven to zero. That is what "drifts away from the path points" was.

They now generate the perpendicular force a banked wing or a flexing body actually produces, bounded by what the body can physically supply - **lift** for the bird, **tail/fin thrust scaled by swim effort** for the fish. BankTurnGain is the rate (1/s) that converts a cm/s error into that demand, which is what its own description always claimed. Measured on one corner path, same guidance:

archetype	cross-track before → after	altitude error before → after
Generic6DOF (reference)	174.3 → 174.3	9.4 → 9.4
Multirotor (reference)	173.6 → 173.6	22.8 → 22.8
Bird	234.4 → 183.9	69.7 → 61.5
Fish	203.7 → 198.4	64.0 → 17.4

Both drone archetypes are byte-identical (their gate checksums are unchanged), because the change is scoped to the two integrators that lacked the loop. Kill switch `r.LcmNav.Flight.BodyManeuver 0`.

⚠ **Do not "simplify" the bird by removing its LiftControl1 trim as a duplicate vertical controller.** Measured and reverted: it is the bird's only means of balancing lift against gravity - lift is $\text{LiftCoeff} \times \text{airspeed}^2$, which at cruise exceeds body weight, so without the trim the bird climbs away (altitude error 61.5 → 186.4 cm). The trim sets lift *magnitude* for equilibrium; the manoeuvre force supplies *perpendicular authority*. They are not redundant.

25.11 Fish schooling on pawns

The Flight Movement component has a **Schooling** group (LCM Flight|Schooling). Turn on **Enabled** and, when a FlyTo task drives the body, it steers as a school member instead of by the task's avoidance (reactive push, wall whiskers, flock separation): personal space, collision anticipation, heading matching and closing up with other agents (pawns and a Mass school alike), and turning away from rocks and glass before reaching them. The task's route still says where to go, and a pawn with its own route always counts as informed.

The settings are the same as on a Mass crowd; the table and the leadership, predator and burst-and-coast options are in the Mass Traits Reference. Two differ for pawns:

- **Contact Push** (default 0.5) pushes apart overlapping bodies, scaled by overlap depth.
- A predator is any body half again the fish's size or more that is not schooling with it, so a larger pawn on its own flight model (the Fish Tank's predator) sets off the escape.

Console multipliers for tuning without editing assets: `r.LcmNav.FishSchool.ScaleRepulsion`, `ScaleAnticipation`, `ScaleAlign`, `ScaleAttract`, `ScaleWall`, `ScaleEscape`, `ScaleContactPush`, `ScaleBrake` (all 1 = as authored), `r.LcmNav.FishSchool.MaxCommandDeg` (-1 = the config's) and `r.LcmNav.FishSchool.Leadership` (0 = every fish steers for the goal).

25.12 Hard turns: progress does not depend on hitting a sphere

⚠ **If a tighter Acceptance Radius makes your agents stick, this is why.** A waypoint used to be retired only by `HasReachedOrPassedWaypoint`: *within Acceptance Radius, or past the plane through it normal to the incoming leg*. At a corner sharper than the body's minimum turn radius - $v / \text{VelocityP}$, which is **400 cm** at the shipped defaults - the agent cannot pass within a 100 cm radius, and the plane has rotated with the path so it is not past that either. **The gate becomes unsatisfiable**, the index stalls on a point now *behind*

the agent, and because pure pursuit measures its look-ahead *from the current index*, the aim point sits behind too and drags the agent backwards.

Measured on 375 cm waypoint spacing (what ChunkSize 3000 produces), 135° corner:

acceptance radius	gate only	+ projection advance
50 cm	stuck - 39.9 s stalled, 1214 backtrack ticks	arrives, 0 backtrack
100 cm	stuck - 40.0 s stalled, 1214 backtrack ticks	arrives, 0 backtrack
200 cm	arrives	identical
400 cm	arrives	identical

The index now *also* advances by **projecting the agent onto its path**, so progress is monotone and independent of the acceptance gate. It is **inert** wherever the gate already works (45° and 90° corners are byte-identical; 200 cm and 400 cm radii are byte-identical). On the demo map at ChunkSize 3000 with 7 agents it took arrivals from 1/7 to 3-4/7 and mean cross-track from 392 cm to ~225 cm.

△ **r.LcmNav.PathProjectWindow (default 3) is load-bearing, not an optimisation.** On a hairpin the return leg runs spatially alongside the outbound one, so an unbounded nearest-segment search would snap the agent onto the return leg before it had rounded the corner and cut the whole turn. Kill switch for the whole behaviour: `r.LcmNav.PathProjectAdvance 0`.

△ Acceptance Radius still does not control path adherence - it never did. It decides when a point is *retired*, not how closely the agent flies. Corner-cutting is bounded by physics: minimum turn radius is v^2/a_{eff} , so waypoints spaced closer than that will be passed wide however you tune this.

25.12.1 Scope: every follower rework in this section is 6-DOF ONLY

● The behaviours documented below - progress-based stall detection, the visibility-clamped look-ahead, the backward speed profile, and the projection index advance - apply **only** to a pawn driven by an enabled `ULcmFlightMovementComponent` in **FlightDynamics (6-DOF)** mode.

The **Kinematic (legacy point-mass)** model and pawns with no component at all (which the task integrates itself) keep their original behaviour exactly: the acceptance-gate-only waypoint advance, the unclamped look-ahead target, the one-corner speed cap, and the displacement-based stall timer. Those are separate shipped locomotion models whose feel users have already built content against.

△ **Both defaults point away from 6-DOF** (`LocomotionModel = Kinematic`, `bUseFlightMovementComponent = false`), so a **default-configured pawn gets none of this**. It is opt-in, by design. `LcmFlightLocomotion::IsSixDoFBody` is the single predicate; do not re-derive it.

The same rule covers the attitude rework (`r.LcmNav.Flight.LegacyAttitudeOutside6DOF`, default 1) and the component's guidance-reference limiter and slide-velocity correction, which are likewise 6-DOF only. The limiter in particular **must not** reach Kinematic: `ELcmFlightStyle::LinearDirect` exists to answer a command directly and `PhysicsWeighted` to answer it with inertia, and handing `LinearDirect` a reference already ramped at `MaxAcceleration` makes the two indistinguishable.

25.12.2 Stall recovery measures PROGRESS, not motion

An agent that is flying in a circle is, to every speed-based health check, perfectly healthy. That is not a hypothetical: on `Vertical_Skyscraper` at ChunkSize 3000 the flyer went **828 m to gain 35 m** (tortuosity 23.8, 53 loop events) and every recovery counter read zero - `stallFrac` 0.00, `escapeTicks` 0, `replans` 0, `paths` 1. It orbited the first storey until the run timed out.

The cause was one line. The stall timer reset whenever the pawn had moved more than 50 cm from `StallEscapeAnchor` - and then moved the anchor to the pawn:

```
if (FVector::DistSquared(NowPos, StallEscapeAnchor) > 2500.0f) // ">50 cm = progress"
{ StallEscapeAnchor = NowPos; StallEscapeSeconds = 0.0f; }
```

An orbiting agent clears 50 cm every few frames, so the timer could never accumulate. **The one recovery path that could have broken the loop was held open by the loop itself.**

Progress is now **remaining arc length along the path** (`ULcmSteeringMath::RemainingPathLengthCm`), which falls monotonically along any correct traversal and does not fall at all while orbiting.

☹ **Straight-line distance to the goal is the wrong scalar and must not be substituted.** A correct route routinely moves *away* from the goal - the horizontal leg of a vertical switchback, or any detour round an obstacle - so a goal-distance watchdog fires on healthy navigation.

⚠ `StallBestRemainCm` **must** be reset to `TNumericLimits<float>::Max` every time `CurrentPath` is replaced. A fresh path legitimately has a larger remaining length; without the reset it reads as permanent no-progress and arms recovery forever.

Vertical_Skyscraper, CS 3000		
	legacy	progress-based
outcome	stuck at Z=-6999 (first storey)	climbs the tower
closest approach	15855 cm	600 cm (143 cm with a shorter look-ahead)
tortuosity	23.8-31.6	5.6-6.8
loop events	53-68	0-13
replans	1	19-39

The legacy failure is bit-reproducible (15855 cm / 31.56 / 68 loops / 1 path on every run), which makes this map a far better regression instrument than the 7-agent demo scenario.

Kill switch: `r.LcmNav.ProgressStallDetect 0`. Threshold: `r.LcmNav.ProgressEpsilonCm` (default 50).

25.12.3 The look-ahead target must be REACHABLE, not merely on the path

Pure pursuit measures its look-ahead **along the path**, then flies at the result in a **straight line**. Those are the same thing only where the path is locally straight. In a vertical shaft with offset floor openings, a target 400-1000 cm along the path sits on the far side of a **slab**, and the straight line to it goes through solid. The agent charges a point it cannot reach, is blocked, retries, and orbits - the user-visible *"it reaches for a further point before finishing the nearest one, then gets stuck"*.

`ULcmSteeringMath::GetPurePursuitTargetVisible` walks the look-ahead back until the agent→target segment is clear in the SVO, and falls back to `Path[CurrentIndex]` - the nearest unconsumed waypoint, which is always a safe aim because the planner already proved the path traversable. At most 4 line-of-sight tests per tick.

⚠ **It fails OPEN.** With no chunk data - unstreamed, or the segment leaves the single chunk this test can see - it returns the unmodified target, i.e. exactly legacy behaviour. Never infer a blockage from missing data; that would brake the follower at every chunk boundary.

Vertical_Skyscraper, CS 3000, progress detector on		
	look-ahead unclamped	visibility-clamped
arrives	no, no	yes, yes
time to goal	never	80.12 s, 80.25 s
closest approach	6615, 6615 cm	144, 145 cm
loop events	46, 46	0, 0
replans	40, 40	15, 15

Both fixes are needed. The progress detector alone breaks the orbit-lock but still ends 66 m short; the visibility clamp alone leaves the stall detector blind to any orbit it does not prevent.

Kill switch: `r.LcmNav.PursuitVisibleTarget 0`.

Regression on Mixed_Indoor_Outdoor (same pinned single-agent instrument, 2 runs per arm): loop events fell **48/46** → **3/1** there too, with `trackErrMax` unchanged (2326/2406 → 2683/2133, overlapping). Δ Replans rose 23/10 → 76/64: that is the progress detector doing its job - the agent now *notices* it is getting nowhere instead of orbiting silently - but it is real planner load. Raise `r.LcmNav.ProgressEpsilonCm` if that rate is too high for your budget.

Δ **chunkSize 8000 cannot navigate this map at all, for an unrelated reason.** It yields leaves up to 1000 cm, too coarse to resolve the floor openings, so the slabs seal the shaft into disconnected components and no path exists to plan (`portalsInAgentComp=0`, `open set exhausted`). The agent never moves because there is nothing to follow - do not read that as a follower defect.

25.12.4 Speed planning: the agent slows down *for corners it can already see*

Fixing the index advance stops the agent sticking, but it does not stop it going into a corner **too fast**. Those are separate defects with the same symptom, and the second one is speed-dependent - which is exactly the "fine when it's slow, sticks when I speed it up" report.

The speed governor's corner cap used to look **exactly one waypoint ahead**: it capped speed at $\sqrt{v_{\text{corner}}^2 + 2 \cdot a \cdot d}$ where d is the distance to the *single* next waypoint. That is the right equation solved over the wrong distance. Braking distance grows as v^2 , waypoint spacing is fixed by the voxel grid, and the body's response lag is a fixed $v / \text{velocityP}$ - so past some speed the brake command is issued too late to be achievable, and the agent simply carries its speed through the corner. Measured at a 135° corner on 375 cm spacing, `MaxSpeed 1200`: the old cap held mean speed to 823 cm/s - *precisely* $\sqrt{v_{\text{corner}}^2 + 2 \cdot 900 \cdot 375}$, i.e. it did exactly what it was written to do, and that was not enough. Peak deviation from the path was still 371 cm. In a corridor, 371 cm is a wall.

The governor now builds a proper **backward velocity profile** over a window (`r.LcmNav.SpeedProfilePoints`, default 8):

1. Give every upcoming waypoint the speed *its own* curvature allows, $v_i = \sqrt{a \cdot R_i}$.
2. Sweep **backward**: $v_i = \min(v_i, \sqrt{v_{i+1}^2 + 2 \cdot a \cdot d_i})$, so each corner's limit propagates upstream as far as the braking distance actually needs.
3. Brake from the agent's position to **every** limit in the window and obey the tightest, charging the body's response distance $v / \text{velocityP}$ against each available braking distance.

Peak cross-track at a 135° corner, 375 cm spacing, by `MaxSpeed`:

MaxSpeed	ungoverned	one-corner cap (old)	backward profile	mean speed, old → new
300	94.5 cm	94.5 cm	95.0 cm	284 → 281
600	201.5 cm	184.8 cm	93.4 cm	517 → 506
900	287.2 cm	268.1 cm	89.8 cm	693 → 688
1200	397.8 cm	371.4 cm	103.5 cm	823 → 827

Tracking error is now **flat in speed** instead of growing linearly with it, and it is not bought with travel time - mean speed is unchanged (at 1200 it is marginally *higher*, because the agent stops holding a compromise speed down the straight and instead runs fast then brakes late-but-sufficiently).

⊖ `r.LcmNav.SpeedProfilePoints 1` narrows the window to a single corner but is **NOT** the legacy cap, and must not be used as a way to restore it. The old cap took the incoming leg from the agent's own position (`Path[i] - CurrentLoc`); the profile takes it from the path (`Path[i] - Path[i-1]`), because path geometry is the only definition available for the waypoints further out. The braking distances differ, and on a `LinearDirect` body that difference shows up as overshooting the waypoint. Non-6-DOF bodies therefore run the original block verbatim rather than a 1-wide profile.

On the demo map (`Mixed_Indoor_Outdoor`, `BT_FlightBehavior_Loop`, single agent, pinned start/goal, 3 runs per arm) exactly one follower metric moves, and it is the one that matters:

metric	one-corner cap	backward profile	
worst-case cross-track	3570 / 3244 / 3282 cm	2536 / 2560 / 2544 cm	–24%, no overlap
mean cross-track	1226 / 1197 / 1320	839 / 1232 / 1047	overlaps - no effect
commanded deflection	13.7 / 7.7 / 2.8°	1.8 / 5.0 / 16.1°	overlaps - no effect
avoidance-dominated ticks	10 / 7 / 4 %	2 / 4 / 10 %	overlaps - no effect

The worst-case figure is not merely better, it is **tight**: ± 12 cm across runs whose replan counts ranged 16-47, against ± 170 cm for the one-corner cap. That is the signature of a bound that now holds structurally rather than by luck. *Mean* cross-track not moving is consistent with the mechanism - the profile acts at corners, and the mean is dominated by the straights, where both arms are identical.

⚠ **Do not measure this with the 7-agent configuration.** ORCA/flocking interference, a looping BT that disperses agents differently each run, and an unpinned start together produce a **57× spread within a single arm** (wall-avoidance term 0.08 to 4.60), which is far larger than the effect. Arrival count alone came back 0, 1 and 3 of 7 across runs of the *same* build. Pin the start and goal and use one agent. Note also that *reach=* is meaningless under a looping BT - it cycles targets by design.

⊖ **Do not “improve” step 3 by walking the response distance forward along the path and evaluating the profile at that single lagged point.** It reads as the more principled model of a lagging body, and it is worse at every speed (166 / 238 / 288 cm against 95 / 163 / 247): once the lagged point passes the corner, the limit there is the *post*-corner straight-leg limit, so the governor releases the brake while the body is still entering the turn. The lag credit is sound for accelerating and unsound for braking - only the pessimistic direction may use it.

CHAPTER 26

LCM Nav3D - Mass Entity Trait Reference

How to build a Mass crowd/agent with LCM Nav3D. Add these traits to a **Mass Entity Config** asset (Create → Miscellaneous → Mass Entity Config) and use it on an AMassSpawner.

One rule: every agent needs **exactly one movement trait**. Everything else is an orthogonal add-on. The editor validator (below) enforces this.

26.1 The anatomy of a config

A Mass Entity Config is a list of traits. A working Nav3D agent needs one from each of three categories, and no more than one from the movement category.

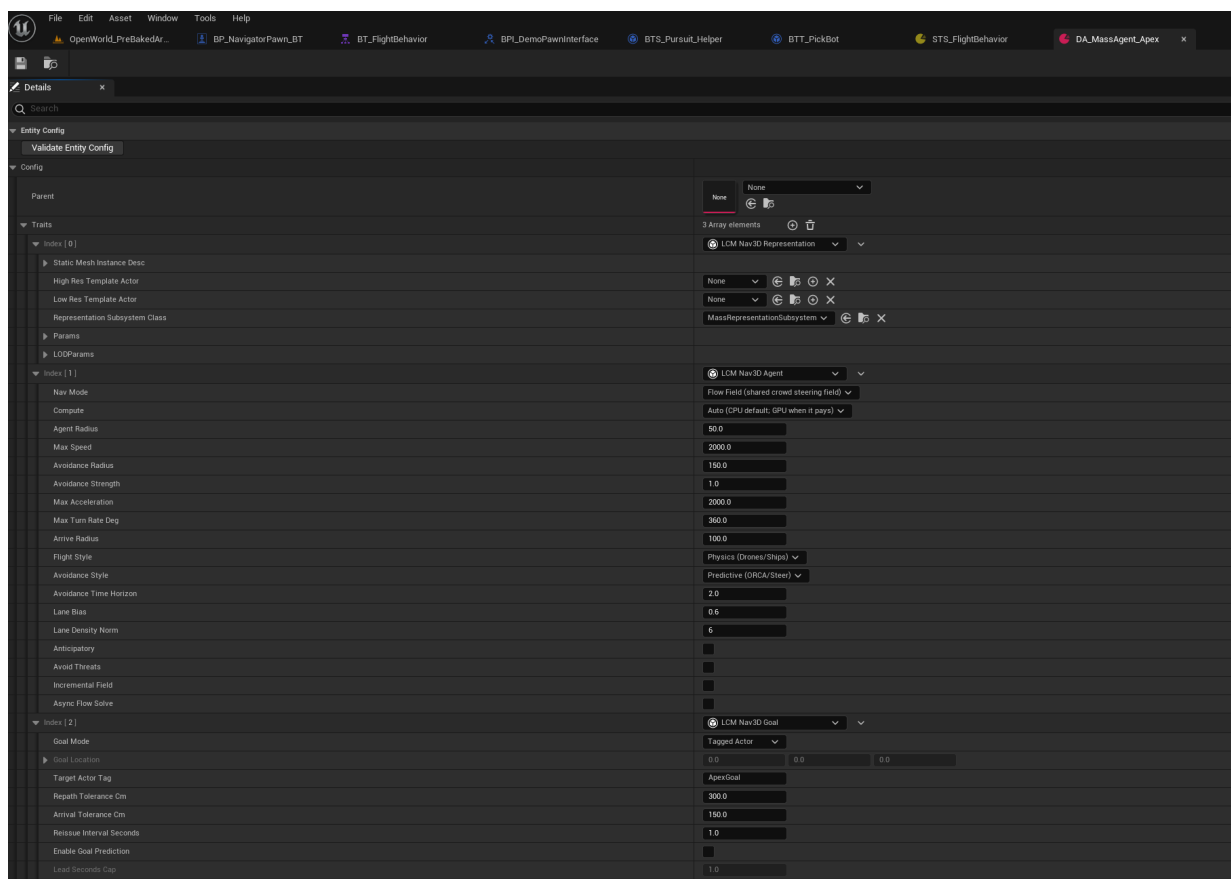


Figure 1 - A three-trait config, one per category.

Index	Trait	Category	Why it is here
[0]	LCM Nav3D Representation	Rendering - <i>pick one</i>	Without a rendering trait your agents move correctly and are invisible

Index	Trait	Category	Why it is here
[1]	LCM Nav3D Agent	Movement - exactly one	Its Nav Mode picks Path Following or Flow Field
[2]	LCM Nav3D Goal	Goal	Here Goal Mode = Tagged Actor, targeting tag ApexGoal

Things the screenshot shows that are easy to miss:

- **Validate Entity Config** at the top runs the trait-combination check on demand. It also runs on save and on cook.
- **Compute** - leave it on Auto. This release solves on the CPU, so Auto and CPU behave identically; the selector is kept for forward compatibility.
- **Greyed rows are not disabled features.** Goal Location is greyed because Goal Mode is Tagged Actor, and Lead Seconds Cap because Enable Goal Prediction is off. The panel hides what the current configuration will ignore.

26.1.1 This config is also the worked example of the one rule

Look at [1] and [2] together. The Agent's Nav Mode is Flow Field, but LCM Nav3D Goal is a Path Following add-on - it drives the *path-request* fragment, and a Flow Field agent never issues one. So this combination **fails validation**, with:

LCM Nav3D Goal only works with a Path Following agent (it drives the path-request fragment). For a Flow Field agent, set the goal on the 'LCM Nav3D Agent' trait's Goal / Goal Mode properties instead.

The fix is one of two things, and which one depends on what you want:

You want	Change
A 10K-scale crowd sharing a goal	Remove the Goal trait. Set the goal on the Agent trait's own Goal Mode properties.
Per-agent precise routes	Set Nav Mode to Path Following. The Goal trait is then correct as it stands.

Press **Validate Entity Config** on any config you build. It catches this, the duplicate-movement-trait case, and the mixed Path+Flow case - all three of which are silent at runtime and present as "my agents do not move".

26.2 The traits

26.2.1 LCM Nav3D Agent - the one movement trait (start here)

The single authoring entry point. Pick a **Navigation Mode**:

Navigation Mode	What it does	Fragments composed
Path Following	Per-agent A* through the SVO (HPA* across chunks). Best for hero/mid-count agents, precise routes, infinite worlds.	NavAgent, PathRequest, PathResult

Navigation Mode	What it does	Fragments composed
Flow Field	One shared Dijkstra flow field per goal, $O(1)$ /agent sampling. Best for 10K-scale crowds sharing goals (finite worlds).	FlowField (state) + FlowConfig Shared (const shared config)

Mode-specific properties hide automatically via `EditCondition`. Common movement knobs (radius, speed, accel, turn-rate, avoidance style, flight style, lane bias) apply to both.

Dense crowds and fish schools (Flow Field)

Three settings on the Agent trait for crowds whose bodies must not overlap:

- **Hold Avoidance Between Scans** (default off). The neighbour scan runs every other frame (`r.LcmNav.Mass.FlowAvoidStride`). Off, the frames in between steer with no avoidance, so agents feel half their separation. On, those frames reuse the last result. Crowds tuned with it off get about twice the avoidance they were tuned for, so re-check their arrivals.
- **Contact Push** (default 0, Reactive style). Pushes apart two agents whose bodies overlap: a velocity of (overlap depth / radii sum) x Max Speed x this. Arrived agents that overlap keep separating, so a crowd no longer piles onto its goal. 0.5 is a firm, smooth push.
- **Schooling**. Steer as a fish school instead of by the Avoidance Style: keep personal space, anticipate collisions, match neighbours' heading and close up to them, and turn away from rocks and glass before reaching them. The flow field still says where to go. All distances are in body lengths (BL), so one config works at any scale. The same settings sit on a pawn's Flight Movement component (see the Flight Locomotion Reference).

Setting	Default	What it does
Enabled	off	Use the school model instead of the generic avoidance.
Body Length Cm	0 (= 4 x collision radius)	The unit every other distance is measured in.
Body Width BL	0.25	Body width as a fraction of length. Contact and personal space are measured between fish-shaped bodies.
Perception BL / Max Neighbours	4 / 7	How far a fish notices others, and how many of the nearest it responds to.
Repulsion BL / Align BL	1.0 / 2.5	Personal space (inside it the fish only moves away), and the range where it matches heading; beyond it, it closes up.
Blind Angle Deg	60	Rear cone where neighbours are not seen for alignment and attraction.
Goal / Repulsion / Align / Attract Weight	1.0 / 2.0 / 0.6 / 0.3	How much each term steers.
Anticipation Weight / Seconds	1.5 / 1.5 s	Steer off a collision course before it happens.
Wall Weight / Wall Range BL	3.0 / 2.0	Turn away from rocks and glass before reaching them.
Steer Smoothing Seconds	0.2 s	How quickly the heading follows the terms.
Max Command Angle Deg	60	The command stays this close to the nose, so the body turns instead of skidding.
Min Speed Fraction	0.35	Slowest a schooling fish swims (fish do not hover).

Setting	Default	What it does
Leadership: Informed Fraction	1.0	Fraction of the school that steers for the goal; the rest follow their neighbours (Couzin et al. 2005).
Leadership: Uninformed Goal Scale / Goal Standoff BL / Goal Contention BL	0.1 / 0 / 2.0	How much followers still steer for the goal; where the goal's pull fades; when a goal another fish holds counts as reached.
Predator: Threat Range BL / Escape Weight	5 / 4	A body half again this fish's size or more that is not schooling with it is fled from, by the fountain manoeuvre.
Predator: Alarm Spread / Alarm Decay Seconds / Alarm Cohesion	0.8 / 1.5 s / 1.0	How alarm passes between schoolmates, how fast it fades, and how much an alarmed school tightens.
Swimming: Burst And Coast	off	Kick, then glide, turning mostly at the kick. Kick Period, Burst Fraction and Burst Speed Swing shape it.

The Fish Tank demo maps (Behavior Tree, StateTree, Mass) use it.

26.2.2 + LCM Nav3D Goal - auto-pathing (Path Following only)

Points a **Path Following** agent at a fixed location or a tagged actor and keeps it pathed there (re-pathing on drift, optional lead-prediction for moving targets). **Not for Flow Field** - set the goal on the Agent trait's *Goal Mode* properties instead. The validator warns if you get this wrong.

26.2.3 + LCM Nav3D Perception - SVO-LOS sight (either mode)

Makes an agent both a *senser* and *perceivable* (faction-based, true-3D SVO line-of-sight). Writes a per-agent "closest visible threat" surface read by EQS, StateTree, or steering. Opt-in avoidance: `r.LcmNav.Mass.PerceptionAvoid 1` makes Flow Field agents veer from their own perceived threat.

26.2.4 □ Rendering - pick ONE

Trait	Use when	LOD?	Animation
LCM Nav3D ISM Render	Turnkey, hundreds-low thousands, "just draw a mesh".	No	Yes - 2 per-instance custom-data floats: [0] speed01 (walk/run blend), [1] stable phase (de-sync). Feed a VAT/WPO material.
LCM Nav3D Representation	LOD-culled crowds; optional hero actor up close. Wraps Epic MassRepresentation with sane defaults (draws ISM at all visible LODs, culls far). Needs a viewer/player in the level for LOD distance.	Yes	Via the mesh material / actor
<i>(Niagara GPU crowd)</i>	10K+ (deferred / roadmap)	-	VAT

26.3 Combination matrix

	Agent (Path)	Agent (Flow)	Goal	Perception	ISM Render	Representation
Agent (Path)	-	X conflict	✓	✓	✓	✓
Agent (Flow)	X conflict	-	⚠ use Agent goal	✓	✓	✓
Goal	✓	⚠ redundant	-	✓	✓	✓
Perception	✓	✓	✓	-	✓	✓
ISM Render	✓	✓	✓	✓	-	X pick one
Representation	✓	✓	✓	✓	X pick one	-

- X **conflict** - two movement stacks; the editor validator errors and (at runtime) the two steerers mutually exclude → the entity would not move. Keep one.
- ⚠ - allowed but pointless; the validator explains.

26.4 The editor validator

ULcmMassConfigValidator runs on every UMassEntityConfigAsset (right-click → Validate, or on save/cook). It errors on:

1. **Two movement stacks** (Path + Flow, in any mix of unified/legacy traits).
2. **Goal without a Path Following agent.**

So a broken config can't ship. Requires the **Data Validation** plugin (bundled).

26.5 Migration from the old traits

The pre-2026-07 traits still work (byte-identical archetypes) but are marked (**Legacy**) and superseded by the unified trait:

Legacy trait	Replace with
LCM Nav3D Agent (Legacy)	LCM Nav3D Agent, NavMode = Path Following
LCM Nav3D Flow Steer (Legacy)	LCM Nav3D Agent, NavMode = Flow Field

Existing content (e.g. DA_MassAgent_Apex) keeps working - no forced migration.

Default-value deltas when migrating (the unified trait has ONE default per property; the two legacy traits differed): ArriveRadius is now **100** (legacy path 80 / legacy flow 150) and AvoidanceStyle defaults to **Reactive** (the legacy Flow Steer defaulted to Predictive/ORCA). If your config relied on a legacy default, set the value explicitly after switching traits. TargetActorTag defaults to ApexGoal1 (matches legacy).

26.6 World Partition / large-world (City Sample scale) - REQUIRED READING

26.6.1 The vanishing-crowd trap (stock engine behavior)

AMassSpawner::BeginDestroy calls DoDespawning - **the spawner destroys every entity it spawned when the spawner actor is destroyed** (engine MassSpawner.cpp). In a World Partition map a placed spawner is **spatially loaded by default**, so when the player leaves the spawner's cell, the spawner streams out and **silently deletes the entire crowd**. Symptom: agents work near the spawn area, then all disappear when the player walks away.

Fix (pick one):

1. Use **LCM Nav3D Crowd Spawner** instead of the stock Mass Spawner - identical (EQS generators etc. all inherited) but defaults to *not* spatially loaded, so its crowd's lifetime is never tied to the player's position.
2. On a stock AMassSpawner: Details → World Partition → **Is Spatially Loaded** = **false**.

This follows the AAA rule: **population lifetime must be owned by something persistent** (an always-loaded actor/subsystem), never by a streamed actor. Distance-based *despawning* should be an explicit population policy (an ambient ring, à la City Sample's crowd), not a streaming side effect.

26.6.2 City-scale checklist

- **Navigation:** a city does not fit the finite default (`WorldExtent` = 100 m). Use the manager's **Infinite World** mode (Hierarchical HPA*); the flow crowd keeps its own nav chunks resident via crowd-driven streaming (`r.LcmNav.Mass.CrowdStreaming`, default on) - agents don't lose nav when the player (the only invoker) leaves.
- **Rendering:** LCM Nav3D Representation culls agents *visually* past its LOD distances (defaults: Off at 200-250 m - deliberately conservative). For city vistas, raise `LODParams.BaseLODDistance/VisibleLODDistance` on the trait. This is visual LOD only - entities keep simulating; it is NOT the vanishing-crowd trap above.
- **Ambient population ring** (constant-density crowds around the player with bounded cost, the City Sample pattern) is a roadmap feature - until then, crowds are persistent: they exist wherever you spawned them.

26.7 Useful CVars

CVar	Default	Effect
<code>r.LcmNav.Mass.PerceptionAvoid</code>	0	1 = Flow agents veer from their perceived closest threat.
<code>r.LcmNav.Mass.PerceptionAvoidRadiusCm / ...Strength</code>	600 / 1.0	Perception-avoidance range + strength.
<code>r.LcmNav.Mass.ISMAnimRefSpeedCmS</code>	600	Speed (cm/s) mapping to ISM custom-data <code>speed01</code> = 1.
<code>r.LcmNav.Mass.Anticipatory</code>	-1 (per-trait)	Route Flow crowds around where obstacles are <i>heading</i> .
<code>r.LcmNav.Mass.FlowAvoidHold</code>	-1 (per-trait)	Force Hold Avoidance Between Scans on (1) or off (0).
<code>r.LcmNav.Mass.FlowContactPush</code>	-1 (per-trait)	Force one Contact Push gain for every Reactive-style crowd.
<code>r.LcmNav.Mass.FlowMissRecovery</code>	1	A finite-world Flow agent pushed off the field's covered space (against a rock or a wall) steers back to the nearest covered point instead of stopping. 0 = old behaviour (it holds).
<code>r.LcmNav.FishSchool.Enable</code>	1	0 = every crowd and pawn with Schooling on falls back to its generic avoidance.

Part V

Appendix

LCM Nav3D - API Reference

The classes, functions and properties you can reach from Blueprint, the Details panel, a Behaviour Tree, a StateTree or a Mass config.

Internal C++ - solver internals, replication contracts, subsystems with no Blueprint surface - is deliberately not listed. If a type is not here, you are not expected to call it.

27.1 Classes

27.1.1 LcmAIPerceptionBridge

Mirrors UAIPerceptionComponent-sensed actors into a decaying threat tracker for the tactical EQS surface.

Functions

- `HandleTargetPerceptionUpdated` - Bound to the paired UAIPerceptionComponent's `OnTargetPerceptionUpdated`.
- `GetTrackedThreats` - Current tracked-threat actors (post-decay).

Properties

- `DecayWindowSeconds` - Seconds a lost-sight threat stays queryable before it decays out.
- `PerceptionComp` - The paired perception component (found on the owner at `BeginPlay`).

27.1.2 LcmBTDecorator_ChunkLoaded

Decorator that passes only when the SVO chunk containing the queried world position is currently loaded (cheap single-endpoint hash lookup, no path probe).

Properties

- `QueryLocationKey` - Blackboard key holding the world position to probe. Accepts Vector or Object (Actor location is used in the latter case).
- `bRequireFullResolution` - When true (default), only fully-resolved chunks (promoted to full resolution) satisfy. When false, any loaded chunk does.

27.1.3 LcmBTDecorator_HasLineOfSight

Decorator that passes when the agent has volumetric line of sight to the target via the Lazy Theta* LOS walker, honouring AgentRadius through the JFA clearance gate.

Properties

- `TargetKey` - Blackboard key for the LOS target. Vector = world location, Object = AActor (its current location is used).
- `AgentRadius` - Agent collision radius for the JFA clearance gate. Higher = more conservative (rejects narrow gaps).

27.1.4 LcmBTDecorator_IsInCover

Decorator that passes when the agent is in cover relative to a threat: its leaf SweepPenalty is at or below MaxSweepPenalty AND the threat's line of sight to it is blocked.

Properties

- ThreatKey - Threat key. Vector = world location, Object = AActor.
- MaxSweepPenalty - Highest SweepPenalty (0.255) the agent's leaf may carry to count as safe. Default 32 ≈ "lightly swept, mostly safe".
- AgentRadius - Agent collision radius for the LOS walker's clearance gate.

27.1.5 LcmBTDecorator_PathLengthBelow

Decorator that passes when the true-3D LCM Nav3D path distance from the agent to the goal is below MaxLengthCm (uses the synchronous tactical-library path probe; do not run per-frame).

Properties

- GoalKey - Blackboard key for the goal endpoint. Vector = world location, Object = AActor (its current location is used).
- MaxLengthCm - Maximum allowed path length, in centimetres, for the decorator to pass.
- AgentRadius - Agent collision radius (cm) used by the path probe's clearance gate.

27.1.6 LcmBTDecorator_ThreatVisibleViaPerception

Decorator that passes when AIPerception currently perceives a hostile AND the agent has clear volumetric LOS to it (the "can-shoot-now" gate).

Properties

- ThreatKey - Optional explicit threat key. When unset (NAME_None), the decorator uses AIPerception's closest currently-perceived actor.
- AgentRadius - Agent collision radius (cm) used by the LOS walker's clearance gate.

27.1.7 LcmBTService_NavigationLoad

Service that each tick writes the navigation manager's SVO streaming stats into a blackboard key so designers can gate expensive LCM Nav3D queries on idle streaming.

Properties

- LoadedChunkCountKey - Int blackboard key receiving the count of currently loaded SVO chunks.

27.1.8 LcmBTService_PathProgressMonitor

Service that each tick writes a Euclidean approach-progress fraction (1 - dist/initial_dist) and remaining distance to the goal into blackboard keys.

Properties

- GoalKey - Blackboard key for the goal. Vector = world location, Object = AActor (its current location is used).
- ProgressKey - Float blackboard key receiving the progress fraction (0.1) toward the goal.
- RemainingCmKey - Float blackboard key receiving the remaining straight-line distance (cm) to the goal.
- GoalChangeTolerance - Distance (cm) the goal must move before the progress baseline is re-anchored.

27.1.9 LcmBTService_SquadSnapshot

Service that each tick aggregates perceived allies within SquadRadius and writes ally count, squad centroid, and cohesion radius into blackboard keys.

Properties

- AllyTag - Tag filter - only perceived actors with this tag count as ally.

- **SquadRadius** - Radius (cm) around the agent within which allies are counted.
- **bIncludeSelf** - When true, the agent itself is included in the squad aggregate.
- **NumAlliesKey** - Int blackboard key receiving the number of allies in the squad.
- **CentroidKey** - Vector blackboard key receiving the squad's centroid (average ally position).
- **CohesionRadiusKey** - Float blackboard key receiving the cohesion radius (max ally distance from the centroid, cm).

27.1.10 LcmBTService_ThreatFieldSampler

Service that each tick samples the SVO SweepPenalty risk field at the agent's location, writing the threat level and an "in threat zone" flag (level $\geq$ AlertThreshold) into blackboard keys.

Properties

- **ThreatLevelKey** - Int blackboard key receiving the current SweepPenalty (0.255).
- **InThreatZoneKey** - Bool blackboard key receiving Level $\geq$ AlertThreshold.
- **AlertThreshold** - SweepPenalty value (0.255) at or above which the agent counts as in a threat zone.

27.1.11 LcmBTTask_ChunkPrewarm

Task that proactively forces SVO chunk generation at a target location and waits (up to TimeoutSeconds) for the chunk to become ready before succeeding.

Properties

- **TargetKey** - Blackboard key for the location whose chunk to prewarm. Vector = world location, Object = AActor (its current location is used).
- **bRequireFullResolution** - When true, the task waits for the chunk to be fully resolved (post-promotion); when false, any loaded chunk satisfies.
- **TimeoutSeconds** - Maximum time (seconds) to wait for the chunk before the task fails.

27.1.12 LcmBTTask_DynamicObstaclePush

Task that steers the agent down the SweepPenalty gradient (away from rising threat) until it clears the obstacle or times out.

Properties

- **PushSpeed** - Movement speed (cm/s) while pushing away from the obstacle.
- **ProbeStep** - Step distance (cm) used to sample the SweepPenalty gradient around the agent.
- **ExitThreshold** - SweepPenalty value (0.255) at or below which the agent is considered clear and the task succeeds.
- **MaxDuration** - Maximum time (seconds) to push before the task succeeds and yields.

27.1.13 LcmBTTask_EvadeCBS

Task that retreats from the threat gradient using context-based steering (CBS) over SVO danger sampling, blending an anti-threat direction with ray-evaluated safe headings until clear or timed out.

Properties

- **EvadeSpeed** - Movement speed (cm/s) while evading.
- **ProbeStep** - Step distance (cm) used to sample the SweepPenalty gradient around the agent.
- **ExitThreshold** - SweepPenalty value (0.255) at or below which the agent is considered safe and the task succeeds.
- **MaxDuration** - Maximum time (seconds) to evade before the task succeeds and yields.
- **AgentRadius** - Agent collision radius (cm) used when evaluating SVO danger for the CBS rays.
- **LookAheadDistance** - Distance (cm) ahead along each CBS ray over which SVO danger is sampled.
- **NumCBSRays** - Number of context-based-steering rays cast around the agent to select a safe heading.

27.1.14 LcmBTTask_FindCover

Behavior Tree task that locates a navigable SVO leaf with line-of-sight blocked from the blackboard enemy actor (i.e. cover). Returns Succeeded once a valid cover location is written back to the blackboard, or Failed if no cover within SearchRadius qualifies.

Properties

- EnemyKey - The Blackboard Key for the Enemy actor (Who are we hiding from?)
- SearchRadius - Search Settings (Exposed to Behavior Tree)
- CoverHeightOffset - Vertical offset (cm) added to candidate cover positions so the pawn stands above the cover floor rather than embedded in it.
- bPrioritizeClosest - When true, the closest qualifying cover position wins. When false, the search picks the first qualifying candidate (lower latency).

27.1.15 LcmBTTask_FlyTo

Behavior Tree task that flies an AI pawn from its current position to a blackboard target via the LCMNav3D pathfinder. Owns one async path request token (re-issued on dynamic replanning), runs per-tick steering (configurable flight style + avoidance style + wall avoidance + swarm separation), and finishes the task on arrival within AcceptanceRadius.

Functions

- OnPathFound - The Callback function for Async Pathfinding

Properties

- AcceptanceRadius - Settings exposed to the Behavior Tree Node
- FlightSpeed - Cruise speed of the pawn along the path, in cm/s.
- QueryOptions - Per-request pathfinding options (solver choice, smoothing, heuristic mode, Filter Class, Require Complete Path). Forwarded as-is into FindPathAsync. An empty Filter Class uses the AI controller's Default Navigation Filter Class.
- TimeoutSeconds - Report Failed after this many seconds of flight. 0 (default) = no limit.
- SeparationRadius - How close is "too close" to another agent? (e.g. 150 units)
- SeparationWeight - How hard should we push away from neighbors? (0.0 = No push, 2.0 = Strong push)
- bUseFlightMovementComponent - Delegate locomotion to a ULcmFlightMovementComponent on the pawn (for ultra-realistic 6-DOF drone/bird/fish dynamics). When ON and the pawn has an enabled component, this task only produces the guidance velocity (pure-pursuit + avoidance + arrival slow-down) and the component owns position + orientation. OFF (default) = the built-in kinematic integrator below. The kinematic-only knobs below (accel/banking/flight style) hide when this is ON - the component owns them via its FlightConfig.
- MaxAcceleration - How fast can we accelerate? (Higher = Snappier, Lower = Drifty/Heavy) [Kinematic only - owned by the flight component when delegating.]
- BankingAmount - How much do we bank into turns? (Visual only) [Kinematic only.]
- MaxBankingAngle - Max banking angle in degrees (e.g., 45 degrees) [Kinematic only.]
- DecelerationDistance - Distance to start slowing down (Arrival behavior). Still used when delegating - it shapes the guidance target speed near the goal.
- FlightStyle - Choose how this agent moves [Kinematic only - the component's FlightConfig has its own model/style when delegating.]
- AvoidanceTimeHorizon - How many seconds into the future do we look? (Standard: 2.0s)
- CollisionMargin - Radius multiplier for safety (e.g. 1.2 = Avoid by 20% extra margin)
- AvoidanceStyle - Choose how this agent avoids neighbors
- WallCheckDistance - WALL AVOIDANCE How far to check for walls? (e.g. 150 units)
- WallPushForce - How hard to push away from walls? (Must be strong!)
- PathCenteringForce - PATH CORRECTION How hard to pull agents back to the center line? (e.g. 500) 0 = Free drift (current behavior) 1000 = Strict line following (Train on tracks)
- bEnableGoalPrediction - When true, AND r.LcmNav.Chase.Enable != 0, the task resolves moving Object-key goals through FLcmGoalProvider and uses constant-velocity lead prediction (linear-extrapolation intercept) as the planner goal. When false, the legacy snapshot behavior is preserved bit-identically.

- `LeadSecondsCap` - Upper bound on lead time (seconds). Forwarded as `LeadSecondsCap` to `FLcmGoalProvider::GetLeadPosition`. Clamps blow-up when the target is receding faster than the agent can close. Override at runtime via `r.LcmNav.Chase.LeadSecondsCap` (≥ 0). Typical 0.5 to 2.0.
- `DriftReplanCm` - Distance (cm) between the last-planned goal and the current lead position that triggers an automatic replan. Override at runtime via `r.LcmNav.Chase.DriftReplanCm` (≥ 0).
- `bForceEnableChase` - Per-task override for the `r.LcmNav.Chase.Enable` CVar gate. When true, chase activates from `bEnableGoalPrediction` alone - you don't need to know the CVar exists. Default false preserves the legacy two-gate behavior (CVar AND `bEnableGoalPrediction`) bit-identically. The CVar is retained as a project-wide kill switch / debug toggle.

27.1.16 LcmBTTask_FollowFlowField

Task that computes an LCM Nav3D path to the goal and follows its waypoints (LinearDirect steering) until arrival or timeout.

Properties

- `GoalKey` - Blackboard key for the goal. Vector = world location, Object = AActor (its current location is used).
- `MoveSpeed` - Movement speed (cm/s) along the followed path.
- `WaypointAdvanceRadius` - Distance (cm) within which the current waypoint is considered reached and the next is selected.
- `ArrivalRadius` - Distance (cm) from the goal at which the task succeeds.
- `AgentRadius` - Agent collision radius (cm) used by the path probe's clearance gate.
- `MaxDuration` - Maximum time (seconds) to follow the path before the task fails.

27.1.17 LcmBTTask_FollowTarget

Task that holds a 3D offset from a moving target: escort, wingman, overwatch drone. It does not terminate on arrival the way a move task does, because the slot it is holding keeps moving.

The agent matches the target's velocity and corrects the residual offset error on top of that, so a follower that is tracking well sits on its slot rather than trailing behind it by a fixed lag. The target's velocity is measured from its observed motion, so it works for a target moved by any means, including one with no movement component.

The slot is re-checked as the target moves and pulled in toward the target if it would otherwise sit inside geometry: a leader hugging a wall does not drag its escort into that wall.

Properties

- `TargetKey` - Blackboard key for the target to follow. Object = AActor (the useful form), Vector = a fixed point, which reduces the node to a hold.
- `Offset` - The slot to hold, in the frame chosen by `Frame`. Default is 400 cm behind and 200 cm above.
- `Frame` - How `Offset` is interpreted. `World` = fixed world axes, so the slot never rotates. `Leader Rotation` = the offset rotates with the target's actor rotation, for a wingman that stays on one wing through a turn. `Leader Velocity` = the offset is built from the target's direction of travel, which is the right choice when the target's own rotation is not meaningful. The travel frame is held through the target's stops rather than collapsing.
- `MaxSpeedCmS` - The follower's own top speed (cm/s). A target faster than this will out-run the follower.
- `CorrectionGain` - How hard (1/s) the follower corrects offset error on top of matching the target's velocity.
- `SlotDeadbandCm` - Inside this distance the follower stops correcting, so a formation settles instead of hunting.
- `AgentRadiusCm` - Agent radius used when checking the slot for clearance.
- `bRequireLineOfSight` - Require the slot to see the target; a blocked slot is pulled in until it can.
- `MaxDurationSeconds` - Ceiling (seconds) on the escort, after which the task **succeeds**. 0 = follow until something else stops it.
- `bSucceedWhenInFormation` - Succeed once the slot has been held continuously for `FormationHoldSeconds`. Off by default. Useful for a "form up, then advance" sequence.
- `FormationToleranceCm` - How close to the slot counts as in formation.
- `FormationHoldSeconds` - How long the slot must be held before succeeding.

- `bUseFlightMovementComponent` - Route movement through `LcmFlightMovementComponent` when the pawn has one.

27.1.18 LcmBTTask_Hover

Task that holds a position, optionally facing something, optionally for a fixed time.

Hovering is not the same as commanding zero velocity. A body with momentum that is told to stop coasts until drag absorbs it, which for a winged agent can be a long way. This task holds a **point**, so the agent returns to it.

Properties

- `Anchor` - Where It Started holds wherever the agent was when the task began. `Explicit Point` holds the point named by `AnchorKey`.
- `AnchorKey` - Blackboard key giving the anchor when `Anchor` is `Explicit Point`. `Vector` = world location, `Object` = `AActor`.
- `HoverDurationSeconds` - How long to hover before succeeding. 0 = hover until the parent aborts or a decorator exits.
- `AgentRadiusCm` - Agent radius used when checking an authored anchor.
- `FaceTargetKey` - Optional blackboard key to face while hovering. Leave unset to hold rotation.
- `FaceTurnRateDegPerSec` - Turn rate (deg/s) used when facing.
- `bUseFlightMovementComponent` - Route movement through `LcmFlightMovementComponent` when the pawn has one.

27.1.19 LcmBTTask_Land

Task that brings the agent down to a landing point, proving the descent corridor is clear as it goes. `LandTarget` chooses how the landing point is found: an authored world height, the surface directly below, or a search for a flat enough site nearby.

If the SVO is not resident under the agent (a streaming gap), the task waits rather than failing, because landing in the wrong place is worse than pausing.

Properties

- `LandTarget` - How the touchdown point is chosen. `Explicit Z` descends to the authored `TouchDownZ`. `Surface Below` traces down and lands on whatever is actually there. `Landing Site` searches within `SiteSearchRadiusCm` for a site no steeper than `MaxSlopeDegrees`.
- `TouchDownZ` - Target world Z height (cm), used when `LandTarget` is `Explicit Z`.
- `DescentRate` - Vertical descent speed in cm/s.
- `TouchDownTolerance` - Distance (cm) above the touchdown point at which the agent counts as landed.
- `MaxDescentCm` - How far down (cm) to look for a surface. Beyond this the task reports no surface and fails.
- `AgentRadiusCm` - Agent radius used to size the descent corridor test and the site search.
- `SiteSearchRadiusCm` - Radius (cm) searched for a landing site when `LandTarget` is `Landing Site`.
- `MaxSlopeDegrees` - Steepest surface (degrees from horizontal) still accepted as a landing site.
- `MaxDurationSeconds` - Ceiling (seconds) on the whole descent. 0 disables. Guards against a descent that never progresses.
- `bUseFlightMovementComponent` - Route movement through `LcmFlightMovementComponent` when the pawn has one.

27.1.20 LcmBTTask_LookAt

Task that rotates the agent to face a target, easing in, and succeeds once it is pointing within `AcceptableErrorDegrees`.

The alignment test is a true 3D angle, so a target directly above or below the agent is handled correctly. See the note under `LcmBTTask_TurnInPlace` for why this matters and when the engine's stock rotation task is the better choice.

Properties

- **TargetKey** - Blackboard key for what to face. Vector = world location, Object = AActor.
- **InterpSpeed** - Smoothing rate for the eased turn. Higher is faster.
- **TurnRateDegPerSec** - When above 0, turn at this constant rate instead of easing. Takes precedence over **InterpSpeed**.
- **AcceptableErrorDegrees** - Angle within which the agent counts as facing the target.
- **MaxDuration** - Ceiling (seconds) on the turn.
- **bFailOnTimeout** - When true, reaching **MaxDuration** fails the task. Default false, so a turn that runs out of time still succeeds and the tree continues.
- **bRequireLineOfSight** - Require sight of the target. During a streaming gap the task waits rather than failing.
- **bHoldStation** - Hold position while turning, so the agent does not drift during the turn.
- **AgentRadiusCm** - Agent radius used for the line of sight check.
- **bUseFlightMovementComponent** - Route movement through **LcmFlightMovementComponent** when the pawn has one.

27.1.21 LcmBTTTask_MoveToAndFace

Task that moves the agent to a destination and then turns to face something once it arrives. The two halves are one task on purpose, so the turn is guaranteed to follow the arrival.

If no separate face target is set, the agent faces its destination. This is the "arrive at the perch and look at the door" node.

It is not a replacement for **LcmBTTask_FlyTo**: it plans one route on entry and follows it. Use **FlyTo** for long range travel with replanning and avoidance, and this for the last leg.

Properties

- **MoveTargetKey** - Blackboard key for where to move to. Vector = world location, Object = AActor.
- **FaceTargetKey** - Blackboard key for what to face on arrival. Leave unset to face the destination.
- **MoveSpeed** - Movement speed (cm/s).
- **ArrivalRadius** - Distance (cm) within which the destination counts as reached.
- **bNavigateToTarget** - Plan a route to the destination instead of moving in a straight line.
- **MaxMoveSeconds** - Ceiling (seconds) on the move half. Reaching it fails the task.
- **FaceAcceptableErrorDegrees** - Angle within which the agent counts as facing the face target.
- **FaceTurnRateDegPerSec** - Turn rate (deg/s) for the facing half.
- **MaxFaceSeconds** - Ceiling (seconds) on the turn.
- **bFailOnFaceTimeout** - When true, a turn that runs out of time fails the task. Default false, so the move still counts.
- **bRequireLineOfSight** - Require sight of the face target before the turn counts as done.
- **AgentRadiusCm** - Agent radius used for routing and sight checks.
- **bUseFlightMovementComponent** - Route movement through **LcmFlightMovementComponent** when the pawn has one.

27.1.22 LcmBTTTask_OrbitTarget

Task that circles the agent around a target, holding **OrbitRadius** while moving tangentially at **OrbitSpeed**.

Properties

- **TargetKey** - Blackboard key for the orbit centre. Vector = world location, Object = AActor (its current location is used).
- **OrbitRadius** - Distance (cm) the agent maintains from the target while orbiting.
- **OrbitSpeed** - Tangential movement speed (cm/s) around the orbit.
- **bClockwise** - When true, orbit clockwise (viewed from above); when false, counter-clockwise.
- **bMatchTargetZ** - When true, the agent matches the target's world Z height while orbiting; when false, it holds its own height.
- **AgentRadiusCm** - Agent radius used when testing each step of the orbit for clearance.
- **MaxDurationSeconds** - Ceiling (seconds) on the orbit. 0 = orbit until something else stops it.
- **Revolutions** - Stop after this many full turns. 0 = never stop on revolutions.
- **bRequireLineOfSight** - Require sight of the orbit centre. Sight lost for longer than **LostPatienceSeconds** fails the task.

- `LostPatienceSeconds` - How long (seconds) sight of the centre may be lost before the task fails.
- `bUseFlightMovementComponent` - Route movement through `LcmFlightMovementComponent` when the pawn has one.

27.1.23 LcmBTTask_Patrol

Task that moves the agent through a list of waypoints in order, pausing at each, optionally looping. By default it **routes** between waypoints through the navigation data rather than flying the straight line, so a patrol whose waypoints are separated by geometry follows the building instead of passing through it.

While dwelling at a waypoint the agent holds station, so a body with momentum stays on the waypoint rather than drifting off it.

Properties

- `Waypoints` - Ordered list of patrol waypoints, in world coordinates.
- `MoveSpeed` - Movement speed (cm/s) between waypoints.
- `ArrivalRadius` - Distance (cm) within which a waypoint counts as reached.
- `DwellSeconds` - Time (seconds) the agent dwells at each waypoint before advancing.
- `bCycle` - When true, the patrol loops back to the first waypoint after the last; when false, it ends.
- `bNavigateBetweenWaypoints` - Plan a route for each leg instead of moving in a straight line. **Defaults on.** When no route can be planned (for example the navigation data is not resident yet) the leg falls back to the direct line and the next leg is planned again, so a patrol never freezes waiting for data.
- `FaceTravelRateDegPerSec` - Turn rate (deg/s) used to face the direction of travel. 0 leaves rotation alone.
- `MaxDurationSeconds` - Ceiling (seconds) on the whole patrol. 0 = patrol until something else stops it. Worth setting when `bCycle` is on, since a cycling patrol has no natural end.
- `AgentRadiusCm` - Agent radius used when planning legs.
- `bUseFlightMovementComponent` - Route movement through `LcmFlightMovementComponent` when the pawn has one.

27.1.24 LcmBTTask_TakeOff

Task that climbs the agent to a target altitude, proving the air above it is clear as it goes.

The ascent corridor is re-tested while climbing rather than only at the start, because a climb that has been stopped by a ceiling and a climb that is merely slow look identical from the outside. If the SVO is not resident the task waits rather than reporting a blocked climb.

Properties

- `TakeOffTarget` - Height Above Start climbs `TargetHeightCm` above wherever the agent began, which is the natural form for "get off the pad". Absolute Z climbs to the world height in `TargetZ`.
- `TargetHeightCm` - Height (cm) to climb above the start, when `TakeOffTarget` is Height Above Start.
- `TargetZ` - World Z height (cm) to climb to, when `TakeOffTarget` is Absolute Z.
- `AscentRate` - Climb speed in cm/s.
- `ArrivalTolerance` - Distance (cm) below the target height at which the climb counts as done.
- `MaxDurationSeconds` - Ceiling (seconds) on the climb. 0 disables.
- `AgentRadiusCm` - Agent radius used to size the ascent corridor test.
- `bUseFlightMovementComponent` - Route movement through `LcmFlightMovementComponent` when the pawn has one.

27.1.25 LcmBTTask_TurnInPlace

Task that rotates the agent toward a target at a constant rate, without translating, and succeeds once it is pointing within `AcceptableErrorDegrees`.

Why not the stock `UBTTask_RotateToFaceBBEntry`? For a walking agent, use it. For a flying one, note two things about it. Its success test uses a 2D angle that ignores the Z axis entirely, so a target directly above the agent reads as already faced. And it applies rotation through the AIController focus system, which drops pitch unless the pawn opts in with `bUseControllerRotationPitch`. On a volumetric agent that combination

means the stock task can report success while pointing level at a target overhead. This task measures the real 3D angle and applies pitch.

Properties

- **TargetKey** - Blackboard key for what to face. Vector = world location, Object = AActor.
- **MaxRotationRateDegPerSec** - Constant turn rate in deg/s.
- **AcceptableErrorDegrees** - Angle within which the agent counts as facing the target.
- **MaxDuration** - Ceiling (seconds) on the turn.
- **bFailOnTimeout** - When true, reaching **MaxDuration** fails the task. Default false.
- **bRequireLineOfSight** - Require sight of the target. During a streaming gap the task waits rather than failing.
- **bHoldStation** - Hold position while turning.
- **AgentRadiusCm** - Agent radius used for the line of sight check.
- **bUseFlightMovementComponent** - Route movement through **LcmFlightMovementComponent** when the pawn has one.

27.1.26 LcmDynamicObstacleComponent

Marks an actor as a dynamic LCMNav3D obstacle.

Two operating modes, selected by `r.LcmNav.DynObs.EventDriven`:

1. **EVENT-DRIVEN** (default, `EventDriven=1`) -- binds to `RootComponent->TransformUpdated`. Per-event the component unions the new bounding box into a pending dirty region and arms a one-shot coalesce timer (`r.LcmNav.DynObs.CoalesceMs`, default 33 ms = one frame at 30 fps). On timer expiry the union of old-footprint + all coalesced new-footprints is flushed in a single `UpdateDynamicObstacle` call. Component tick is disabled in this mode. Zero CPU cost when the actor is stationary; instant response (within one coalesce window) when it moves.
2. **POLLING** (`EventDriven=0`) -- legacy behavior: `TickComponent` at 10 Hz polls `GetActorLocation` and fires only when displacement exceeds `UpdateDistanceThreshold` (default 50 cm). Provided for deterministic tests where event jitter would perturb traces.

Functions

- **RemoveTrackedComponent** - Runtime/BP removal of a previously added sub-component. Components listed in `TrackedComponents` (the designer list) are not affected.
- **AddTrackedComponent** - Runtime/BP registration of an obstacle sub-component (e.g. a child mesh attached after `BeginPlay`). Idempotent; rebinds immediately.

Properties

- **UpdateDistanceThreshold** - [Polling mode only] How far does the object need to move before triggering an SVO rebuild? Keep around half your `MinVoxelSize` (~50 cm). Ignored when `r.LcmNav.DynObs.EventDriven=1` (event-driven mode uses a coalesce timer instead of a distance threshold).
- **TrackedComponents** - Designer-facing list - pick the collision/mesh components that actually represent this obstacle. Empty = whole actor (legacy).

27.1.27 EnvQueryContext_LcmKnownThreats

EQS context exposing tag-marked threat actors as query targets for tactical tests.

Properties

- **ThreatTag** - Actors carrying this tag are treated as threats.

27.1.28 EnvQueryContext_LcmThreatsFromAIPerception

EQS context exposing the querier's AI-perception-tracked threats as query targets.

27.1.29 EnvQueryGenerator_LcmHemisphericalRing3D

Hemispherical-ring shell of unblocked 3D points around a context.

Properties

- `GenerateAround` - Context the shell is centred on (the target; defaults to the querier).
- `RadiusMin` - Inner radius of the sampling shell (cm).
- `RadiusMax` - Outer radius of the sampling shell (cm).
- `NumRadialSteps` - Number of concentric radial steps between `RadiusMin` and `RadiusMax`.
- `NumInclinationRings` - Number of inclination (polar/elevation) rings from horizon to pole.
- `NumAzimuthSectors` - Number of azimuth (longitudinal) sectors around each ring.
- `bUpperHemisphereOnly` - Restrict to the upper hemisphere (high-ground / perch queries).

27.1.30 `EnvQueryGenerator_LcmPointsAlongPath`

Unblocked points sampled along the LCM Nav3D path from one context to another.

Properties

- `FromContext` - Path start (defaults to the querier).
- `ToContext` - Path goal (set this to your destination context).
- `AgentRadius` - Agent capsule radius used to plan the LCM Nav3D path that is sampled (cm).
- `Spacing` - Distance between consecutive sample points along the path (cm).

27.1.31 `EnvQueryGenerator_LcmPortalProximity`

Candidate points at macro-graph portal centres near a context (chokepoints).

Properties

- `GenerateAround` - Context the portal search is centred on (defaults to the querier).
- `Radius` - Only portals within this radius of the context are emitted (cm).

27.1.32 `EnvQueryGenerator_LcmPointsInSVOVolume`

Generates uniform-density candidate points in an AABB, filtered to unblocked SVO volume leaves.

Properties

- `GenerateAround` - Context the sampling volume is centred on (defaults to the querier).
- `HalfExtent` - Half-extent of the sampling AABB (cm).
- `Spacing` - Grid spacing between candidate points (cm).

27.1.33 `EnvQueryTest_LcmLineOfSight3D`

Visible to the context via true-3D SVO line of sight. Boolean.

Properties

- `Context` - Context to test visibility against (defaults to known threats).
- `AgentRadius` - Agent capsule radius used for the volumetric SVO LOS sweep (cm).

27.1.34 `EnvQueryTest_LcmReachability3D`

An LCM Nav3D path exists from the context to the item. Boolean.

Properties

- `Context` - Context the path is planned from (defaults to the querier).
- `AgentRadius` - Agent capsule radius used when testing for a valid LCM Nav3D path (cm).

27.1.35 `EnvQueryTest_LcmPathDistance3D`

LCM Nav3D path distance (NOT euclidean) from the context to the item. Float (cm).

Properties

- `Context` - Context the path distance is measured from (defaults to the querier).
- `AgentRadius` - Agent capsule radius used to plan the path that is measured (cm).

27.1.36 EnvQueryTest_LcmThreatExposure

Summed SweepPenalty along the item→threat segments. Float (lower = safer).

Properties

- Context - Threat context whose members the exposure is summed against (defaults to known threats).
- NumSamplesPerSegment - Number of sample points evaluated along each item→threat segment.

27.1.37 EnvQueryTest_LcmCoverScore3D

Fraction of context threats the item is occluded from. Float (higher = better cover).

Properties

- Context - Threat context the item must be occluded from to count as cover (defaults to known threats).
- AgentRadius - Agent capsule radius used for the occlusion LOS sweep to each threat (cm).

27.1.38 EnvQueryTest_LcmHeightAdvantage

Vertical advantage of the item over the context (high ground). Float.

Properties

- Context - Context the item's vertical advantage is measured against (defaults to known threats).

27.1.39 LcmFlightMovementComponent

Opt-in pawn movement component that turns nav guidance velocities (RequestVelocity from FlyTo tasks) into realistic 6-DOF flight/swim motion via ULcmFlightDynamics, sweeps the transform, and broadcasts the anim state. Inert until bEnabled is set.

Functions

- RequestVelocity - Feed the guidance command for this frame (world-space desired velocity). Persists until replaced or cleared, so a slower controller tick still produces continuous motion.
- GetRigidState - The full 6-DOF integrator state from the last step.
- GetFlightState - The animation / secondary-motion drive signals from the last step.
- ClearGuidance - Stop consuming guidance and coast (dynamics still decelerate the body).

Properties

- bEnabled - Master opt-in. Default false → the component is inert (base behaviour), guaranteeing byte-identical motion to before it was added.
- FlightConfig - Body/aero/control tuning. The LocomotionModel + Archetype selectors live at the top of this struct and drive which parameters are shown (only the selected model/archetype's fields appear).
- bBroadcastAnimState - Each tick, push the flight state to the pawn's AnimInstance (and the owner) if it implements ILcmFlightAnimInterface. Default on; costs nothing unless an implementer exists.
- RigidState - Persistent integrator state, seeded from the pawn at BeginPlay and re-synced from the updated component each tick (so external teleports and swept-collision corrections are respected).
- FlightState - Last-step anim outputs.
- PendingDesiredVelocity - Latest guidance command and whether one is active.
- bHasGuidance - True when RequestVelocity was called since the last tick - gates the dynamics step so an unfed component coasts instead of integrating stale guidance.

27.1.40 LcmFlyToLocationAsyncAction

The LCM Fly To Location latent Blueprint node: flies a pawn to a point or an actor through LCM Nav3D and fires On Success or On Fail, like AI Move To. No Behavior Tree, StateTree or AI Controller needed. Runs the StateTree Fly To task's own code (FLcmSTTask_FlyTo::EnterFlight / TickFlight / ExitFlight), so the pawn flies exactly as under that task. One active request per pawn; a newer one aborts the older. See Tutorial 13.

Functions

- **FlyToLocation** - Fly a pawn to a location, or to an actor (following it if it moves). Inputs: Pawn, Destination, Target Actor, Acceptance Radius, and advanced Filter Class, Chase Moving Target, Flight Speed, Timeout Seconds.
- **StopFlyTo** - Stop whatever LCM Fly To is flying this pawn (On Fail, Aborted). The pawn holds where it is.
- **IsFlyToActive** - True while an LCM Fly To is flying this pawn.
- **Cancel** - Cancel this request (On Fail, Aborted).

Properties

- **OnSuccess** - Arrived. Carries the result (Success) and where the pawn was.
- **OnFail** - Did not arrive. Carries the result (NoPath, Failed, Aborted, TimedOut, InvalidRequest) and where the pawn was.

27.1.41 LcmFlyingPawn

Note: APawn already inherits from INavAgentInterface - re-listing it here triggers C4584 (duplicate base class). We just override the virtuals.

Properties

- **CollisionSphere** - Root sphere collision component for the flying pawn.
- **MovementComponent** - LCM Nav3D-aware floating movement component driving the pawn along nav paths.

27.1.42 LcmGameplayAbility_RequestPath

GameplayAbility wrapper around the LCM Nav3D async path request, gating it through the standard GAS cost/cooldown/cancellation pipeline (budget via ULcmNavAttributeSet) and broadcasting the resulting path on completion.

Functions

- **OnApexPathFound** - Async FindPathAsync callback (game thread): populates the result, broadcasts OnPathReady, and ends the ability.

Properties

- **StartLocation** - Path-search start.
- **GoalLocation** - Path-search goal.
- **QueryOptions** - Forwarded to FindPathAsync verbatim.
- **BudgetCost** - Cost in PathRequestBudget consumed per activation. Default 1. Reduce for "cheap" replans (e.g. drift-triggered) and raise for expensive cross-chunk queries.
- **OnPathReady** - Fired on completion (success or failure). Subscribers should receive PathPoints by-const-ref.

27.1.43 LcmMassCrowdSpawner

World-Partition-safe Mass spawner: never streams out, so its crowd's lifetime is not tied to the player's distance from the spawn point (fixes the "all agents disappear when the player leaves the area" WP trap).

Properties

- **SpawnBatchPerFrame** - Opt-in startup smoothing. Spawn at most N entities per frame instead of the whole crowd in one frame. 0 (default) = stock engine behaviour. A large one-frame spawn (e.g. 6000 agents) hitches startup ~70 ms; spreading it (e.g. 500) smooths that frame-pacing spike. Only applies when "Auto Spawn On Begin Play" is enabled. NOTE: this does NOT fix the editor's "Handled ensure: CurrentPhase==." Mass phase-manager spam - that is an engine-level startup race (observed on 5.2) (fires with 2 agents, on finite maps, independent of spawn load; handled + compiled out of Shipping), not a spawn-frame cost.

27.1.44 LcmMassFlowSteerTrait

[DEPRECATED - M3, 2026-07-03] Legacy flow-field crowd trait. Superseded by the unified `ULcmMassNavAgentTrait` ("LCM Nav3D Agent") with `NavMode=FlowField`, which composes a byte-identical archetype. Kept fully functional; prefer the unified trait for new content. Authoring trait for a flow-field-steered crowd agent.

Properties

- `GoalMode` - `FixedLocation` (use `Goal`) or `TaggedActor` (chase an actor by tag - the crowd follows it as it moves; the shared field re-solves on the move).
- `Goal` - Shared crowd goal (world space, `FixedLocation` mode). All agents with this goal share one field.
- `TargetActorTag` - Tag of the moving target actor (`TaggedActor` mode).
- `bChaseLead` - Lead the moving target by its velocity (intercept ahead of it).
- `LeadSecondsCap` - Upper bound on lead time (s).
- `MaxSpeed` - Movement speed (cm/s).
- `AgentRadius` - Collision radius (cm) - also the minimum agent size for field generation.
- `AvoidanceRadius` - Neighbours within this distance contribute local avoidance steering (cm).
- `AvoidanceStrength` - Weight of avoidance vs flow-following (0 = off, 1 = balanced).
- `MaxAcceleration` - Max acceleration (cm/s²). Smooth ease in/out; 0 = instant (legacy floaty). Applies only to `PhysicsWeighted` flight (`LinearDirect` is instant).
- `MaxTurnRateDeg` - Max turn rate (deg/s). The key knob: smooth banking turns vs robotic snapping. Applies only to `PhysicsWeighted` flight (`LinearDirect` is instant).
- `ArriveRadius` - Arrival / slow-down radius around the goal (cm).
- `FlightStyle` - `LinearDirect` (instant) vs `PhysicsWeighted` (turn-rate + acceleration inertia).
- `AvoidanceStyle` - `None` / `Reactive` (boids) / `Predictive` (ORCA, smooth mutual passing) / `CBS`.
- `AvoidanceTimeHorizon` - ORCA time horizon (s) - `Predictive` avoidance only.
- `LaneBias` - Lateral lane-spread strength (0 = off). `r.LcmNav.Mass.FlowLaneBias` (>=0) overrides.
- `LaneDensityNorm` - Neighbour count at which lane spreading saturates.
- `bAnticipatory` - `ANTICIPATORY` navigation: route the crowd around where moving obstacles are `HEADING` (their predicted swept path), not just where they are now - so it goes `OVER` a side gap that is about to close instead of squeezing into it. Costs a little more (predicted-occupancy stamping + slightly more re-solving) and can detour a touch wider. Off = today's reactive field. (`r.LcmNav.Mass.Anticipatory` overrides this: 0 forces off, >=1 forces on.)
- `AnticipatoryStrength` - How wide a berth to give predicted obstacle occupancy (higher = earlier / wider detour).

27.1.45 LcmMassGoalTrait

Authoring trait: assign a fixed-location or tagged-actor goal to a Nav3D Agent so it auto-paths there (requires the "LCM Nav3D Agent" trait too).

Properties

- `GoalMode` - `FixedLocation` (use `GoalLocation`) or `TaggedActor` (path to an actor by tag).
- `GoalLocation` - World location goal (`FixedLocation` mode).
- `TargetActorTag` - Tag of the target actor (`TaggedActor` mode). First actor with this tag wins.
- `RepathToleranceCm` - Re-issue the path if the resolved goal drifts more than this (cm).
- `ArrivalToleranceCm` - Treat the agent as "arrived" within this distance; stop re-pathing (cm).
- `ReissueIntervalSeconds` - Minimum seconds between path re-issues per agent (anti-flood safety).
- `bEnableGoalPrediction` - [Chase] Lead a moving `TaggedActor` target by its velocity so agents intercept rather than trail. `TaggedActor` mode only. Master kill switch: `r.LcmNav.Mass.Chase.Enable`.
- `LeadSecondsCap` - [Chase] Upper bound on lead time (s). Override fleet-wide with `r.LcmNav.Mass.Chase.LeadSeconds` (>=0).

27.1.46 LcmMassNavISMRenderTrait

Authoring trait: drop on the Mass config, set a mesh, get instanced visuals.

Properties

- **Mesh** - Mesh drawn (instanced) at each agent's transform.
- **UniformScale** - Uniform scale applied to each instance.

27.1.47 LcmMassNav3DTrait

[DEPRECATED - M3, 2026-07-03] Legacy path-based agent trait. Superseded by the unified `ULcmMassNavAgentTrait` ("LCM Nav3D Agent") with `NavMode=PathFollowing`, which composes a byte-identical archetype. Kept fully functional so existing configs (`DA_MassAgent_Apex`) keep working; prefer the unified trait for new content.

Authoring trait: attaches the path nav fragments and exposes radius/speed, pathfinding, and natural-movement settings.

Properties

- **AgentRadius** - Collision radius forwarded to the solver (cm).
- **MaxSpeed** - Maximum movement speed for the steering processor (cm/s).
- **MobilityClassIndex** - Mobility-class index (reserved for per-class cost rules; 0 = default).
- **AvoidanceRadius** - Neighbours within this distance contribute local avoidance steering (cm).
- **AvoidanceStrength** - Weight of avoidance vs path-following (0 = off, 1 = balanced).
- **PathSolver** - Any-angle LazyThetaStar (smooth) vs grid-locked AStar (square turns).
- **SmoothingMode** - Curved (CatmullRom) / shortcut (Linear) / raw staircase (None).
- **MaxAcceleration** - Max acceleration (cm/s²). Limits speed change so agents ease into motion.
- **MaxTurnRateDeg** - Max turn rate (deg/s). The key knob: replaces robotic square turns with smooth arcs. Lower = wider, lazier turns; 0 = instant snapping (legacy).
- **LookAheadDistance** - Pure-pursuit look-ahead distance (cm). Larger = smoother cornering.
- **ArriveRadius** - Arrival / slow-down radius around the goal (cm).
- **FlightStyle** - LinearDirect (magic/biological - instant) vs PhysicsWeighted (inertia).
- **AvoidanceStyle** - None / Reactive (boids) / Predictive (ORCA, smooth passing) / CBS.
- **AvoidanceTimeHorizon** - ORCA time horizon (s) - used only by Predictive avoidance.
- **LaneBias** - Density-aware lateral lane-spread strength (0 = off).
- **LaneDensityNorm** - Neighbour count at which lane spreading saturates.
- **bObstacleReactivity** - Re-plan when a moving obstacle invalidates the route this agent is following. Global kill switch: `r.LcmNav.Mass.ObstacleReactivity`.

27.1.48 LcmMassNavAgentTrait

[M3] Unified authoring trait: one trait, a `NavMode` selector. `PathFollowing` composes the path archetype (`NavAgent` + `PathRequest` + `PathResult`); `FlowField` composes the flow archetype (`FlowField` state + a const shared config fragment). Mode-specific properties hide via `EditCondition` on `NavMode`.

Properties

- **NavMode** - Movement policy. `PathFollowing` = per-agent A* path; `FlowField` = shared crowd steering field. Drives which fragments + config + tag the trait adds.
- **Compute** - Compute selector governing both modes. This release solves on the CPU, so Auto and CPU behave identically; the selector is retained for forward compatibility.
- **AgentRadius** - Collision radius (cm).
- **MaxSpeed** - Maximum movement speed (cm/s).
- **AvoidanceRadius** - Neighbours within this distance contribute local avoidance steering (cm).
- **AvoidanceStrength** - Weight of avoidance vs path/flow following (0 = off, 1 = balanced).
- **MaxAcceleration** - Max acceleration (cm/s²). 0 = instant.
- **MaxTurnRateDeg** - Max turn rate (deg/s) - replaces robotic square turns with smooth arcs. 0 = instant.
- **ArriveRadius** - Arrival / slow-down radius around the goal (cm). NOTE: the legacy traits defaulted differently per mode (path 80 / flow 150); this unified default is 100 - set explicitly if migrating a config that relied on the old default.
- **FlightStyle** - LinearDirect (instant, no inertia) vs PhysicsWeighted (turn-rate + accel inertia).
- **AvoidanceStyle** - None / Reactive (boids) / Predictive (ORCA) / CBS. NOTE: the legacy Flow Steer trait defaulted to Predictive (ORCA); this unified default is Reactive (matching the legacy path trait) - pick Predictive explicitly for smooth mutual passing in dense flow crowds.
- **AvoidanceTimeHorizon** - ORCA look-ahead horizon (s) - Predictive avoidance only.

- **LaneBias** - Density-aware lateral lane-spread strength (0 = off).
- **LaneDensityNorm** - Neighbour count at which lane spreading saturates.
- **MobilityClassIndex** - Mobility-class index (reserved for per-class cost rules; 0 = default).
- **PathSolver** - Any-angle LazyThetaStar (smooth) vs grid-locked AStar (square turns).
- **SmoothingMode** - Curved (CatmullRom) / shortcut (Linear) / raw staircase (None).
- **LookAheadDistance** - Pure-pursuit look-ahead distance (cm). Larger = smoother cornering.
- **bObstacleReactivity** - Re-plan when a moving obstacle invalidates the route being followed.
- **FlightConfig** - Body / aero / control tuning for the 6-DOF model - the SAME model as the pawn Lcm Flight Movement Component. The LocomotionModel + Archetype selectors are at the top of this struct and drive which parameters show; default LocomotionModel = Kinematic → the crowd moves exactly as before until you opt in.
- **FlightDynamicsLODDistance** - Dynamics LOD distance (cm): beyond this from the viewer, agents use the cheap kinematic integrator (full 6-DOF only near camera) - the scale knob for large crowds. 0 = always full. Kill switch: `r.LcmNav.Mass.FlightLOD`.
- **bAnticipatory** - Route around where moving obstacles are HEADING (predicted swept volume).
- **AnticipatoryStrength** - Detour strength around predicted occupancy (higher = wider berth).
- **bAvoidThreats** - Route the crowd AROUND actors tagged "ApexThreat" (danger zones). NOTE: agents DETOUR around threats, which can slow arrival - measured -30% threat crossings but 13-25% fewer arrivals on tight corridors. Enable for threat/danger gameplay.
- **ThreatDetourStrength** - Threat detour strength (higher = wider berth around danger zones).
- **bIncrementalField** - [Perf] D* Lite INCREMENTAL corridor field for large / churn-heavy crowds (re-cost only changed cells; safe full-rebuild fallback). Shared per goal.
- **bAsyncFlowSolve** - [Perf P2] Solve the flow field on a WORKER thread (removes solve spikes from the frame; the field publishes a frame later). Recommended for large crowds.

27.1.49 LcmMassPerceptionTrait

Authoring trait: makes an LCM Nav3D Mass agent both a sensor and a perceivable (faction-based SVO-LOS perception), exposing sight/FOV/eye-height settings.

Properties

- **FactionId** - Faction id (different factions are mutually hostile / perceivable).
- **SightRadius** - Maximum sight distance (cm).
- **FOVDegrees** - Full field-of-view angle (degrees).
- **EyeHeightOffset** - Eye height offset from the agent origin (cm).
- **AgentRadius** - Agent radius for the SVO LOS walk (cm).
- **PerceptionIntervalFrames** - Perceive once every N frames (LOD throttle).

27.1.50 LcmMassRepresentationTrait

Preconfigured UMassVisualizationTrait: LOD-switched crowd rendering that draws the instanced static mesh at all visible LODs by default (no "see nothing" trap), and self-provides the LOD-collector + actor fragments so it works as one trait.

27.1.51 LcmNavAreaModifier

A cost-area volume (box, sphere or capsule). Every LCM Nav3D planner prices routes through it: travel inside costs the area's Default Cost times the distance, and each entry pays its Fixed Area Entering Cost. An agent's navigation query filter can override either per class, or exclude the class. See Tutorial 10.

Functions

- **NotifyAreaChanged** - Call after changing Area Class, Cost Multiplier, Shape, Priority or the box extent at runtime. Moving the actor needs no call.
- **SetAreaVelocity / ClearAreaVelocity / GetAreaVelocity** - The velocity planners assume for a Movable volume. Only needed when it is teleported, or not attached to the thing that carries it.
- **RegisterWithApex** - Makes the volume visible to planners. BeginPlay calls it when Auto Register is on.
- **GetCostMultiplier** - This volume's own Cost Multiplier (ignores Area Class and filters).
- **GetBounds** - World-space box enclosing the volume's real, rotated shape.

Properties

- **AreaClass** - Stock NavArea class this volume belongs to. Its Default Cost and Fixed Area Entering Cost apply here. Empty = use Cost Multiplier on its own.
- **CostMultiplier** - Used only when Area Class is empty. 1.0 = no effect; 2.0 = twice as dear; 0.5 = preferred lane. Default 2.0.
- **Shape** - Box, Sphere (radius = extent X) or Capsule (along local Z).
- **Priority** - Where volumes overlap, the highest Priority decides the cost. An area the agent's filter excludes always wins.
- **bMovable** - Tick when the volume moves. Planners replan as it moves, and route caches that depend on static areas alone are kept.
- **PredictionSeconds** - For a Movable volume: how far ahead agents plan for its motion. Default 1.5 s.
- **bPatrol**, **PatrolOffset**, **PatrolSpeed** - Move the volume back and forth on its own.
- **BoundingBox** - The volume's extent. Use the editor gizmo to size it.
- **bAutoRegisterWithApex** - Register on BeginPlay. Default on.
- **AreaTag** - Optional designer label.

27.1.52 LcmNavArea_Storm, _PreferredLane, _TollGate, _Territory, _Restricted, _NoFly

Ready-made NavArea classes: x5 travel, x0.5 travel, +2000 to enter, free until a filter prices it, free until a filter excludes it, and impassable for everyone.

27.1.53 LcmNavFilter_Prey, LcmNavFilter_Civilian

Ready-made navigation query filters: Territory x10, and Restricted excluded.

27.1.54 LcmNavAreaQueryLibrary

Blueprint function library exposing the LCM Nav3D nav-extension registry so designers can query cost overlays and registered NavLinks from BP graphs.

Functions

- **GetRegisteredLinkCount** - Count of currently-registered NavLinks.
- **GetRegisteredAreaCount** - Count of currently-registered NavArea modifiers.
- **GetCostMultiplierAt** - Travel multiplier at a world position, under each area's own defaults. Returns 1.0 when no area applies. Where areas overlap, the highest Priority wins.
- **GetCostMultiplierAlongPath** - The route's length-weighted travel multiplier, under each area's own defaults.
- **GetCostMultiplierAtForFilter** - The same at a point, as a given navigation query filter sees it. **bIsExcluded** is true when the filter (or a null area) forbids the point.
- **EvaluatePathCostForFilter** - Prices a waypoint chain as one agent pays for it: length, length weighted by travel cost, entering costs, and whether it crosses an excluded area.
- **GetPathCostBreakdown** - Totals plus one entry per area crossed, for a route and a filter.
- **GetNavLinkTraversalCostAlongPath** - The summed **TraversalCostCm** of the NavLinks a route uses.
- **GetAllLinkEndpoints** - All registered link endpoints as FVector pairs (Start, End). Useful for debug visualisation.
- **FindNearestNavLink** - Finds the nearest registered NavLink to **WorldPos** within **MaxDistanceCm**. Returns **nullptr** if none.

27.1.55 LcmNavAttributeSet

GAS attribute set surfacing per-agent path-request budgets so projects can throttle expensive LCM Nav3D queries through the standard GameplayEffect pipeline.

Functions

- **OnRep_PathRequestBudget** - RepNotify for PathRequestBudget; mirrors the replicated change to GAS.
- **OnRep_MaxPathRequestBudget** - RepNotify for MaxPathRequestBudget; mirrors the replicated change to GAS.

Properties

- `PathRequestBudget` - Current path-request budget. Consumed by 1 per successful `ULcmGameplayAbility_RequestPath` activation. Default 5.
- `MaxPathRequestBudget` - Hard cap clamp for regen. Default 5.

27.1.56 `LcmNavigationInvokerComponent`

Attach this to the Player or VIPs. The SVO Manager will only generate voxel data for chunks within this radius.

Properties

- `GenerationRadius` - How far around this actor should we build navigation data? E.g., 10000 = Build chunks within a 100m radius.

27.1.57 `LcmNavigationManagerSVO`

LCMNav3D world singleton - the central navigation manager actor. Spawns the four subsystem components (`ULcmSVOSTreamingComponent`, `ULcmSVOGenerationComponent`, `ULcmPathfindingComponent`, `ULcmSVORenderComponent`); owns the SVO data containers (`FiniteData` for finite worlds, `ActiveVirtualChunks` for infinite worlds); owns `FLcmMacroGraph` + `FLcmPortalBuilder`; hosts the multicast RPCs for dynamic-obstacle invalidation and flow-field-ready notifications. Exactly one instance per `UWorld`; auto-discovered via `UGameplayStatics::GetActorOfClass`.

Functions

- `UpdateDynamicObstacleReplicated` -----
--- Server-authoritative entry: applies the dynamic-obstacle update locally AND multicasts it to all connected peers (clients + listen-server). Safe to call from any context -- becomes a no-op + warning when called on a client (clients route writes through the server). The flow-field-ready signal is a multicast-only callback path. The flow-field buffers are NOT replicated -- this RPC informs peers that a goal at (`GoalLocation`, `ChunkKey`) is reachable on the server, so they can prewarm their own local generation or trigger gameplay.
- `UpdateDynamicObstacle` - Call this when a large object moves to update the nav graph locally.
- `ToggleDrawDebugVolumes` - Flips `bDrawDebugVolumes` via `SetDrawDebugVolumes`. Convenience entry for a single-key toggle binding.
- `SetMacroTopologyCell` - [Procedural Topology] Called by your procedural-generation system (or World Partition minimap scanner) to mark a coarse cell blocked/unblocked.
- `SetDrawDebugVolumes` - Runtime show/hide for the SVO debug voxel volumes. Sets `bDrawDebugVolumes` and ALWAYS rebuilds the render proxy. The scene proxy captures the flag at build time (`FLcmSVOSceneProxy::bForceDraw`), so a rebuild is required for BOTH show and hide - toggling the bool alone does nothing. Bind to a key for in-game/PIE show/hide.
- `Multicast_UpdateDynamicObstacle` - Server-only entry: multicasts a dynamic-obstacle update to every connected client so each peer's `ALcmNavigationManagerSVO` replays `UpdateDynamicObstacle(Bounds)` locally (RCU clone is identical on every host). Clients that fire this directly are warn-and-ignored.
- `Multicast_FlowFieldReady` - Server-only entry: multicasts a "flow-field finished generating" notification to clients so they can prewarm a local sample for the given goal + chunk. The flow-field buffers are NOT replicated; clients regenerate on demand using shared cache keys.
- `ForceRedrawSVO` - Keep the core function so Blueprints can still call it at runtime if needed
- `FindPath` - Calculates a path from Start to End using the SVO. Returns true for a complete route, and also for one that stops short of the goal (part of the way not streamed in yet).
- `FindPathDetailed` - Find Path that says what it found: Complete, Partial (stops short of the goal; follow it and plan again from its end) or Failed.
- `GetLastPathResult` - Complete, Partial or Failed, for the last Find Path call.
- `GetLastPathCostFailure` - Why the last Find Path failed, if cost areas refused it: Goal In Excluded Area, or No Route Under Filter.
- `SetCostView` - Switches the Cost View at runtime (Off, Regions, Heat).
- `FindCover` - Find a hidden spot safe from the Enemy.
- `DrawDebugSVO` - Visualizes the octree in the viewport (Green = Free, Red = Blocked). One-shot manual call: flushes prior persistent lines and draws with a 10 s lifetime. The per-frame Tick path uses `DrawDebugSVOTick` instead.

- `DrawDebugNeighborsAt` - Visualizes neighbors of the node at the specific location (for debugging connectivity).
- `DrawDebugInfiniteWorld` - Renders the real-time state of the infinite world systems.
- `DebugDrawFlowField` - Flow Field Debug Visualization Generates a temporary Flow Field to the Target-Location and draws it as a 3D Vector Heatmap. Green = Low Cost (Near Target), Red = High Cost (Far from Target).
- `BuildSVOInEditor` - Editor-time SVO build (button in the Details panel). Populates FiniteData from live collision geometry WITHOUT entering PIE, so the stock UE EQS Testing Pawn - and any LCM Nav3D EQS query built from the generators & tests - resolves the SVO and renders true-3D candidate points in the editor viewport, exactly the in-editor workflow Recast navmesh gives for 2.5D EQS. Finite mode only; warns + no-ops while playing (the SVO already bakes on BeginPlay) and in infinite-world mode (which streams chunks at runtime). Press once, then drop an EQSTestingPawn. // Editor-time SVO build. Surfaced as the prominent "Build SVO" button in the // editor module's banner (FLcmNavManagerDetails); BlueprintCallable so it can // also be triggered from script. Intentionally NOT a CallInEditor button - that // rendered a detached row below the property sections instead of in place.
- `BuildSVO` - Clears old data and builds a new SVO from scratch (Finite Mode only).

Properties

- `bIsInfiniteWorld` - World Type. OFF = a single fixed-bounds SVO built for the whole level (best for arenas / linear levels). ON = a streamed, unbounded world using Hierarchical HPA* (chunks generate on-demand around invokers). Drives the Generation Strategy below.
- `GenerationMode` - Auto-derived from the World Type - NOT set by hand. An infinite world always uses the Hierarchical HPA* strategy (macro routing + on-demand chunk streaming), the only infinite mode; a finite world is always Static Finite. Shown read-only so it can never be set to a value that contradicts the World Type (e.g. infinite + Fixed Bounds, which would silently break streaming). Tick "Infinite World" above to switch strategy.
- `bAutoBuildOnPlay` - Should the SVO automatically build itself when the game starts? (Finite mode only)
- `WorldExtent` - The total size of the navigable world (Cube width). Only used for Finite worlds.
- `ChunkSize` - Size of each chunk in World Units (e.g., 50,000 for 500m chunks). Must match World Partition Grid.
- `MinVoxelSize` - The smallest possible voxel size. Recursion stops here. (e.g., 100 = 1 meter)
- `ClearancePadding` - The padding added to voxel collision checks. Set this to your largest agent's radius.
- `ObstacleQueryChannels` - Physics collision channels to treat as solid walls.
- `AllowedObstacleClasses` - If empty, ALL overlapping geometry is voxelized. If populated, ONLY these actor classes are voxelized.
- `CoarseCellSize` - The size of a single topological macro-cell. 2500 units (25 meters) is the AAA standard for coarse LOD pathfinding.
- `bUseFlowFields` - When true, the request classifier can elect the flow-field path instead of per-agent A* for swarms of $\geq \text{FlowFieldThreshold}$ agents sharing a goal. Off by default; opt-in for swarm gameplay.
- `FiniteObstacleUpdateInterval` - [Perf fix, 2026-06-07] FINITE mode: minimum seconds between dynamic- obstacle SVO applications. In finite mode every UpdateDynamicObstacle deep-clones the ENTIRE world SVO (RCU copy-on-write) - with event-driven obstacle components flushing every ~33 ms per moving actor, that was a clone storm ("massive fps drop while anything moves"). Updates arriving inside the window are UNIONED and applied as ONE clone+swap at the trailing edge, so the final state always lands. 0 = legacy immediate behaviour. Infinite mode is unaffected (per-chunk updates are cheap by construction).
- `FlowFieldThreshold` - Minimum number of agents sharing a goal before the classifier elects the flow-field path instead of per-agent A* (requires `bUseFlowFields`).
- `FlowFieldClusterCellSize` - Voxel-quantization grid size (cm) used to bucket flow-field goal positions into cache keys. Two requests whose goals fall into the same cell share a cached flow field. Larger values $\rightarrow$ more sharing (cheaper) but coarser direction snapping.
- `bUseGPUVoxelization` - Uses Compute Shaders for lightning-fast voxelization. Disable only if targeting low-end mobile.

- `MaxGenerationsPerFrame` - How many chunks can we voxelize per frame? (1 is usually best to avoid Game Thread stutters).
- `MaxFlowSolvesPerFrame` - [Perf P3a] Cap goal-anchored flow-field solves STARTED per frame (spreads the GT cost so a burst cannot spike the frame). 0 = unlimited. `r.LcmNav.Mass.MaxCorridorSolvesPerFrame` overrides ($>=0$).
- `MaxPathRequestsPerFrame` - [Perf] Max PathFollowing path requests dispatched per frame (spreads a spawn burst across frames; requests are async). `r.LcmNav.Mass.MaxPathsPerTick` overrides ($>=0$).
- `MaxRequestsPerFrame` - How many paths can we calculate per frame? (50 is a safe number to keep FPS high).
- `bChunkQuantizedMacroCache` - Share one cross-chunk macro route across a crowd heading to the same goal, instead of recomputing the identical chunk-level route per agent. This is the big win for LARGE Mass/AI crowds in infinite/hierarchical worlds: it removes the per-agent macro-pathfinding cost that otherwise spikes the game thread when many agents spawn or replan at once (measured ~30x cheaper request processing, worst-frame 97ms -> 12ms at 2000 agents). The macro route is chunk-level; each agent's exact start/end are still applied per agent, so paths stay correct. Leave OFF to reproduce legacy bit-exact pathfinding (e.g. for deterministic benchmarking). Runtime override: `r.LcmNav.MacroCache.ChunkQuantized` (force-on). Infinite/hierarchical only.
- `bDrawDebugVolumes` - If true, automatically renders the voxel grid when generation finishes.
- `bDrawInfiniteWorldDebug` - If true, draws Invoker Bubbles, the Generation Queue, and loaded chunks in real-time.
- `bDrawPathfindingDebug` - Master switch for Agent Flight Paths and Waypoints.
- `CostView` - Draws the cost areas while playing: Regions, or Heat (the octree tinted by price).
- `CostViewFilter` - Whose eyes the Cost View looks through: a navigation query filter class.
- `DebugRedrawCoalesceSeconds` - Minimum time between debug-mesh redraws (coalesce cadence). Forwarded to the SVO debug renderer; higher = fewer rebuilds when many obstacles move at once.
- `MacroGraph` - A lightweight map of every chunk in the world (HPA*).
- `RenderComponent` - The Central Renderer used to draw the SVO via the GPU Render Thread.
- `DynamicMacroTopology` - [Procedural Topology] Thread-safe map of the global layout of massive blocking geometry (coarse-grid coordinate -> blocked). Populated by `SetMacroTopologyCell`.
- `StreamingComponent` - SUBSYSTEM COMPONENTS
- `GenerationComponent` - SVO voxelization scheduler subsystem. Pulls pending chunk-generation requests off the manager's queue and dispatches CPU/GPU voxelization.
- `PathfindingComponent` - Pathfinding request queue + dispatcher subsystem. Routes async `FindPathAsync` calls to the CPU worker pool, per the request options.

27.1.58 LcmNavLinkComponent

3D vertical NavLink component (subclass of `UNavLinkCustomComponent`) with designer-set world-space `LinkStart/LinkEnd` for ladders, shafts, and teleporters that Recast's surface-projected links cannot represent.

Functions

- `IsVerticalLink` - True iff this link spans more than `MinVerticalCm` along Z - the 3D-advantage case (ladders, jump shafts).
- `GetLinkSpanCm` - Convenience: `distance(LinkStart, LinkEnd)` - useful for designers comparing the link's geometric length vs its travel cost.

Properties

- `LinkStart` - World-space start point. For ladders/teleporters this is the bottom (or originating end if not vertical).
- `LinkEnd` - World-space end point. For vertical ladders this is the top.
- `bBidirectional` - When true (default), the link can be traversed Start→End AND End→Start. Set false for one-way patterns (teleporter, drop-down, etc.).
- `bAutoRegisterWithApex` - Auto-register with the LCM Nav3D link registry on `BeginPlay` so v2 solver integration discovers the link without extra code.
- `TraversalCostCm` - Traversal cost (cm-equivalent) added to the path through this link. Default 0 = no penalty (geometric distance counts as-is).

27.1.59 LcmNavVolume

Defines the playable area for offline SVO generation. Acts as the master bounds for slicing and baking ALcmNavigationChunk actors.

Functions

- **BakeNavigationData** - Editor-only one-shot bake. Slices the volume bounds into the configured chunk grid, voxelizes each chunk on the game thread, and writes a ULcmSVODataAsset per chunk into OutputAssetPath. Pairs with the runtime ALcmSVOSTreamingProxy actor that loads each asset on level streaming.

Properties

- **ChunkSize** - Edge length (cm) of a single bake chunk. Must match the runtime manager's ChunkSize when bSnapToGlobalGrid is true.
- **MinVoxelSize** - Smallest voxel produced by SVO subdivision (cm). Tighter values produce finer paths but inflate the SVO node count cubically.
- **ClearancePadding** - Inflation (cm) applied to obstacle SAT collision checks at bake time. Set to the radius of your largest agent so the SVO carves out enough clearance for them.
- **bSnapToGlobalGrid** - If true, forces chunks to align to a global 3D grid (Required for World Partition). If false, spawns exactly ONE chunk that perfectly matches this volume's bounds (Best for Arena/Linear games).
- **BakeObstacleChannels** - Physics collision channels treated as solid obstacles at bake time. Empty means "all overlapping geometry".
- **BakeAllowedClasses** - Whitelist of actor classes to voxelize. If empty, all overlapping geometry on the configured channels is included.
- **OutputAssetPath** - Content path under which ULcmSVODataAsset outputs are written by BakeNavigationData. Per-chunk assets get a _X_Y_Z suffix.

27.1.60 LcmSmartObjectBehaviorDef_CoverPocket

SmartObject behaviour definition for a designer-placed cover pocket, validated at claim time by SweepPenalty + threat-hint LOS with a FindBestCover fallback when the placed pocket has gone hot.

Properties

- **DwellSeconds** - Time the pawn stays in cover before considering the state complete. 0 = stay forever (use a parent transition condition to exit).
- **MaxAcceptableSweepPenalty** - Highest SweepPenalty (0.255) the slot leaf may carry to count as safe. Above this the StateTree task triggers the FindBestCover fallback. Default 32 = "lightly swept, mostly safe".
- **bUseThreatHint** - Direction hint (world-space) to validate cover against. The pocket is only valid if LCM Nav3D LOS from this point to the slot is blocked AND SweepPenalty is below the cap.
- **ThreatHintLocation** - World-space point the cover is validated against; the pocket is valid only if LCM Nav3D LOS from here to the slot is blocked.
- **FallbackSearchRadiusCm** - When the placed pocket fails validation, the task searches within this radius for a procedural cover via FindBestCover. 0 disables fallback (failure makes the task return Failed).
- **AgentRadius** - Agent radius for the LOS walker's clearance gate.

27.1.61 LcmSmartObjectBehaviorDef_PatrolWaypoint

SmartObject behaviour definition for one patrol waypoint; designers chain placed waypoints via NextWaypointTag to author a patrol route as content.

Properties

- **DwellSeconds** - Dwell time at this waypoint before moving to the next.
- **NextWaypointTag** - GameplayTag of the next smart object in the chain. Empty = end-of- chain; the parent state machine drops out of patrol on Empty.
- **bChainCycles** - When true, the patrol restarts at the chain's head when the chain ends. When false, the pawn lingers at the final waypoint.

- `bUseFacingHint` - Optional facing yaw to assume at this waypoint (degrees). When unset (kept default), keeps the inbound flight rotation.
- `FacingYawDeg` - Yaw (degrees, world-space) the pawn faces at this waypoint when `bUseFacingHint` is set.

27.1.62 `LcmSmartObjectBehaviorDef_Perch`

`SmartObject` behaviour definition for a volumetric perch: fly here, hover or stand, and dwell - a 3D-only smart-object pattern LCM Nav3D's free-space SVO supports where surface-projected Recast navmesh cannot.

Properties

- `DwellSeconds` - How long the pawn dwells at the perch after arrival.
- `RequiredClearanceCm` - Agent collision radius the perch must accommodate. Read at claim time by `FLcmSTTask_UseSmartObject`. **Note:** at production chunk sizes the underlying voxel clearance field is not populated, so this threshold does not currently reject anything. Treat it as authoring intent recorded for future use, not as an active gate, and size perches so that any agent you send to them fits.
- `bUseFacingHint` - When true, the pawn rotates to face `FacingHint` after arrival. When false, the pawn keeps its inbound flight rotation.
- `FacingYawDeg` - Yaw the perched pawn should face (degrees, world-space). Bound only when `bUseFacingHint` is true.
- `FacingInterpSpeed` - Per-tick speed at which the pawn aligns to `FacingYawDeg` during dwell.

27.1.63 `LcmSVODataAsset`

A lightweight, streamable asset that holds pre-calculated SVO navigation data. Designed to be loaded asynchronously by World Partition.

Properties

- `ChunkCoordinate` - The mathematical grid coordinate this chunk belongs to
- `ChunkBounds` - The physical world bounds of this chunk
- `SerializedSVOData` - The raw, compressed binary data of the SVO tree
- `SerializedMacroNode` ----- Optional macro-graph slice for this chunk: the just-finalized `FLcmMacroNode` (`PortalEdges` + `InternalEdges` + `WorldCenter`) captured after `BuildTruePortals` + `PrecomputeIntraChunkEdges` complete during the bake. Serialized via `FLcmMacroNodeArchive` (magic 'LCMN', V1). Empty array = legacy bake - streaming proxy falls back to `UpdateMacroNode` + `GeneratePortalsForChunk` (fallback portals only; the runtime `BuildTruePortals` + intra-chunk-edge precompute still runs separately when neighbors load). Non-empty = the cold-build work is skipped and the precomputed macro node is injected directly into `Manager->MacroGraph.Nodes[ChunkCoordinate]`. Independent from `SerializedSVOData` - backward-compatible addition; older baked assets deserialize with this field empty.

27.1.64 `LcmTacticalQueryLibrary`

Static Blueprint-callable wrappers for LCMNav3D tactical primitives.

All functions are pure CPU and synchronous - they execute on the calling thread (Blueprint VM thread, BT service tick, EQS test evaluation, etc.) and return immediately. None of them dispatch GPU work or async jobs. Designed to be called at EQS-test frequency (tens to hundreds per query per frame) without budget concerns.

Functions

- `IsSegmentLOSClear` - Tests whether a straight line from Start to End is clear of unblocked SVO leaves for an agent of the given radius. Wraps the same Lazy Theta* neighbour-graph LOS walker used by any-angle pathfinding. Returns false if the SVO at Start cannot be resolved (out of bounds or unloaded chunk).

- `IsReachableSync` - Cheap reachability test - returns true iff a path from Start to Goal exists. Implemented by calling the underlying A* with early-out; faster than `FindApexPathSync` when you don't need the waypoint list.
- `IsPositionBlocked` - Returns true iff `WorldPos` resolves to a blocked SVO leaf (or to no leaf at all - out-of-bounds and unloaded chunks both report blocked by default for tactical-query safety). Pair with `GetSweepPenaltyAt` when you need to distinguish "blocked" from "merely risky".
- `GetSweepPenaltyAt` - Reads the `SweepPenalty` byte at `WorldPos`. Higher values indicate the voxel was recently swept by a dynamic obstacle's predicted hull - a heuristic measure of "how dangerous is this position right now?". Returns 0 when the position resolves to a chunk that hasn't been loaded, when the voxel itself is blocked (treated as max danger by default in path costing - caller may want to treat blocked separately via `Is Position Blocked`), or when no dynamic obstacle has touched it. Value range: 0 (clear). 255 (maximum recent threat).
- `FindBestCover` - Wraps `LcmSVOPathSolver::FindBestCover`. Finds the best cover position for Agent-Location against ThreatLocation within SearchRadius (cm), preferring positions with line-of-sight broken to the threat. @param `OutCoverLocation` receives the best cover position when the function returns true. When the function returns false, this is left at `ZeroVector`.
- `FindApexPathSync` - Synchronous "is there a path from Start to Goal" probe. Wraps the same A* core that `FindPathAsync` uses but runs on the calling thread for use inside EQS tests that need to score candidates by LCM Nav3D-path distance (NOT euclidean) before the agent commits. Returns true when a path was found; `OutPath` receives the waypoint sequence. Returns false when start or goal don't resolve to a loaded chunk, when their containing voxels are blocked, or when no path exists between them. Pay-as-you-call cost - typical short-range paths complete in <0.1 ms, long-range cross-chunk paths in 1-5 ms. If you only need the boolean "is reachable?" answer without the waypoint sequence, prefer `IsReachableSync` (cheaper - bails as soon as goal is popped).